

Ingegneria del software

Appunti ragionati per la preparazione all'esame

Indice

1	Modelli di processo di sviluppo software	8
1.1	Il processo di sviluppo	8
1.2	Modelli di processo	9
1.3	Prospettiva storica	10
1.4	Tipi di modelli di processo	11
1.4.1	Code & Fix	11
1.4.2	Modello a cascata (Waterfall)	11
1.4.2.1	Le indicazioni di Royce	11
1.4.2.2	Studio di fattibilità	12
1.4.2.3	Varianti	12
1.4.3	V-Model	13
1.4.4	Modelli evolutivi	13
1.4.4.1	Prototipazione	13
1.4.4.2	Sviluppo iterativo-incrementale	14
1.4.5	Modello a spirale	14
1.4.6	Unified Process (UP/RUP)	15
1.4.7	Component-Based Software Engineering (CBSE)	16
1.4.8	Modello trasformatzionale e metodi formali	16
1.5	Approcci plan-driven e metodi agili	17
1.5.1	Pratiche di Extreme Programming	17
1.5.2	Limiti dei metodi agili	17
1.6	Come scegliere il modello	18
1.7	Riepilogo	19
2	Ingegneria dei requisiti	20
2.0.1	Perché i requisiti sono decisivi	20
2.0.2	Costo della scoperta tardiva	21
2.1	Livelli di dettaglio e rappresentazione	21
2.1.1	Requisiti utente	21
2.1.2	Requisiti di sistema	21
2.1.3	Modi di rappresentazione	22
2.2	Requisiti funzionali e non funzionali	23
2.2.1	Requisiti funzionali	23
2.2.2	Requisiti non funzionali	23
2.3	Qualità e verifica dei requisiti	24
2.3.1	Checklist essenziale	24
2.3.2	Dalla specifica al test	24
3	Use case	25
3.0.1	Utilità nel ciclo di sviluppo	25
3.1	Elementi di un modello di use case	25
3.1.1	Confine del sistema	26
3.1.2	Attori e ruoli	26
3.1.3	Associazioni	26
3.2	Scenari e descrizione testuale	27

3.2.1	Template di uno use case	27
3.2.2	Esempio: prelevare contante	28
3.2.3	Regole per i passi	28
3.3	Deviazioni, ripetizioni e scenari alternativi	29
3.3.1	Deviazioni semplici e complesse	29
3.3.2	Ripetizioni	29
3.3.3	Template di uno scenario alternativo	29
3.4	Relazioni nel diagramma UML	30
3.4.1	Inclusione (include)	30
3.4.2	Estensione (extend)	30
3.4.3	Generalizzazione di attori	31
3.4.4	Generalizzazione di use case	31
3.5	Mockup, qualità della scrittura e limiti	32
3.5.1	Screen mockup	32
3.5.2	Regole di buona scrittura	32
3.5.3	Punti di forza e limiti	32
3.5.4	Riepilogo	32
4	Design architetturale	33
4.0.1	Creatività e riuso	33
4.1	Architettura software e attributi di qualità	33
4.1.1	Perché esplicitare l'architettura	33
4.1.2	Componenti e connettori	33
4.1.3	Impatto sugli attributi di qualità	34
4.2	Stili architetturali strutturali	35
4.2.1	Layered architecture	35
4.2.2	Repository	35
4.2.3	Client-server	36
4.2.3.1	Two-tier e three-tier	36
4.2.4	Peer-to-peer (P2P)	36
4.2.5	Pipe and filter	37
4.3	Stili architetturali di controllo	38
4.3.1	Call-return	38
4.3.2	Manager model	38
4.3.3	Broadcast / publish-subscribe	39
4.3.3.1	Decisioni progettuali negli eventi	39
4.4	Architetture eterogenee	40
4.4.1	Esempio: compilatore	40
4.5	Microservices	41
4.5.1	Proprietà	41
4.5.2	API Gateway e REST	41
4.6	Riepilogo e scelta dello stile	42
5	Design delle componenti (Low-Level Design)	43
5.0.1	Fasi del processo di design	43
5.1	Design by Contract	43
5.1.1	Precondizione, postcondizione e invariante	44
5.1.2	Esempio: conto corrente	44
5.1.3	Responsabilità del cliente e del fornitore	44
5.1.4	Vantaggi	45
5.2	Progettazione degli algoritmi	46
5.2.1	Notazioni e PDL	46
5.2.2	Stepwise refinement	46
5.3	Principi di progettazione	47

5.3.1	Astrazione	47
5.3.2	Decomposizione	47
5.3.3	Modularità e Separation of Concerns	47
5.3.4	Information hiding	48
5.4	Riepilogo	49
6	Introduzione a UML	50
6.1	Modi d'uso e prospettive	50
6.1.1	Sketch, blueprint e linguaggio eseguibile	50
6.1.2	Prospettiva concettuale e software	50
6.1.3	Modello di dominio	51
6.2	Notazione, meta-modello e viste	52
6.2.1	Viste e modello 4+1	52
6.3	I diagrammi di UML 2.5	53
6.3.1	Diagrammi principali nel corso	53
6.4	UML nel processo di sviluppo	54
6.5	Profili, codice e uso pragmatico	55
6.5.1	Profili UML	55
6.5.2	UML e codice	55
6.5.3	Consigli pratici	55
6.6	Riepilogo	56
7	Class diagram e Object diagram UML	57
7.1	Classi, attributi e operazioni	57
7.1.1	Notazione della classe	57
7.1.2	Attributi	57
7.1.3	Molteplicità	57
7.1.4	Operazioni	58
7.1.5	Datatype, costruttori e membri statici	58
7.2	Associazioni e molteplicità	59
7.2.1	Attributo o associazione?	59
7.2.2	Associazioni riflessive	59
7.2.3	Associazioni bidirezionali	59
7.3	Object diagram e classi associative	60
7.3.1	Object diagram	60
7.3.2	Classe associativa	60
7.4	Relazioni tutto-parte e generalizzazione	61
7.4.1	Aggregazione e composizione	61
7.4.2	Generalizzazione ed ereditarietà	61
7.5	Dipendenze e traduzione nel codice	62
7.5.1	Dalle molteplicità alle strutture dati	62
7.6	Riepilogo	63
8	Interfacce e vincoli UML	64
8.1	Interfacce fornite e richieste	64
8.1.1	Realizzazione di un'interfaccia	64
8.1.2	Interfaccia richiesta	64
8.2	Due notazioni equivalenti	65
8.3	Perché usare le interfacce	66
8.4	Vincoli nei Class Diagram	67
8.4.1	Object Constraint Language	67
9	State Machine Diagram	68
9.1	Stato e comportamento	68
9.1.1	Modellazione statica e dinamica	68
9.2	Notazione e semantica	69

9.2.1	Stati, transizioni e pseudo-stati	69
9.2.2	Eventi	69
9.2.3	Run-to-completion	69
9.3	Esempio: ciclo di vita di una copia di libro	70
9.4	Attività interne	71
9.4.1	Entry, exit, do e reazioni interne	71
9.5	Stati composti e concorrenza	72
9.5.1	Stati composti	72
9.5.2	Regioni ortogonali	72
9.6	Quando usare una State Machine	73
9.6.1	Esempio: controllore di un forno	73
9.6.2	Esempio: workflow documentale	73
9.7	Implementazione	74
9.7.1	Switch sullo stato corrente	74
9.7.2	Design Pattern State	74
9.7.3	Tabelle di stato e generazione	74
10	Sequence e Communication Diagram	75
10.1	Fondamenti dei Sequence Diagram	75
10.1.1	Partecipanti, lifeline e attivazioni	75
10.1.2	Messaggi sincroni e asincroni	75
10.1.2.1	Dal diagramma al codice	76
10.2	Ciclo di vita dei partecipanti	77
10.3	Frame di interazione	78
10.3.1	Loop, opt e alt	78
10.3.2	Par, ref e sd	79
10.4	Sequence Diagram di sistema	80
10.5	Sequence Diagram nel design	81
10.5.1	Esempio del registro di cassa	81
10.6	Esempio: prezzo di un ordine	82
10.6.1	Soluzione centralizzata	82
10.6.1.1	Pseudocodice della soluzione centralizzata	83
10.6.2	Soluzione distribuita	84
10.6.2.1	Pseudocodice della soluzione distribuita	85
10.7	Communication Diagram	86
10.7.1	Linee guida	86
11	Activity Diagram	87
11.1	Concetti fondamentali	87
11.1.1	Azioni, nodi e flussi	87
11.1.2	Semantica dei token	87
11.2	Esempio: gestione di un ordine	88
11.2.1	Evoluzione dei token, passo per passo	88
11.3	Nodi di controllo	89
11.3.1	Inizio e terminazione	89
11.3.2	Decisione e fusione	89
11.3.3	Fork e join	89
11.3.4	Nodo finale di flusso	90
11.4	Oggetti, pin e stati	91
11.5	Partizioni e responsabilità	92
11.6	Eventi e interruzioni	93
11.6.1	Invio e ricezione di eventi	93
11.6.2	Regioni interrompibili	94
11.6.3	Parametri e valori di ritorno: fare il bucato	95

11.7	Modellare Use Case e operazioni	96
11.7.1	Use Case ATM: prelevare contante	96
11.7.2	Operazione <code>authenticateUser</code>	97
11.8	Modellare un algoritmo	98
11.8.1	Ricerca binaria	98
12	Component, Deployment e Package Diagram	99
12.1	Component Diagram	99
12.1.1	Concetto di componente	99
12.1.2	Componenti e artefatti	99
12.1.3	Perché componenti e non classi	100
12.1.4	Notazione	100
12.1.5	Interfacce fornite e richieste	100
12.1.6	Porte e vista interna	100
12.1.7	Dipendenze tra componenti	101
12.2	Esempio: sistema di gestione ordini	102
12.3	Deployment Diagram	103
12.3.1	Nodi e connessioni	103
12.3.2	Artefatti e relazione manifest	103
12.3.3	Nodi annidati e tagged value	104
12.4	Esempio: officina meccanica	105
12.4.1	Requisiti degli utenti	105
12.4.2	Possibile architettura hardware	105
12.4.3	Possibile architettura software	105
12.4.4	Integrazione Component e Deployment	106
12.5	Package Diagram	107
12.5.1	Package e namespace	107
12.5.2	Modi di rappresentazione e visibilità	107
12.5.3	Dipendenze tra package	107
12.6	Progettare i package	108
12.6.1	Cicli e dipendenze entranti	108
12.6.2	Esempio di architettura A/B	109
12.6.3	Procedura pratica	109
12.6.4	Risoluzione dei cicli	109
12.6.5	Non soltanto classi	110
13	Software Design Pattern	111
13.1	Progettazione, riuso e cataloghi	111
13.1.1	Perché usare i pattern	111
13.1.2	Il panorama del riuso	111
13.1.3	Libreria e framework	111
13.1.4	GRASP e pattern classici	112
13.1.5	Come si descrive un pattern	113
13.2	Abstract Factory	114
13.2.1	Dalla creazione alla fabbrica	114
13.2.2	Quando usarla	114
13.2.3	Esempio: GUI multiplatforma	114
13.2.4	Conseguenze	114
13.2.5	Struttura d'implementazione	115
13.3	Adapter	116
13.3.1	Problema e scopo	116
13.3.2	Esempio: aggiungere il cerchio	116
13.3.3	Object Adapter e Class Adapter	116
13.4	Facade	117

13.4.1	Un ingresso semplice a un sottosistema	117
13.4.2	Esempi delle slide	117
13.4.3	Conseguenze e confronto con Adapter	117
13.5	Template Method	118
13.5.1	Scheletro e passi variabili	118
13.5.2	Esempio: login riusabile	118
13.5.3	Inversione di controllo e conseguenze	118
13.6	Observer	119
13.6.1	Problema: mantenere viste consistenti	119
13.6.2	Registrazione e notifica	119
13.6.3	Conseguenze	120
13.7	State	121
13.7.1	Comportamento dipendente dallo stato	121
13.7.2	Esempio: distributore automatico	121
13.7.3	Implementazione con il pattern	121
13.7.4	Struttura generale e conseguenze	122
13.8	Quadro riepilogativo	123
14	Software Testing	124
14.1	Terminologia attraverso un caso reale	124
14.1.1	La storia di InvestireOK	124
14.1.2	Debugging del “guru”	124
14.1.3	Quando l’esperto non è disponibile	124
14.2	Errore, fault e failure	125
14.2.1	La catena causale	125
14.2.2	Esempio: reDouble	125
14.2.3	Causa indiretta	125
14.3	Testing e debugging	126
14.3.1	System Under Test e oracolo	126
14.3.2	Differenza dal debugging	126
14.4	Perché il testing è difficile	127
14.4.1	L’impossibilità del testing esaustivo	127
14.4.2	White-box e black-box	127
14.5	Test case e testsuite	128
14.5.1	Definizioni	128
14.5.2	Due esempi	128
14.6	Livelli di testing	129
14.6.1	Unità	129
14.6.2	Integrazione	129
14.6.3	Sistema e accettazione	129
14.6.4	Regressione	129
14.7	Automazione e JUnit 5	130
14.7.1	Dal test manuale al test eseguibile	130
14.7.2	Perché non usare main	130
14.8	Primo caso di test JUnit	131
14.8.1	Classe e metodo di test	131
14.8.2	Esecuzione: verde e rosso	131
14.8.3	Failure ed error nel test runner	131
14.9	Progettare test coesi	132
14.9.1	Una cosa alla volta	132
14.9.2	Fixture: @BeforeEach e @AfterEach	132
14.9.3	Ciclo di vita completo	133
14.10	Asserzioni JUnit	134

14.10.1 Esecuzione e arresto	134
14.10.2 Operazioni principali	134
14.11 Test parametrici	135
14.12 Quadro riepilogativo	136

1 Modelli di processo di sviluppo software

1.1 Il processo di sviluppo

Idea fondamentale

L'ingegneria del software tratta il software come un prodotto industriale: non considera soltanto il codice finale, ma soprattutto il **processo** con cui esso viene concepito, costruito, verificato e mantenuto.

Un **processo di sviluppo software** è un insieme strutturato e organizzato di attività finalizzate alla produzione di un sistema software. Stabilisce:

- **quali attività** devono essere svolte;
- **in quale ordine** devono avvenire;
- quali **regole** e responsabilità governano il lavoro;
- quali risultati intermedi, detti **artefatti**, devono essere prodotti.

Modellare il processo dà ordine, controllo e ripetibilità a un'attività altrimenti difficile da pianificare. Secondo Barry Boehm, un modello determina l'ordine delle fasi, i criteri per passare da una fase alla successiva e aiuta a rispondere a due domande operative: **che cosa fare dopo?** e **per quanto tempo?**

Processo e prodotto

Un processo migliore tende ad aumentare produttività e qualità, ma la relazione non è automatica: un buon processo riduce il rischio di errori sistematici, mentre la qualità finale dipende anche da persone, tecnologia, requisiti e contesto.

1.2 Modelli di processo

Un **modello di processo** è una rappresentazione astratta del processo reale. Non descrive ogni dettaglio operativo: seleziona le attività e le relazioni più importanti per guidare e coordinare il team.

I modelli si collocano lungo uno spettro tra due estremi:

Modello	Regole/pratiche	Grado di prescrittività
RUP	oltre 120	Molto prescrittivo e fortemente documentato
XP	13	Adattivo, ma con pratiche ingegneristiche precise
Scrum	9	Prescrive ruoli, eventi e iterazioni, non le tecniche di sviluppo
Kanban	3	Poche regole e flusso continuo
“Do whatever”	0	Nessuna struttura esplicita

Scrum è meno prescrittivo di XP perché non impone specifiche pratiche ingegneristiche; è però più prescrittivo di Kanban, poiché definisce iterazioni, ruoli ed eventi.

Attenzione alla terminologia

Non esiste una nomenclatura universale: nomi diversi possono indicare lo stesso modello e lo stesso nome può assumere significati diversi secondo autore e contesto. Bisogna verificare sempre le pratiche concrete associate al termine.

La scelta del modello spetta tipicamente a manager e project manager e dipende dal prodotto, dai rischi, dalle competenze, dalla dimensione del team e dai vincoli organizzativi. Non esiste un modello migliore in assoluto.

1.3 Prospettiva storica

Prima del 1965	Produzione artigianale e approccio Code & Fix.
1955–1960	Prime applicazioni pratiche di processi sequenziali in grandi progetti.
1965–1985	“Crisi del software”: costi, ritardi e qualità mostrano i limiti dello sviluppo non disciplinato.
1968	La conferenza NATO consacra il termine <i>software engineering</i> .
1970	Winston Royce descrive formalmente il processo sequenziale poi chiamato Waterfall.
Dal 1975	Si diffondono prototipazione e modelli evolutivi.
1986–1988	V-Model e modello a spirale di Barry Boehm.
Anni Novanta	Web engineering, CBSE, Unified Process ed Extreme Programming.
Dal 2000	Metodi agili, Model-Driven Development e, dal 2009 circa, DevOps.

Fred Brooks, nel saggio *No Silver Bullet*, sostiene che non esiste una singola innovazione capace di eliminare rapidamente la complessità essenziale dello sviluppo software.

1.4 Tipi di modelli di processo

I modelli seguenti rappresentano le principali strategie con cui organizzare lo sviluppo, dalla produzione non strutturata fino ai processi iterativi, guidati dal rischio, basati sul riuso o sui metodi formali.

1.4.1 Code & Fix

Code & Fix è l'approccio più elementare: si comprende il bisogno in modo informale, si scrive il codice, lo si esegue e si correggono gli errori fino alla consegna. Non esistono vere fasi di analisi e progettazione.

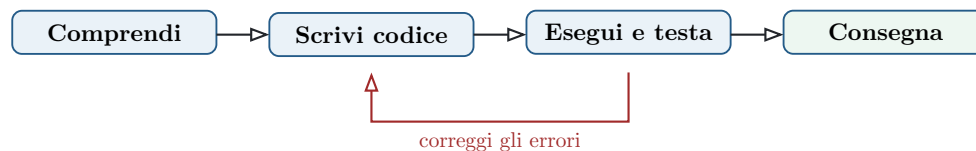


Figura 1: Flusso essenziale del Code & Fix.

È accettabile soltanto per programmi molto piccoli, indicativamente sotto le 1500 righe di codice. Su progetti grandi o multi-sviluppatore, l'assenza di piano, documentazione e controlli rende difficile coordinare il lavoro e governare la qualità.

1.4.2 Modello a cascata (Waterfall)

Il modello a cascata è il primo modello storico strutturato. Fu impiegato in grandi progetti, anche nel settore della difesa statunitense, e formalizzato da Winston Royce nel 1970. Introduce una sequenza di fasi: l'artefatto prodotto da ciascuna fase diventa l'ingresso di quella successiva.

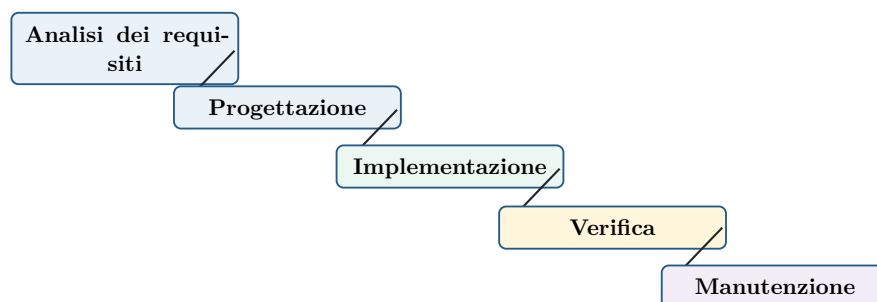


Figura 2: Nel Waterfall il lavoro procede per fasi sequenziali e consegne documentali.

Le fasi fondamentali sono:

1. **Analisi dei requisiti:** raccolta e documentazione delle necessità degli stakeholder.
2. **Progettazione:** definizione dell'architettura e delle soluzioni tecniche.
3. **Implementazione:** traduzione del progetto in codice.
4. **Verifica:** controllo del comportamento del sistema mediante test.
5. **Manutenzione:** correzioni e modifiche successive al rilascio.

1.4.2.1 Le indicazioni di Royce

Royce sottolinea cinque accorgimenti: progettare il programma prima della codifica, mantenere completa e aggiornata la documentazione, realizzare se possibile un prototipo preliminare ("fare il lavoro due volte"), pianificare e monitorare il testing fin dall'inizio e coinvolgere il cliente.

Punti di forza	Limiti
Disciplina, pianificazione ed enfasi su analisi e design.	Rigidità e scarso parallelismo tra le fasi.

Punti di forza	Limiti
Adatto a requisiti stabili e ben compresi.	Il cliente vede tardi il prodotto funzionante.
Artefatti e responsabilità chiaramente identificabili.	Le modifiche tardive sono costose e la documentazione può diventare eccessiva.

1.4.2.2 Studio di fattibilità

Prima dello sviluppo si valuta se il progetto sia conveniente e realizzabile. Lo studio considera costi, tempi, capacità tecniche, ritorno dell'investimento (**ROI**) e alternative come acquistare e personalizzare un prodotto esistente oppure svilupparlo da zero. Il risultato è una decisione **Go/No-go**.

1.4.2.3 Varianti

- **Cascata con prototipazione:** un prototipo “usa e getta” chiarisce requisiti e scelte prima del processo completo.
- **Cascata con feedback:** sono consentiti ritorni alle fasi precedenti quando emergono errori, attenuando la rigidità del modello puro.

1.4.3 V-Model

Il V-Model è una variante del Waterfall che rende esplicita la relazione tra ogni attività di analisi/progettazione e il corrispondente livello di test. Il codice occupa il vertice inferiore della V: a sinistra si scompone il problema, a destra si integra e verifica la soluzione.

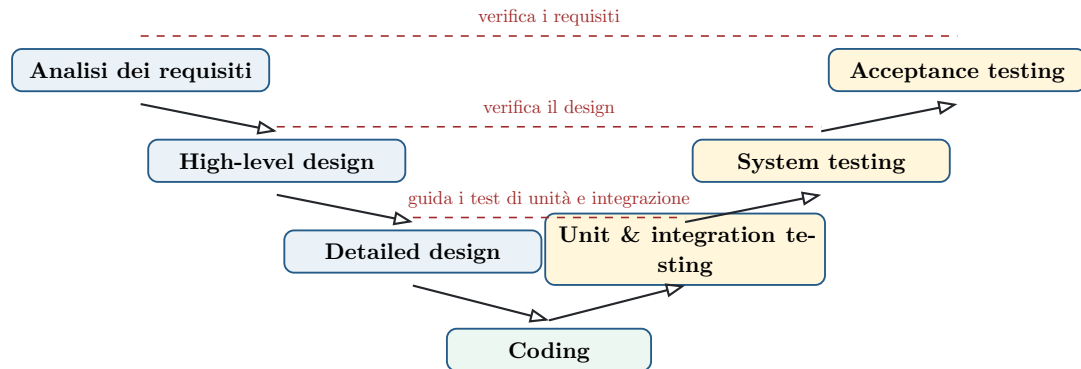


Figura 3: V-Model: ogni specifica preparata sul ramo sinistro è verificata dal test corrispondente sul ramo destro.

Analisi/design	Test corrispondente
Analisi dei requisiti	Acceptance testing con il cliente
High-level design	System testing sul sistema completo
Detailed design	Unit e integration testing

Le relazioni orizzontali hanno due direzioni concettuali. Le specifiche prodotte a sinistra **guidano** la preparazione dei test a destra; quando un test rivela un'incoerenza, il team può dover “mettere mano” alla specifica corrispondente. La preparazione dei test può quindi procedere in parallelo alle attività di analisi e design, anche se la loro esecuzione avviene dopo il coding.

Waterfall e V-Model soffrono quando i requisiti non possono essere congelati, quando il feedback arriva soltanto alla fine o quando errori iniziali si propagano fino alle fasi più costose.

1.4.4 Modelli evolutivi

Idea di base

Il prodotto non viene costruito in un'unica soluzione: si parte da un'implementazione parziale, la si espone agli utenti e la si raffina attraverso release successive.

I tre approcci principali sono prototipazione, sviluppo incrementale e sviluppo iterativo.

1.4.4.1 Prototipazione

Un prototipo è una versione anticipata del sistema, con meno funzionalità o qualità inferiore rispetto al prodotto finale. Può essere:

- **throwaway**: serve a ridurre l'incertezza e viene poi scartato;
- **evolutivo**: viene raffinato fino a diventare il prodotto finale.

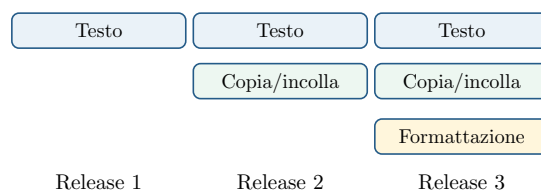
Il ciclo tipico comprende requisiti, quick design, costruzione del prototipo, valutazione del cliente e raffinamento. Conviene prototipizzare soprattutto le parti caratterizzate da requisiti vaghi, rischi tecnici o forti dubbi di usabilità.

Vantaggi	Rischi
Chiarisce requisiti incompleti e migliora la comunicazione.	Un prototipo da scartare ha un costo economico.
Fa emergere presto errori e rischi.	Cliente e team possono essere riluttanti a buttare codice funzionante.
Consente di validare interazione e scelte tecniche.	Scorciatoie e architettura debole possono finire nel prodotto.

1.4.4.2 Sviluppo iterativo-incrementale

Ogni release è pianificata, analizzata, progettata, implementata, testata e valutata dagli utenti. Mentre gli utenti lavorano con una release, il team può costruire la successiva.

Incrementale



Iterativo

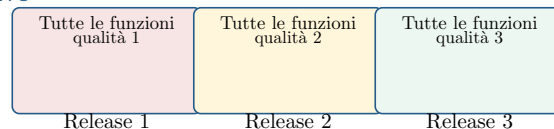


Figura 4: Incrementale: cresce l'insieme delle funzioni. Iterativo: cresce la qualità delle funzioni già presenti.

Nel modello **incrementale** ogni release aggiunge funzionalità senza eliminare quelle precedenti. Nel modello **iterativo** tutte le funzionalità principali sono presenti fin dall'inizio, ma vengono progressivamente migliorate. I due principi sono spesso combinati nello stesso progetto.

Le release possono dare precedenza alle funzionalità a rischio più alto oppure a quelle di maggior valore per l'utente. I vantaggi sono risposta rapida al mercato, feedback continuo, stime più gestibili e scoperta precoce degli errori.

1.4.5 Modello a spirale

Il modello a spirale, proposto da Barry Boehm nel 1988, è pensato per sistemi grandi e complessi. È **risk-driven**: il contenuto di ogni ciclo dipende dai rischi più importanti da affrontare. È anche un meta-modello, perché nel quadrante di engineering può impiegare Waterfall, prototipazione o altri processi.

Rischio

Un rischio è una circostanza sfavorevole che può compromettere progetto o prodotto. La sua esposizione si valuta combinando **probabilità** e **gravità delle conseguenze**: $E(R) = P(R) \times I(R)$.

Esempi sono tecnologie immature, componenti esterni inaffidabili e competenze insufficienti nel team.

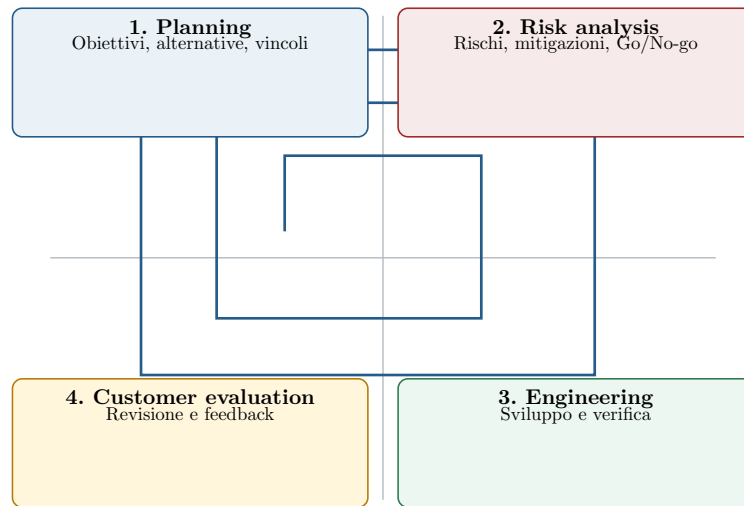


Figura 5: Ogni giro attraversa i quattro quadranti e amplia il prodotto e l’impegno economico.

Il modello affronta presto i problemi critici, ma richiede grande competenza nell’identificazione e nella stima dei rischi. Un rischio importante non riconosciuto può comunque compromettere il progetto.

1.4.6 Unified Process (UP/RUP)

Unified Process nasce dall’unificazione dei metodi di Grady Booch, James Rumbaugh e Ivar Jacobson, i “tre amigos” legati anche alla nascita di UML. La versione commerciale più nota è Rational Unified Process (RUP).

UP è orientato agli oggetti, guidato dai casi d’uso, basato su UML, iterativo e incrementale, attento ai rischi e fortemente plan-driven.

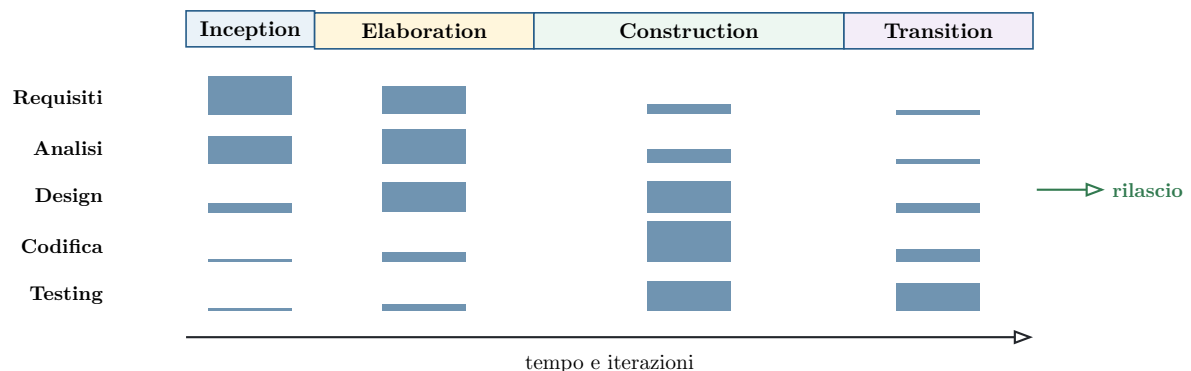


Figura 6: In UP tutte le discipline ricorrono, ma con intensità diversa nelle quattro fasi.

Le quattro fasi sono:

1. **Inception:** fattibilità, requisiti essenziali e rischi principali — “il progetto ha senso?”.
2. **Elaboration:** dominio, casi d’uso e architettura — “sappiamo che cosa costruire e come?”.
3. **Construction:** design di dettaglio, codice e test — “il software funziona?”.
4. **Transition:** deployment, formazione e accettazione — “gli utenti riescono a usarlo?”.

Ogni fase contiene una o più **iterazioni** e termina con una milestone. In ogni iterazione ricorrono requisiti, analisi, design, codifica e testing, ma la loro intensità cambia nel tempo. Il completamento della Transition conduce al rilascio del prodotto agli utenti.

1.4.7 Component-Based Software Engineering (CBSE)

CBSE costruisce sistemi componendo elementi software preesistenti e già testati. Il riuso può ridurre codice, costi e rischi, ma questi benefici dipendono dalla qualità dei componenti e dal costo d'integrazione.

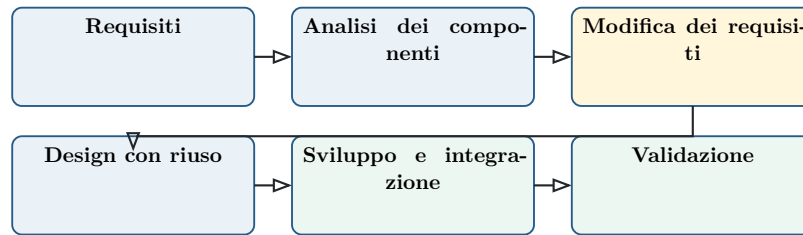


Figura 7: Nel CBSE anche i requisiti vengono negoziati in funzione dei componenti disponibili.

Durante l'analisi si cercano anzitutto componenti già disponibili; se non sono sufficienti, se ne possono acquistare di nuovi oppure sviluppare le parti mancanti. Il passaggio cruciale è la **modifica dei requisiti**: ciò che il cliente desidera deve essere conciliato con ciò che i componenti offrono. I limiti principali sono compromessi funzionali, difficoltà d'integrazione e perdita di controllo sull'evoluzione delle dipendenze esterne.

1.4.8 Modello trasformatore e metodi formali

Il modello trasformatore usa notazioni matematiche, come il linguaggio Z, per passare da requisiti informali a specifiche sempre più vicine all'esecuzione. Ogni trasformazione dovrebbe preservare la correttezza.

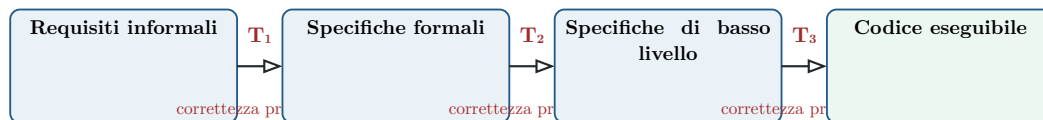


Figura 8: Catena di raffinamenti nel modello trasformatore.

Ogni trasformazione T_i prende una specifica e ne produce una più concreta, senza alterarne il significato previsto. In teoria, prove matematiche sui passaggi possono garantire proprietà molto forti. Il costo e la competenza richiesti ne limitano però l'uso a domini critici. Questo approccio anticipa il moderno Model-Driven Development, in cui modelli formali o semi-formali generano automaticamente parte del codice.

1.5 Approcci plan-driven e metodi agili

Aspetto	Plan-driven	Agile
Pianificazione	Prevalentemente anticipata	Progressiva e adattiva
Requisiti	Stabili e documentati	Evolgono col feedback
Misura del progresso	Piano e artefatti	Software funzionante
Cliente	Coinvolto in momenti definiti	Collaborazione frequente
Documentazione	Estesa e formale	Quanto basta al prodotto e al team
Esempi	Waterfall, V-Model, Spirale, UP	XP, Scrum

Gli approcci plan-driven privilegiano processi definiti, documentazione e controllo formale. Gli approcci agili nascono per rispondere rapidamente al cambiamento e dare priorità a software funzionante e collaborazione col cliente.

1.5.1 Pratiche di Extreme Programming

- iterazioni brevi di due-quattro settimane;
- design semplice secondo il principio KISS;
- refactoring continuo senza cambiare il comportamento osservabile;
- brevi meeting giornalieri;
- conoscenza condivisa soprattutto attraverso il team;
- cliente disponibile e vicino al gruppo di sviluppo.

1.5.2 Limiti dei metodi agili

Sul piano gestionale richiedono persone competenti e motivate; turnover elevato può disperdere conoscenza tacita. Sul piano legale è difficile fissare contratti quando ambito e risultato evolvono. Sul piano della manutenzione, documentazione insufficiente e architettura cresciuta senza controllo possono generare debito tecnico.

1.6 Come scegliere il modello

Preferire un approccio plan-driven quando...	Preferire un approccio agile quando...
Il sistema è grande, complesso o safety-critical.	Il team è piccolo, esperto e co-localizzato.
I requisiti sono stabili e verificabili in anticipo.	I requisiti sono volatili o ancora da scoprire.
Servono tracciabilità, certificazioni e documenti formali.	Gli utenti possono offrire feedback frequenti.
Il lavoro coinvolge molti team distribuiti.	È importante consegnare valore rapidamente.

Regola d'esame

La scelta è contestuale: dimensione, criticità, volatilità dei requisiti, disponibilità del cliente, distribuzione del team e capacità di gestione del rischio contano più dell'etichetta del modello. Nella pratica sono comuni processi ibridi.

1.7 Riepilogo

Primitivo	Code & Fix
Sequenziale	Waterfall, V-Model
Evolutivo	Prototipazione, incrementale, iterativo
Guidato dal rischio	Modello a spirale
Processo unificato	UP / RUP
Basato sul riuso	CBSE
Formale	Modello trasformatzionale
Adattivo	XP, Scrum, Kanban

Il filo conduttore è il compromesso tra pianificazione e adattamento. Più aumenta l'incertezza, più diventano importanti feedback e iterazioni; più aumentano criticità, scala e obblighi di conformità, più servono disciplina, tracciabilità e controllo esplicito.

2 Ingegneria dei requisiti

Un **requisito** descrive una proprietà che il sistema deve possedere: una funzione da offrire, un comportamento osservabile oppure un vincolo da rispettare. Può essere espresso a livelli di astrazione molto diversi, da un obiettivo generale in linguaggio naturale fino a una specifica dettagliata e matematica.

Principio fondamentale

Un requisito descrive principalmente **che cosa** il sistema deve fare, non **come** realizzarlo. Le decisioni implementative appartengono al design, salvo quando tecnologia, piattaforma o standard costituiscono essi stessi un vincolo richiesto.

Per esempio, per un ATM:

- “Il sistema deve controllare la validità della carta inserita” descrive una funzione;
- “Il sistema non deve erogare più di 250 euro per carta in un periodo di 24 ore” esprime una regola operativa misurabile.

2.0.1 Perché i requisiti sono decisivi

Una parte rilevante dei progetti fallisce per problemi che precedono la codifica. I dati riportati negli appunti mostrano che incompletezza dei requisiti e scarso coinvolgimento degli utenti occupano i primi posti.

Motivo di fallimento	Percentuale
Incompletezza dei requisiti	13,1%
Mancato coinvolgimento degli utenti	12,4%
Carenza di risorse	10,6%
Aspettative non realistiche	9,9%
Mancato supporto del management	9,3%
Cambiamento di requisiti e specifiche	8,7%
Carenza di pianificazione	8,1%
Cambiamento di obiettivi	7,5%
Carenza di gestione del settore IT	6,2%
Incompetenza tecnologica	4,3%
Altro	9,9%

2.0.2 Costo della scoperta tardiva

Un difetto introdotto nei requisiti diventa più costoso da correggere quanto più tardi viene scoperto: occorre modificare non soltanto la specifica, ma anche design, codice, test, documentazione ed eventualmente il sistema già distribuito. Validare presto i requisiti è quindi una strategia concreta di riduzione del costo e del rischio, non un adempimento burocratico.

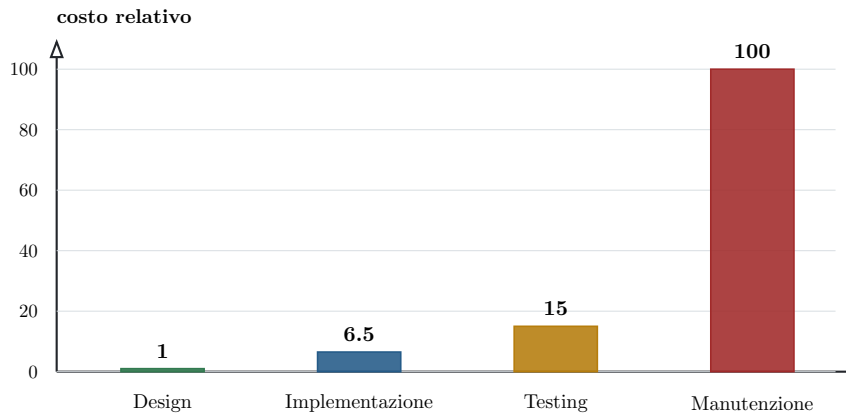


Figura 9: Costo relativo della rimozione di un difetto originato nei requisiti: 1 nel design, 6,5 nell'implementazione, 15 nel testing e 100 in manutenzione.

2.1 Livelli di dettaglio e rappresentazione

La documentazione procede normalmente da una descrizione comprensibile agli stakeholder verso una specifica abbastanza precisa da guidare progettazione, implementazione e test.

2.1.1 Requisiti utente

I **requisiti utente** descrivono in linguaggio naturale le funzionalità offerte e i principali vincoli operativi. Sono scritti per e con il cliente, devono essere comprensibili senza conoscenze tecniche e possono contribuire alla definizione contrattuale del prodotto.

Esempio per un editor grafico di modelli:

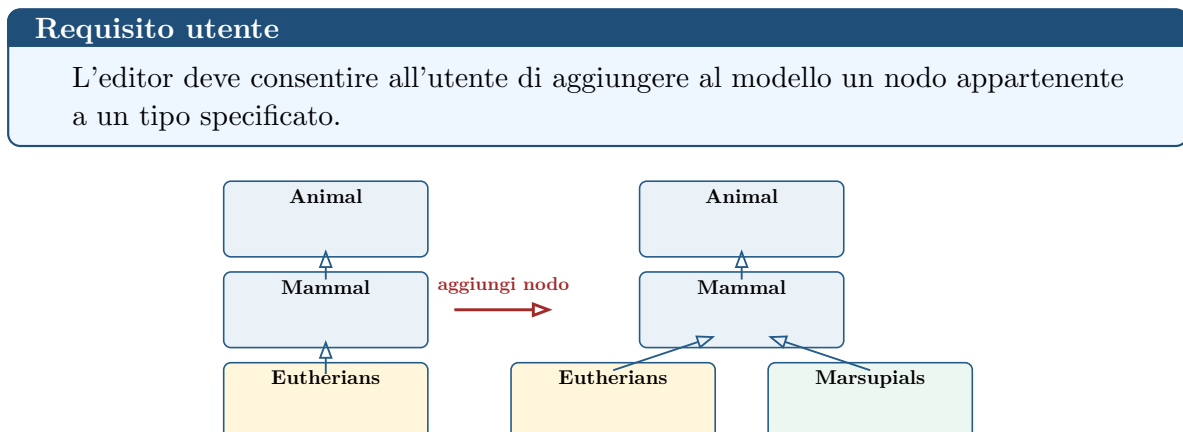


Figura 10: Il requisito utente esprime l'effetto osservabile: aggiungere al design un nuovo nodo e collegarlo correttamente alla gerarchia.

2.1.2 Requisiti di sistema

I **requisiti di sistema** traducono gli obiettivi utente in specifiche dettagliate e verificabili. Possono usare linguaggio naturale strutturato, modelli visuali o linguaggi formali. Nell'esempio precedente la funzione viene precisata così:

1. l'utente seleziona il tipo di nodo da aggiungere;
2. sposta il cursore nella posizione approssimativa del diagramma;
3. conferma l'inserimento del simbolo in quel punto.

Funzione	Aggiunta di un nodo
Descrizione	Aggiunge un nodo a un design esistente; l'utente ne sceglie tipo e posizione. Il nodo aggiunto diventa la selezione corrente.
Input	Tipo del nodo, posizione, identificatore del design.
Origine	Tipo e posizione dall'utente; identificatore dal database.
Output	Identificatore del design aggiornato.
Destinazione	Database dei design, aggiornato al completamento.
Richiede	Grafo del design associato all'identificatore ricevuto.
Precondizione	Il design è aperto e visualizzato.
Postcondizione	Il design è invariato salvo l'aggiunta del nodo specificato nella posizione scelta.
Effetti collaterali	Nessuno.

Dal requisito utente al requisito di sistema

Il primo comunica l'obiettivo agli stakeholder; il secondo elimina progressivamente ambiguità e fornisce dettagli sufficienti per costruire casi di test. I due livelli non sono concorrenti: servono destinatari diversi.

2.1.3 Modi di rappresentazione

I requisiti possono essere rappresentati con strumenti diversi, scelti in base a precisione richiesta, destinatari e costo di formalizzazione.

Forma	Caratteristiche
Nessuna specifica	Si passa direttamente all'implementazione: rapido ma rischioso e adatto soltanto a casi minimi.
Linguaggio naturale	Accessibile, ma può risultare ambiguo, incompleto o incoerente.
Testo strutturato	Template con campi obbligatori, precondizioni, input, output e postcondizioni.
Notazioni visuali	Diagrammi UML e altri modelli che rendono evidenti struttura e relazioni.
Use case	Descrivono le interazioni tra attori e sistema per raggiungere un obiettivo.
User story	Esprimono sinteticamente bisogno, soggetto e valore atteso.
Linguaggi formali	Notazioni matematiche, come Z, precise ma costose e specialistiche.

2.2 Requisiti funzionali e non funzionali

La distinzione fondamentale riguarda ciò che il sistema offre e le qualità o i vincoli entro cui deve operare.

2.2.1 Requisiti funzionali

I requisiti funzionali descrivono **funzioni e servizi**: ciò che il sistema deve fare, quali input tratta, quali risultati produce e come reagisce a determinate condizioni. Dovrebbero essere indipendenti dalla particolare soluzione implementativa.

Esempi per un sistema bancario:

- consentire la consultazione del saldo;
- permettere prelievo e produzione dell'estratto conto;
- richiedere, per il prelievo, l'autenticazione tramite un codice segreto memorizzato sulla carta.

2.2.2 Requisiti non funzionali

I requisiti non funzionali definiscono proprietà di qualità e vincoli sul prodotto o sul suo processo di sviluppo: prestazioni, affidabilità, usabilità, piattaforme, standard, formati, aspetti etici e obblighi legislativi.

Esempi:

- rispondere a un'interrogazione entro 3 secondi;
- produrre documenti in formato PDF;
- essere accessibile a persone con disabilità;
- funzionare su una piattaforma stabilita dall'organizzazione;
- essere facilmente espandibile per esigenze future.

Nell'esempio ATM, offrire prelievo, saldo ed estratto conto e autenticare il prelievo mediante codice segreto sono requisiti **funzionali**. Accessibilità, tempo di risposta, piattaforma ed espandibilità sono invece requisiti **non funzionali**.

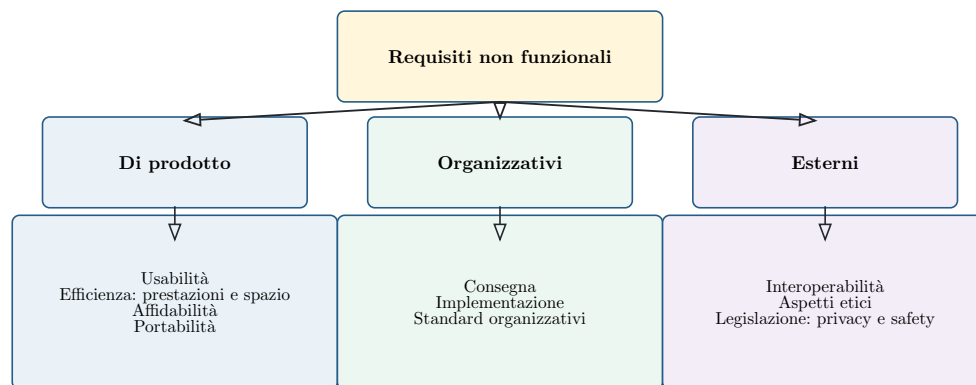


Figura 11: Tassonomia dei requisiti non funzionali: proprietà del prodotto, vincoli organizzativi e vincoli esterni.

Di prodotto	Qualità osservabili nel sistema: usabilità, efficienza, affidabilità e portabilità. L'efficienza comprende prestazioni e memoria occupata.
Organizzativi	Vincoli del processo e delle politiche aziendali: consegna, modalità d'implementazione, linguaggi, strumenti e standard.
Esterni	Vincoli del contesto: interoperabilità, principi etici, leggi, privacy e safety.

2.3 Qualità e verifica dei requisiti

Una checklist aiuta a controllare la specifica prima che gli errori si propaghino nelle fasi successive. Ogni requisito dovrebbe poter essere letto come un'affermazione controllabile, non come un'intenzione vaga.

2.3.1 Checklist essenziale

Necessario	Risponde a un bisogno reale di uno stakeholder o a un vincolo effettivo?
Chiaro	Ha una sola interpretazione ragionevole e usa termini definiti?
Completo	Specifica condizioni, soggetti, comportamento e risultato necessari?
Coerente	Non contraddice altri requisiti o regole del dominio?
Fattibile	Può essere realizzato con tecnologia, tempo e risorse disponibili?
Verificabile	È possibile dimostrare con test, misura, ispezione o analisi che è soddisfatto?
Tracciabile	Sono identificabili origine, motivazione e artefatti che lo implementano?
Prioritizzato	Importanza e urgenza sono note agli stakeholder?
Indipendente dalla soluzione	Evita dettagli tecnici non richiesti come veri vincoli?

Regola d'esame

Parole come “rapido”, “facile”, “adeguato” o “user-friendly” non rendono un requisito verificabile. Occorre sostituirle con criteri osservabili: per esempio “il 95% delle richieste deve ricevere risposta entro 3 secondi”.

2.3.2 Dalla specifica al test

Un requisito ben formulato anticipa il criterio con cui sarà accettato. Precondizioni, input, comportamento atteso, output e postcondizioni costituiscono quindi un ponte tra requirements engineering e testing.

Sintesi

I requisiti efficaci sono comprensibili agli stakeholder, sufficientemente precisi per gli sviluppatori e verificabili dai tester. La loro qualità determina la qualità delle decisioni prese in tutte le fasi successive.

3 Use case

Gli **use case** (casi d'uso), proposti da Ivar Jacobson nel 1992, esprimono i requisiti funzionali dal punto di vista di chi usa il sistema. Descrivono una storia di interazione orientata a uno scopo, senza entrare nella struttura interna o nella tecnologia impiegata.

Il “sistema” può essere un'organizzazione complessa, un prodotto software/hardware, una componente o un sottosistema. In ogni caso viene osservato come una **black box**: interessano i messaggi scambiati con l'esterno e il risultato visibile, non il modo in cui è costruito.



Figura 12: Un use case descrive l'interazione tra attori esterni e sistema attraverso uno scambio ordinato di messaggi.

Use case e diagramma UML

Lo use case è prima di tutto una **tecnica testuale**, indipendente dall'orientamento agli oggetti e da UML. Lo **Use Case Diagram** è invece una rappresentazione visuale UML che riassume attori, casi d'uso, relazioni e confine del sistema; non sostituisce la descrizione testuale.

3.0.1 Utilità nel ciclo di sviluppo

Uno use case collega il bisogno degli stakeholder agli artefatti successivi:

Requisiti	Esprime un obiettivo funzionale dal punto di vista dell'utilizzatore.
Design	Offre un punto di partenza per individuare responsabilità e componenti, senza imporre direttamente le classi.
Testing	Genera scenari e condizioni da trasformare in casi di prova.
Planning	Aiuta a raggruppare funzionalità e pianificare unità di rilascio.

3.1 Elementi di un modello di use case

Un modello completo combina quattro elementi: attori, casi d'uso, associazioni e confine del sistema.

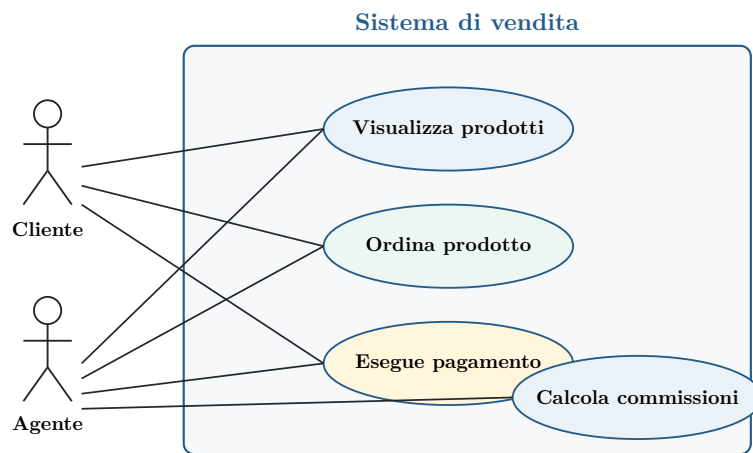


Figura 13: Esempio di Use Case Diagram: gli attori restano fuori dal confine, mentre i casi d'uso appartengono al sistema.

3.1.1 Confine del sistema

Il rettangolo di sistema stabilisce che cosa deve essere costruito e che cosa esiste già nell'ambiente. Il suo posizionamento modifica direttamente i requisiti funzionali: una responsabilità esterna non va specificata come comportamento interno e viceversa.

Errore frequente

Un confine ambiguo produce sovrapposizioni, funzionalità mancanti e aspettative incompatibili. Prima di descrivere gli use case bisogna chiarire sistemi esterni, persone, dispositivi e responsabilità organizzative.

3.1.2 Attori e ruoli

Un **attore** è il ruolo assunto da un'entità esterna durante l'interazione. Può essere una persona, un altro sistema o un dispositivo hardware. Non coincide con l'entità concreta: la stessa persona può agire, in momenti diversi, come cliente e amministratore; molte persone possono ricoprire lo stesso ruolo.

Attore primario	Persegue un obiettivo e ottiene valore dal sistema; normalmente avvia il caso d'uso. Esempio: il cliente che ordina un prodotto.
Attore secondario	Fornisce al sistema un servizio necessario. Esempio: il servizio esterno che autorizza un pagamento.

3.1.3 Associazioni

Una linea tra attore e caso d'uso indica **partecipazione**, non un flusso dati né necessariamente chi avvia l'interazione. Le associazioni rispondono alla domanda: "quali ruoli collaborano alla realizzazione di questo obiettivo?".

3.2 Scenari e descrizione testuale

Uno **scenario** è una sequenza ordinata di interazioni tra attori e sistema. Rappresenta una particolare esecuzione, cioè un singolo cammino, di uno use case.

Use case e scenario

Uno use case raccoglie tutti gli scenari che condividono lo stesso obiettivo finale. Per “prelevare contante” esistono il percorso di successo e percorsi alternativi: PIN errato, fondi insufficienti, ATM senza contante, richiesta di ricevuta o annullamento.

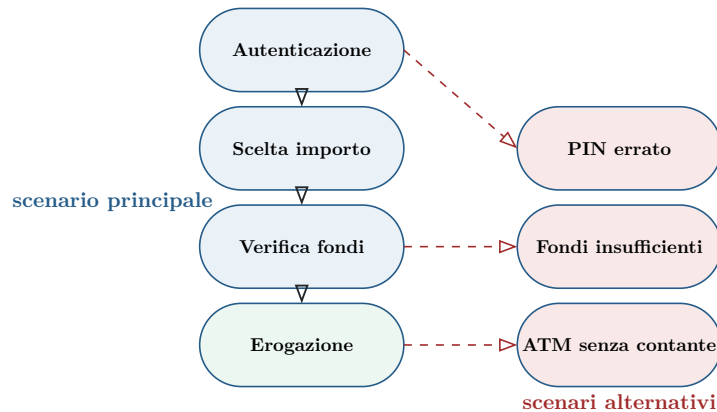


Figura 14: Uno use case comprende un percorso principale e più deviazioni o scenari alternativi.

Gli scenari concreti sono utili per scoprire i requisiti e per progettare i test; generalizzando più scenari si ottiene la descrizione complessiva dello use case.

3.2.1 Template di uno use case

Gli use case sono normalmente scritti con un template, usando il vocabolario del dominio e un linguaggio comprensibile al cliente.

Nome	Breve frase verbale attiva che esprime l’obiettivo, per esempio “Preleva contante”.
Identificatore	Codice univoco, spesso numerico e progressivo.
Breve descrizione	Paragrafo che fissa lo scopo e il risultato atteso.
Attori primari	Ruoli che perseguono l’obiettivo.
Attori secondari	Ruoli o sistemi che forniscono servizi.
Precondizioni	Proprietà che devono essere vere prima dell’avvio.
Scenario principale	Sequenza nominale di passi che conduce al successo.
Postcondizioni	Proprietà vere dopo il completamento corretto.
Scenari alternativi	Elenco delle deviazioni documentate separatamente.

3.2.2 Esempio: prelevare contante

Caso d'uso: Preleva contante **Attore primario:** Cliente **Precondizioni:** collegamento bancario attivo; ATM disponibile e dotato di contante.

Scenario principale

1. Il cliente inserisce la carta.
2. Il sistema legge la carta e chiede il PIN.
3. Il cliente inserisce il PIN.
4. Il sistema convalida il PIN e mostra le operazioni disponibili.
5. Il cliente seleziona "Prelievo" e indica l'importo.
6. Il sistema verifica fondi e limiti giornalieri.
7. Il sistema autorizza l'operazione, aggiorna il conto ed eroga il contante.
8. Il sistema restituisce la carta e, se richiesta, stampa la ricevuta.

Postcondizione: il conto riflette il prelievo e la sessione è conclusa.

3.2.3 Regole per i passi

I passi devono essere concisi, numerati e ordinati temporalmente. Il formato consigliato è: "<numero> Il <soggetto> <azione>". Ogni passo attribuisce un comportamento osservabile al sistema oppure a un attore.

Livello corretto di dettaglio

Scrivere "Il cliente preme il pulsante OK" congela una scelta di interfaccia. È preferibile "Il cliente conferma l'ordine" oppure "Il sistema chiede al cliente di confermare l'ordine".

3.3 Deviazioni, ripetizioni e scenari alternativi

Una **deviazione** si verifica quando l'esecuzione si allontana dal percorso principale.

3.3.1 Deviazioni semplici e complesse

- Una deviazione breve può essere espressa inline con **se/altrimenti**.
- Una deviazione articolata, tipicamente un errore o un caso particolare che non rientra subito nel percorso principale, va documentata come scenario alternativo separato.

Estratto: Trova libro

1. Il cliente seleziona “Trova libro”.
2. Il sistema richiede i criteri di ricerca.
3. Il cliente inserisce i criteri e conferma.
4. Il sistema ricerca i libri corrispondenti.
5. **Se** trova uno o più libri, mostra l'elenco.
6. **Altrimenti**, comunica che non sono stati trovati risultati.

3.3.2 Ripetizioni

Le ripetizioni devono essere esplicite:

- **Per ogni** esprime un'iterazione su un insieme: per ogni libro trovato, il sistema mostra immagine, caratteristiche e prezzo.
- **Fintantoché** esprime un ciclo condizionato: fintantoché i criteri non sono validi, il sistema li richiede nuovamente e li convalida.

3.3.3 Template di uno scenario alternativo

Uno scenario alternativo possiede un identificatore derivato dallo use case principale (per esempio 9.1), breve descrizione, attori, precondizioni, sequenza e postcondizioni. Il primo passo specifica il punto d'innesto:

- “inizia dopo il passo X” quando può attivarsi soltanto in un punto preciso;
- “inizia in qualunque momento” per eventi sempre disponibili, come l'annullamento.

Nel template principale si elencano soltanto i nomi delle alternative, per esempio PINNonValido, FondiInsufficienti e Annulla. Identificatore e punto d'innesto rendono esplicita la relazione con il flusso principale.

3.4 Relazioni nel diagramma UML

Le relazioni strutturano il modello e riducono duplicazioni. Devono essere usate quando migliorano la comprensione, non come decorazione.

3.4.1 Inclusione (**include**)

L'inclusione estrae un comportamento obbligatorio e spesso comune a più use case. Il caso principale trasferisce il controllo al caso incluso e lo riprende al termine, in modo simile a una chiamata di procedura.

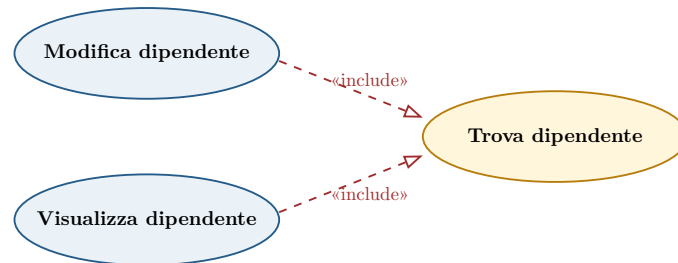


Figura 15: Trova dipendente è un comportamento comune incluso obbligatoriamente dai casi principali.

Semantica dell'include

Il caso principale è incompleto senza il caso incluso. Il caso incluso può essere un obiettivo autonomo oppure soltanto un frammento riutilizzabile.

3.4.2 Estensione (**extend**)

L'estensione aggiunge un comportamento opzionale a un caso base. Il caso base dichiara uno o più **extension point**, ma resta completo e comprensibile anche senza conoscere le estensioni.

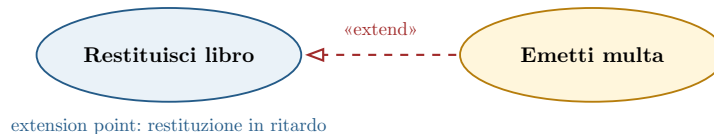


Figura 16: Emetti multa estende opzionalmente Restituisci libro quando la restituzione è in ritardo.

include	extend
Comportamento sempre richiesto.	Comportamento condizionale o opzionale.
Il principale dipende dall'incluso.	Il caso base è completo da solo.
La freccia punta al caso incluso.	La freccia punta al caso base.

3.4.3 Generalizzazione di attori

Un attore specializzato eredita tutte le associazioni dell'attore generale. Per esempio **Cliente** e **Agente** possono specializzare **Acquirente**, evitando di ripetere le relazioni comuni.

3.4.4 Generalizzazione di use case

Un caso figlio rappresenta una variante più specifica del genitore: eredita il comportamento, può aggiungere passi e può ridefinire quelli ereditati.

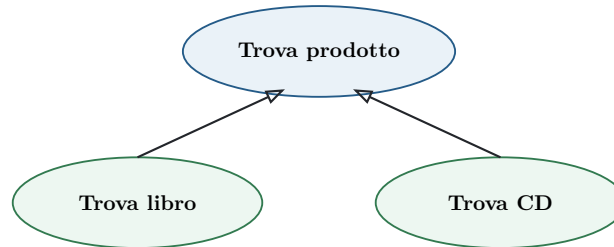


Figura 17: I casi specializzati ereditano l'obiettivo e il comportamento comune del caso generale.

Cautela

Le convenzioni per indicare passi ereditati, aggiunti e ridefiniti possono essere difficili per stakeholder non tecnici. Generalizzazione ed estensione vanno introdotte soltanto se rendono il modello più semplice.

3.5 Mockup, qualità della scrittura e limiti

3.5.1 Screen mockup

Uno use case può essere accompagnato da uno **screen mockup**: uno schizzo dell'interfaccia in uno specifico passo dello scenario. È utile nella negoziazione col cliente, guida lo sviluppatore e chiarisce le decisioni di UI, ma richiede tempo per essere prodotto e mantenuto.

I risultati sperimentali riportati negli appunti attribuiscono ai mockup un miglioramento del 69% nella comprensione degli use case e dell'89% nell'efficienza dei task di comprensione. Possono essere realizzati su carta o con strumenti di prototipazione come Pencil, usando stencil grafici per widget desktop, web e mobile.

3.5.2 Regole di buona scrittura

- mantenere scenario principale e use case brevi e semplici, idealmente entro una pagina A4;
- scegliere un livello intermedio: né troppo astratto né legato a dettagli inutili;
- descrivere interazioni osservabili, non funzionalità interne o algoritmi;
- usare il vocabolario del dominio e frasi comprensibili agli stakeholder;
- distinguere sempre il **cosa** dal **come**;
- evitare la decomposizione funzionale in casi astratti privi di valore autonomo;
- usare **include**, **extend** e generalizzazione soltanto quando semplificano davvero.

Principio guida

Gli use case sono documenti destinati a esseri umani. Precisione e struttura devono migliorare la comprensione, non trasformare la specifica in un esercizio di notazione UML.

3.5.3 Punti di forza e limiti

Punti di forza	Limiti
Esprimono requisiti funzionali in forma narrativa e verificabile.	Catturano male requisiti non funzionali trasversali.
Sono diffusi nei sistemi informativi e nello Unified Process.	Sono meno efficaci in sistemi dominati da algoritmi o con interazione esterna limitata.
Guidano design, pianificazione dei rilasci e testing.	Diagrammi senza specifiche testuali restano superficiali.
Template diversi mantengono una struttura sostanzialmente equivalente.	Un livello di dettaglio errato produce testi vaghi o vincolati prematuramente alla UI.

Regola d'esame

Uno use case non coincide con una funzione interna e non coincide con una singola schermata: descrive un obiettivo dell'attore realizzato attraverso più interazioni osservabili.

3.5.4 Riepilogo

Use case, scenari e diagrammi hanno ruoli complementari: il caso d'uso raccoglie i percorsi orientati a un obiettivo; lo scenario rappresenta un singolo percorso; il diagramma UML offre una mappa sintetica di attori, obiettivi e relazioni. La specifica testuale resta la fonte principale del comportamento richiesto.

4 Design architetturale

Il **design** è il processo che trasforma un problema in una soluzione. I requisiti definiscono il **cosa**; il design sceglie il **come** e struttura la soluzione; l'implementazione rende quella struttura eseguibile e utilizzabile.

L'analogia edilizia è utile: le richieste dei proprietari corrispondono ai requisiti, il progetto dell'architetto al design e i lavori di costruzione all'implementazione.

Due livelli di raffinamento

L'**architectural design** o high-level design identifica architettura, sottosistemi e comunicazioni. Il **component design** o low-level design dettaglia componenti, dati, algoritmi e tecnologie, passando eventualmente da una soluzione indipendente dalla piattaforma a una specifica per .NET, Jakarta EE o altri ambienti.

4.0.1 Creatività e riuso

Il design non è un'attività puramente creativa. Gran parte dei problemi viene risolta adattando conoscenza esistente:

Clonazione	Riuso quasi completo di design o codice, con modifiche limitate.
Design pattern	Soluzione generale a un problema ricorrente, applicabile in domini diversi.
Stile architetturale	Organizzazione generica di componenti e connettori, spesso ottimizzata per una famiglia di sistemi.

Buoni progetti precedenti, sistemi simili, modelli di riferimento, principi, convenzioni e pattern alimentano le decisioni del progettista. Studiare esempi riusciti migliora quindi la capacità di design.

4.1 Architettura software e attributi di qualità

Il **design architetturale** identifica le macro-componenti — moduli, sottosistemi, servizi o classi — e il framework di controllo e comunicazione che le collega. Il risultato è una descrizione dell'architettura software. Mary Shaw e David Garlan hanno contribuito a formalizzare questo campo negli anni Novanta.

4.1.1 Perché esplicitare l'architettura

- guida lo sviluppo e permette di affrontare componenti più semplici di un monolite;
- documenta responsabilità e dipendenze;
- supporta decisioni tecniche e manageriali;
- consente di analizzare proprietà come efficienza e manutenibilità;
- favorisce riuso su larga scala ed evoluzione controllata.

4.1.2 Componenti e connettori

Un diagramma architetturale a blocchi mostra un overview: i blocchi rappresentano componenti, mentre i connettori rappresentano invocazioni, pipe, eventi, scambio di messaggi o accesso a dati condivisi. UML offre il diagramma delle componenti; esistono anche Architecture Description Languages come Darwin e altri ADL.

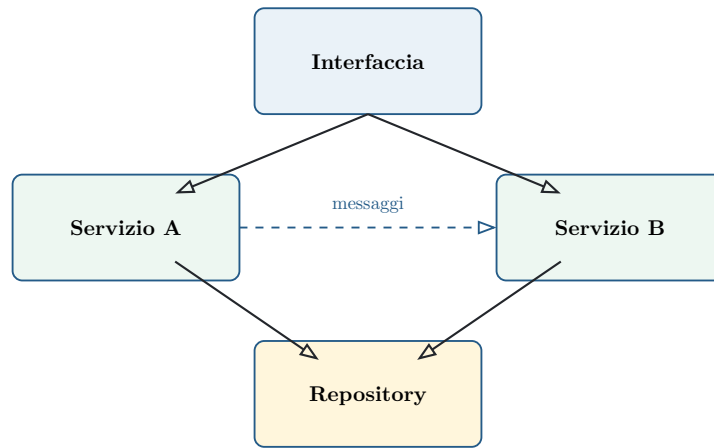


Figura 18: Componenti e connettori descrivono struttura e interazioni principali del sistema.

4.1.3 Impatto sugli attributi di qualità

Proprietà	Strategia architettuale tipica
Performance	Ridurre comunicazioni e usare componenti large-grain.
Security	Adottare livelli e collocare le risorse critiche negli strati interni.
Safety	Concentrare le funzionalità safety-critical in pochi sottosistemi controllabili.
Availability	Aggiungere ridondanza, replica e meccanismi di fault tolerance.
Maintainability	Preferire componenti fine-grain, coesi e sostituibili.

Trade-off inevitabili

Componenti grandi riducono le comunicazioni ma sono più difficili da sostituire; la ridondanza aumenta la disponibilità ma complica safety e consistenza; i livelli migliorano isolamento e security ma aggiungono attraversamenti e latenza. Non esiste un'architettura che massimizzi simultaneamente ogni qualità.

4.2 Stili architetturali strutturali

Uno **stile architetturale** definisce tipi di componenti, tipi di connettori e vincoli di composizione. È un modello generico da istanziare e personalizzare. I grandi sistemi reali sono spesso eterogenei e combinano più stili.

Gli stili **strutturali** evidenziano soprattutto chi comunica con chi; quelli **di controllo** mostrano anche chi determina l'ordine dell'esecuzione.

4.2.1 Layered architecture

Il sistema è organizzato in strati. Ogni livello offre servizi a quello superiore e usa normalmente soltanto il livello immediatamente inferiore, nascondendo i propri dettagli implementativi.

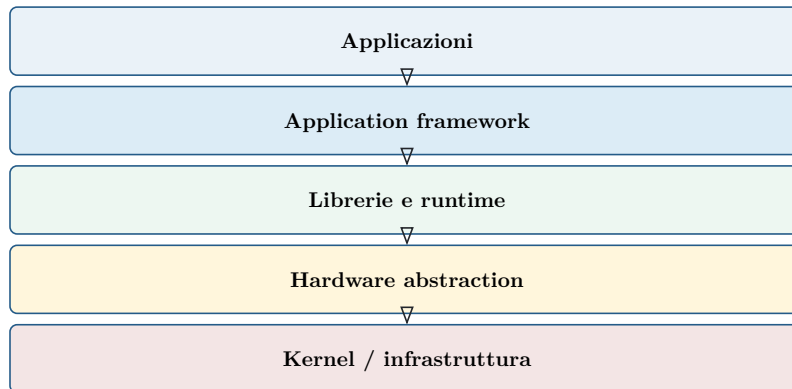


Figura 19: Architettura layered: ogni strato astrae i servizi dello strato sottostante. Android è un esempio noto di organizzazione multilivello.

Vantaggi	Svantaggi
Cambiamenti locali e sostituzione di implementazioni dietro un'interfaccia stabile.	La separazione può essere artificiale e generare molti attraversamenti.
Information hiding e riuso dei livelli.	Gli shortcut tra livelli migliorano talvolta le prestazioni ma rompono l'isolamento.

4.2.2 Repository

I sottosistemi condividono un deposito dati centrale e non comunicano direttamente. È adatto quando più strumenti producono e consumano grandi quantità di informazioni comuni, come nei CASE tool.

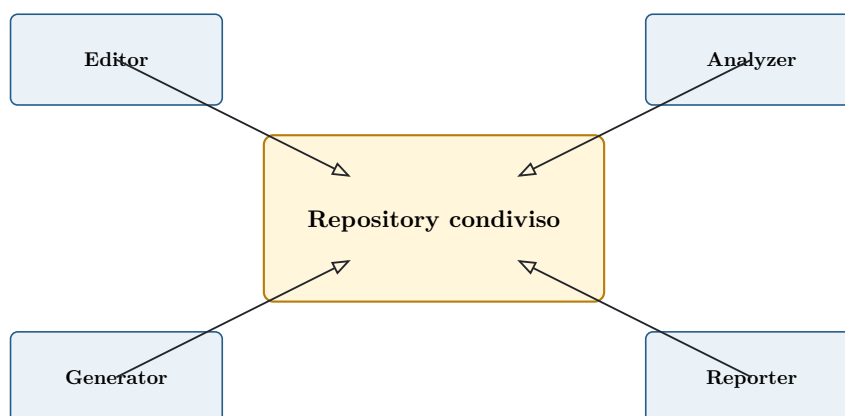


Figura 20: Nel repository model il deposito centrale media ogni condivisione dei dati.

Vantaggi	Svantaggi
Condivisione efficiente e gestione centralizzata di backup e sicurezza.	Schema comune rigido, evoluzione costosa e distribuzione difficile.

4.2.3 Client-server

I server offrono servizi specifici, i client li richiedono attraverso una rete e il protocollo segue tipicamente lo schema request/response.

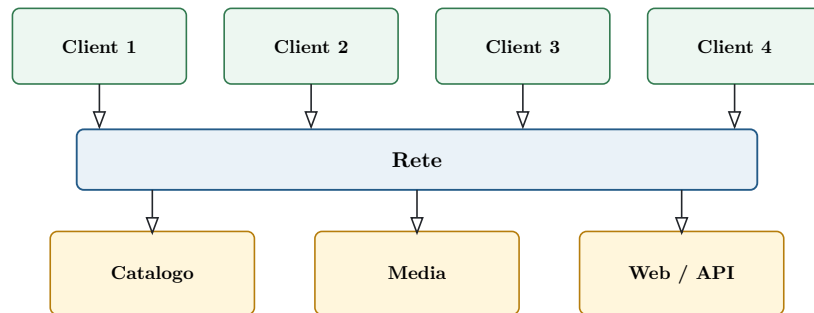


Figura 21: Client e server sono ruoli distinti collegati dalla rete.

Vantaggi	Svantaggi
Server riutilizzabili, scalabilità e possibilità di spostare computazione.	Modelli dati eterogenei, gestione duplicata e dipendenza da organizzazioni diverse.

4.2.3.1 Two-tier e three-tier

Two-tier	Three-tier
Presentation e business logic sono ripartite tra client e server. Un thin client mostra soltanto la UI; un fat client esegue anche elaborazione.	Client per la presentation, application server per la business logic e DB server per la data logic.
Più semplice, ma il client può diventare pesante e difficile da distribuire.	Separa chiaramente presentazione, logica e dati e consente di scalarli indipendentemente.

4.2.4 Peer-to-peer (P2P)

Ogni peer è contemporaneamente client e server: offre servizi e ne richiede ad altri. I ruoli non sono fissi e i nodi si distinguono soprattutto per i dati o le capacità possedute.

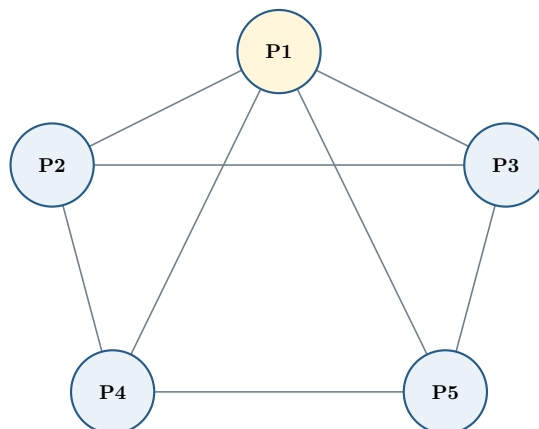


Figura 22: In una rete P2P i nodi collaborano senza una distinzione permanente tra client e server.

Replica e assenza di un singolo server favoriscono scalabilità e fault tolerance: aggiungere un peer può aggiungere dati e capacità. BitTorrent, eMule e reti blockchain sono esempi noti; coordinamento, consistenza e sicurezza restano però complessi.

4.2.5 Pipe and filter

I **filtri** trasformano flussi di input in output; le **pipe** trasportano i dati al filtro successivo.



Figura 23: Pipeline di trasformazioni indipendenti collegate da flussi dati.

L'esempio UNIX `cat input.txt | grep "Italy" | sort > output.txt` compone sorgente, filtri e destinazione. I filtri possono lavorare in parallelo quando producono e consumano il flusso progressivamente.

Vantaggi	Svantaggi
Riuso, composizione intuitiva, facile aggiunta di trasformazioni e concorrenza.	Poco adatto all'interazione; spesso batch; parsing e serializzazione possono costare.

4.3 Stili architetturali di controllo

Gli stili di controllo descrivono come viene determinato l'ordine dell'esecuzione. Nel controllo **centralizzato** un sottosistema avvia e coordina gli altri; nel controllo **event-based** i componenti reagiscono a eventi senza conoscere direttamente i destinatari.

4.3.1 Call-return

Il programma principale, o **driver**, chiama routine e sottoroutine secondo una gerarchia top-down. Il controllo passa alla routine chiamata e ritorna al chiamante al termine; un solo ramo è attivo alla volta, perciò il modello è adatto soprattutto a sistemi sequenziali.

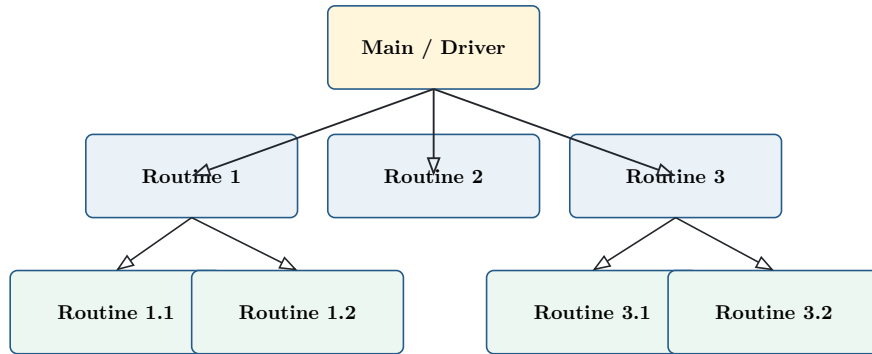


Figura 24: Call-return: il controllo discende dalla radice e ritorna lungo la gerarchia di chiamate.

4.3.2 Manager model

Un manager centrale avvia, termina e coordina processi che possono operare in parallelo. Interroga continuamente sensori, interfaccia e risultati di computazione, quindi attiva processi di calcolo e attuatori.

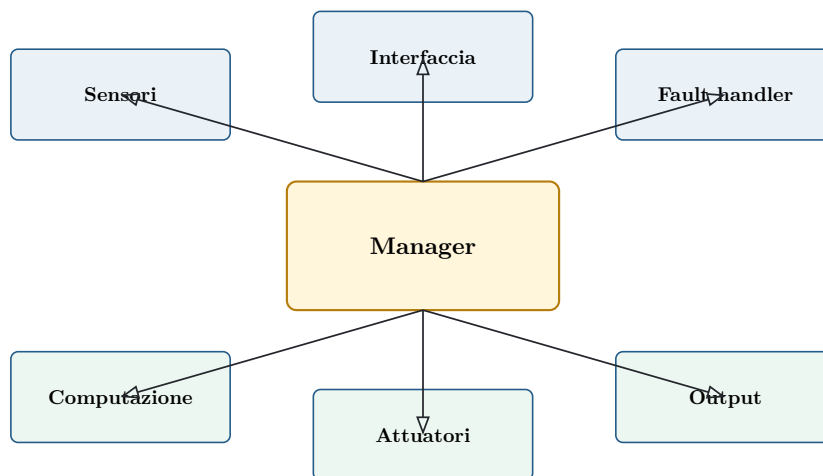


Figura 25: Il manager coordina processi concorrenti e dispositivi del mondo esterno.

È particolarmente adatto a sistemi real-time di controllo e guida. Un **attuatore** traduce un segnale di controllo in un'azione fisica.

4.3.3 Broadcast / publish-subscribe

Un componente pubblica un evento su un event bus; tutti i componenti sono in ascolto e quelli registrati reagiscono. Il produttore non invoca direttamente il consumatore: si parla di **implicit invocation**.

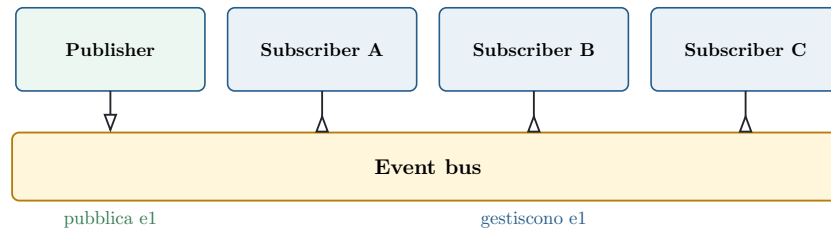


Figura 26: Nel publish-subscribe produttori e consumatori sono disaccoppiati dall'event bus.

Vantaggi	Svantaggi
Componenti sostituibili, estendibili e riusabili con dipendenze ridotte.	Gestione non deterministica, scambio dati complesso, performance e test più difficili.

4.3.3.1 Decisioni progettuali negli eventi

Il disaccoppiamento non elimina la necessità di un contratto: produttori e consumatori devono condividere nome, significato e struttura dell'evento. Occorre inoltre decidere come gestire duplicati, perdita e ordine dei messaggi.

Payload	I dati possono viaggiare con l'evento oppure essere recuperati da un repository mediante un identificatore.
Consegna	Il bus può garantire consegna al massimo una volta o almeno una volta; nel secondo caso i consumatori devono tollerare duplicati.
Ordinamento	Eventi concorrenti possono arrivare in ordine diverso da quello di pubblicazione.
Idempotenza	Elaborare più volte lo stesso evento non dovrebbe produrre effetti ulteriori indesiderati.
Osservabilità	Log, correlation ID e tracing aiutano a ricostruire flussi che attraversano componenti disaccoppiati.

Esempio

L'evento `OrdineConfermato` può attivare indipendentemente fatturazione, aggiornamento inventario e spedizione. Il servizio che conferma l'ordine non deve conoscere né invocare direttamente questi consumatori.

4.4 Architetture eterogenee

I sistemi reali raramente seguono uno stile puro. Gli stili possono essere combinati gerarchicamente — una componente interna adotta uno stile diverso da quello globale — oppure all'interno della stessa componente.

4.4.1 Esempio: compilatore

Un compilatore combina una pipeline di trasformazioni con un repository condiviso. Lexer, parser, analisi semantica, ottimizzazione e generazione del codice formano un pipe-and-filter; la symbol table è consultata da più fasi come repository.

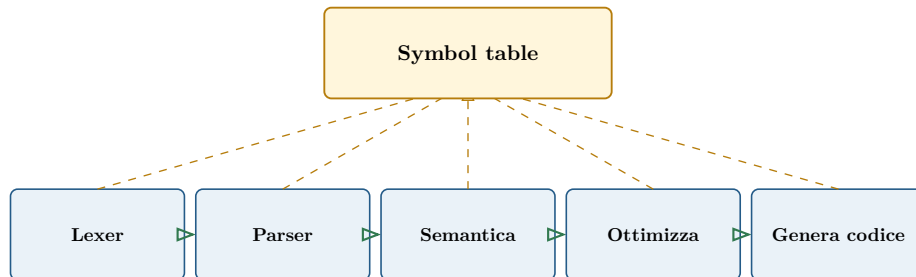


Figura 27: Architettura ibrida di un compilatore: pipeline di fasi e repository condiviso.

Criterio di scelta

Lo stile è uno strumento di ragionamento, non un vincolo ideologico. Componenti diverse possono richiedere organizzazioni diverse in funzione di dati, controllo e attributi di qualità.

4.5 Microservices

I microservizi evolvono le architetture service-oriented e contrastano il monolite, nel quale UI, business logic e accesso ai dati vengono distribuiti come un'unica applicazione.

4.5.1 Proprietà

- ogni servizio è indipendente e responsabile di una capacità di business coesa;
- servizi diversi possono usare tecnologie diverse;
- comunicano tramite API di rete, spesso HTTP/REST o messaggistica;
- possono essere distribuiti, scalati e aggiornati autonomamente.

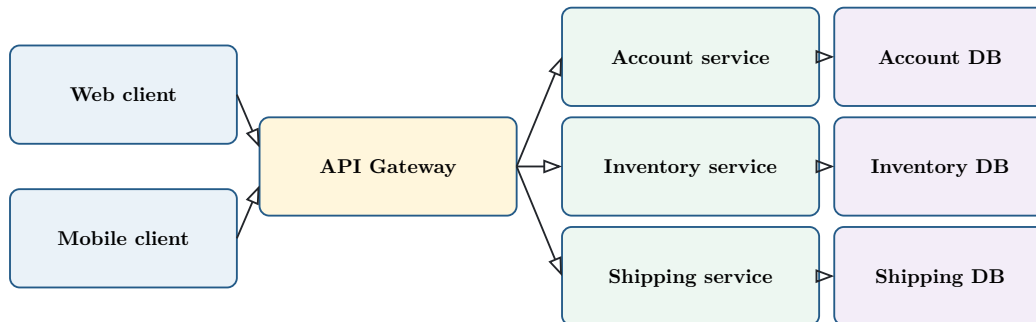


Figura 28: Il gateway espone un'interfaccia unificata e instrada le richieste verso servizi indipendenti.

4.5.2 API Gateway e REST

L'API Gateway offre ai client un endpoint unico, chiama i servizi necessari e aggrega le risposte. Riduce la conoscenza dell'architettura interna da parte dei client, ma può diventare un collo di bottiglia o un punto critico.

REST usa le operazioni HTTP per manipolare risorse:

CRUD	Metodo HTTP
Create	POST
Read	GET
Update	PUT / PATCH
Delete	DELETE

Vantaggi	Svantaggi
Scalabilità selettiva, deployment indipendente, fault isolation e libertà tecnologica.	Complessità distribuita, latenza di rete, test end-to-end e osservabilità difficili.
Team autonomi e servizi focalizzati su capacità specifiche.	Richiede infrastrutture per orchestrazione, monitoring, discovery e gestione dei dati distribuiti.

Docker e Kubernetes sono spesso usati per packaging e orchestrazione, ma non definiscono da soli un'architettura a microservizi.

4.6 Riepilogo e scelta dello stile

Layered	Separazione e information hiding tra livelli.
Repository	Condivisione attraverso un modello dati centrale.
Client-server	Ruoli stabili di fornitore e consumatore di servizi.
P2P	Peer simmetrici che offrono e richiedono servizi.
Pipe and filter	Trasformazioni indipendenti connesse da flussi.
Call-return	Controllo gerarchico e sequenziale.
Manager	Coordinamento centralizzato di processi concorrenti.
Broadcast	Reazione disaccoppiata a eventi pubblicati.
Microservices	Servizi distribuibili in modo indipendente attorno a capacità di business.

Sintesi

Requisiti → design architetturale → architettura software → possibile conformità a uno o più stili. La scelta deve partire dagli attributi di qualità e dai trade-off, non dalla popolarità di una tecnologia.

Il high-level design definisce la struttura globale; il low-level design dettaglia i componenti. Poiché progettare significa soprattutto selezionare, combinare e adattare soluzioni note, i sistemi reali risultano quasi sempre un mix ragionato di stili.

5 Design delle componenti (Low-Level Design)

Il Low-Level Design raffina l'architettura fino a descrivere componenti implementabili: classi, interfacce, attributi, operazioni, strutture dati e algoritmi. Se il High-Level Design stabilisce **quali sottosistemi** esistono e come comunicano, il Low-Level Design precisa **come è costruito ciascun sottosistema**.

5.0.1 Fasi del processo di design

Il processo non è rigidamente sequenziale: le attività si sovrappongono e possono consultare direttamente i requisiti. Ogni passaggio produce però un artefatto più preciso.

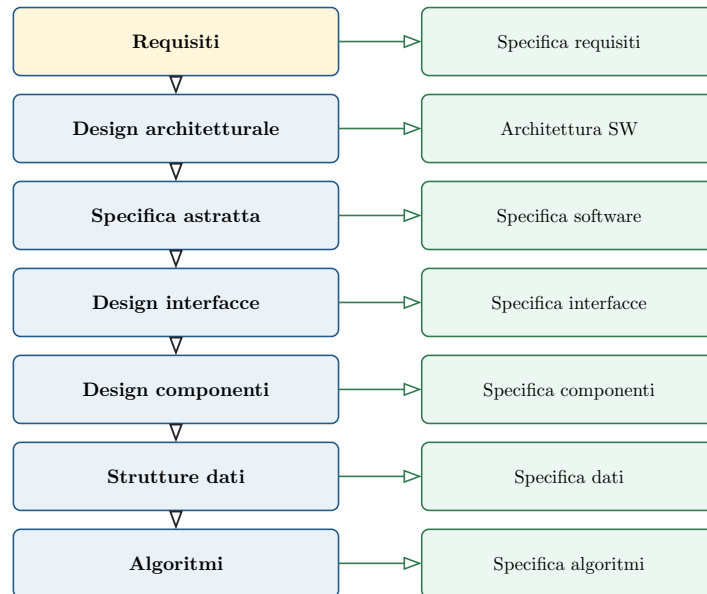


Figura 29: Raffinamento dalle esigenze agli algoritmi e relativi artefatti di design.

Architettura	Componenti, sottosistemi e connettori.
Specifica astratta	Responsabilità di alto livello dei componenti.
Interfacce	Servizi esposti, operazioni e dati scambiati.
Componenti	Classi, attributi, metodi e collaborazioni interne.
Strutture dati	Rappresentazioni adatte ai dati del problema.
Algoritmi	Procedimenti che implementano le operazioni.

Nel Weather Service, per esempio, l'architettura individua `WeatherService`, `LocalInformation`, `SocialNetwork` e `Subscriber`; il design delle interfacce definisce operazioni come `GetHistoricalTemperature`; il component/data design introduce attributi tipizzati; l'algorithm design seleziona ordinamenti e collezioni adeguate.

5.1 Design by Contract

Il **Design by Contract** (Bertrand Meyer, Eiffel, 1988) specifica l'interfaccia di una componente prima della codifica mediante un accordo preciso tra chi offre un servizio e chi lo usa. L'obiettivo è rendere espliciti obblighi, garanzie e responsabilità.

Metafora del contratto

Il **cliente** o caller deve rispettare le condizioni richieste dal servizio. Il **fornitore** o supplier garantisce il risultato soltanto quando tali condizioni sono soddisfatte.

5.1.1 Precondizione, postcondizione e invariante

Precondizione	Proprietà booleana che deve valere prima dell'operazione; è responsabilità del chiamante.
Postcondizione	Proprietà garantita dopo l'operazione se la precondizione era valida.
Invariante	Proprietà che ogni oggetto deve rispettare negli stati osservabili, prima e dopo ogni operazione pubblica.

Per una radice quadrata intera:

`sqrt(x)` **pre:** $x \geq 0$ **post:** $x = \text{result} \times \text{result}$

Durante l'esecuzione interna di un metodo l'invariante può essere temporaneamente violato, ma deve essere ripristinato prima che il controllo torni al cliente.

5.1.2 Esempio: conto corrente

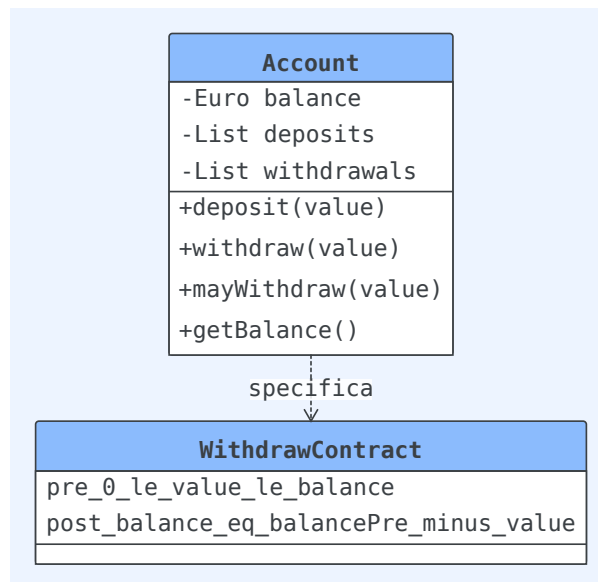
Per `withdraw(value)`:

Precondition `value >= 0 value <= balance`

Postcondition `balance = balance@pre - value`

Class invariant `balance >= 0 balance = sum(deposits) - sum(withdrawals)`

`balance@pre` indica il saldo prima della chiamata. Il contratto garantisce che il prelievo modifichi esattamente il saldo e che l'oggetto resti coerente.



Codice 1: Classe `Account` e contratto dell'operazione `withdraw`.

5.1.3 Responsabilità del cliente e del fornitore



Figura 30: Il contratto distribuisce esplicitamente obblighi e garanzie.

Violazione	Responsabile
Precondizione non valida	Cliente/chiamante
Postcondizione o invariante non validi, con pre valida	Fornitore/implementazione

Senza questa attribuzione si rischiano troppo pochi controlli — ciascuno li delega all'altro — oppure controlli duplicati e incoerenti tipici di un defensive programming indiscriminato.

5.1.4 Vantaggi

- guida codifica e design delle operazioni;
- documenta il comportamento meglio di commenti informali;
- genera test black-box dalle pre/postcondizioni;
- localizza più rapidamente la responsabilità di un failure;
- favorisce operazioni semplici con responsabilità circoscritte.

Eiffel integra nativamente i contratti; altri ecosistemi offrono `assert` o librerie come JML, `icontract`, `deal` e `Code Contracts`.

5.2 Progettazione degli algoritmi

Il design degli algoritmi è la parte più vicina alla codifica. Si parte dalla descrizione della classe target, si riusa quando possibile un algoritmo noto e si progetta una soluzione nuova soltanto quando necessario.

Il percorso tipico è: selezione della notazione, raffinamento progressivo e — nei sistemi critici — prova formale della correttezza, per esempio mediante triple di Hoare.

5.2.1 Notazioni e PDL

Le notazioni visuali includono activity diagram, flowchart, box diagram, structured chart e decision table. Le notazioni testuali includono PDL e pseudocodice.

Il **Program Design Language** combina il vocabolario del linguaggio naturale con strutture di controllo simili a un linguaggio di programmazione. La narrativa consente di mantenere inizialmente un alto livello di astrazione.

```
SORT(TABLE, SIZE)
  IF SIZE > 1
    DO UNTIL NO ITEMS WERE INTERCHANGED
      FOR EACH ADJACENT PAIR IN TABLE
        IF FIRST ITEM > SECOND ITEM
          INTERCHANGE THE TWO ITEMS
```

5.2.2 Stepwise refinement

Lo stepwise refinement sostituisce progressivamente descrizioni narrative con istruzioni precise fino a ottenere una soluzione direttamente traducibile in codice.



Figura 31: Ogni raffinamento riduce l’astrazione e aggiunge decisioni verificabili.

Esempio: per calcolare la differenza tra massimo e minimo di tre interi si passa da “acquisisci A, B, C e calcola massimo e minimo” a chiamate `CALCOLA_MASSIMO/CALCOLA_MINIMO`, quindi si raffina ciascuna operazione con confronti e assegnamenti espliciti.

Quando fermarsi

Il raffinamento termina quando ogni passo è sufficientemente preciso da essere tradotto senza introdurre nuove decisioni sostanziali durante la codifica.

5.3 Principi di progettazione

I principi di design orientano verso software manutenibile, comprensibile, testabile, riusabile, riparabile e portabile. Sono indipendenti dal paradigma OO.

Un **modulo** è qualsiasi entità software nominata che contiene e offre servizi: funzione, script, classe, package, programma o sottosistema. Un modulo può includerne o usarne altri, creando dipendenze.

5.3.1 Astrazione

L'astrazione consente di concentrarsi sulle proprietà rilevanti a un livello, ignorando temporaneamente i dettagli. Riduce la complessità e rende possibile descrivere il comportamento senza fissarne subito l'implementazione.

Forma	Che cosa astrae	Esempio
Funzionale	Algoritmo che realizza un servizio.	<code>sort(list)</code>
Dati	Struttura concreta dietro operazioni pubbliche.	Stack: <code>push/pop</code>
Controllo	Meccanismo interno di coordinamento.	Semaforo

Nel Selection Sort si può passare da “ordina L” alla ricerca iterativa del minimo e infine ai cicli e scambi completi. Ogni livello risponde a domande diverse senza invalidare quelli più astratti.

5.3.2 Decomposizione

Risolvere sottoproblemi separati richiede in genere meno effort che affrontare l'intera complessità in una volta. Tuttavia, aumentando i moduli cresce anche il costo d'integrazione.

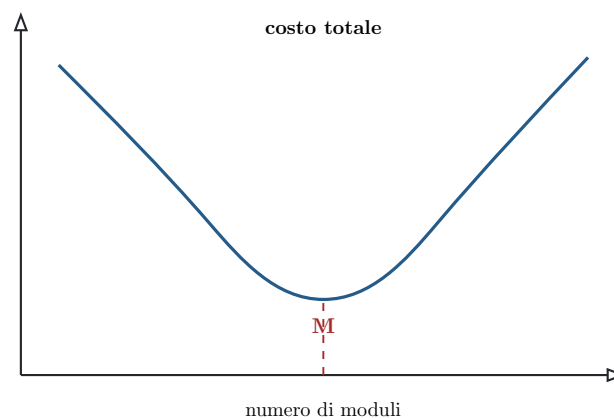


Figura 32: Esiste un numero di moduli M che bilancia complessità interna e costo d'integrazione.

Troppo pochi moduli concentrano la complessità; troppi moltiplicano interfacce, coordinamento e test d'integrazione. La decomposizione va quindi ottimizzata, non massimizzata.

5.3.3 Modularità e Separation of Concerns

La modularità aggiunge un criterio qualitativo alla decomposizione: ogni modulo dovrebbe occuparsi di un concern coerente e avere una sola ragione significativa per cambiare.



Figura 33: Separare calcolo e presentazione permette di sostituire la UI senza modificare la logica.

La stessa idea porta ai livelli presentation, business e data logic e a pattern come MVC.

Criteri fondamentali

Una buona modularizzazione **massimizza la coesione** interna e **minimizza l'accoppiamento** tra moduli. Moduli coesi e poco accoppiati sono più facili da comprendere, testare, modificare e riusare.

5.3.4 Information hiding

Ogni modulo espone l'interfaccia — ciò che offre — e nasconde le decisioni interne — come lo realizza. Finché il contratto pubblico rimane stabile, l'implementazione può cambiare senza effetti a cascata.



Figura 34: Il cliente dipende dall'interfaccia stabile, non dalle decisioni nascoste.

In Java e C++ l'information hiding è supportato da **private**, **protected** e **public** e dall'incapsulamento. Il beneficio generale è la riduzione dell'accoppiamento, non il semplice uso di modificatori sintattici.

5.4 Riepilogo

Design by Contract	Precondizioni, postcondizioni e invarianti assegnano responsabilità a cliente e fornitore.
Algorithm design	PDL e stepwise refinement portano dalla narrativa al codice.
Astrazione	Mostra ciò che conta al livello corrente.
Decomposizione	Divide il problema bilanciando complessità e integrazione.
Modularità	Massimizza coesione e minimizza accoppiamento.
Information hiding	Protegge le decisioni interne dietro interfacce stabili.

Dal problema al codice

Requisiti → High-Level Design (architettura e connettori) → Low-Level Design (classi, contratti, dati e algoritmi) → implementazione. Ogni raffinamento aggiunge precisione senza perdere la tracciabilità verso il problema originario.

6 Introduzione a UML

UML (**Unified Modeling Language**) è una famiglia unificata di notazioni grafiche per descrivere, progettare e documentare sistemi. È particolarmente diffusa per sistemi object-oriented, ma può modellare anche hardware, organizzazioni e domini di business. Lo standard è mantenuto dall'OMG (**Object Management Group**).

Che cosa UML non è

UML non è una metodologia di sviluppo, non è normalmente un linguaggio di programmazione e non è vincolato allo Unified Process. Può essere usato in processi plan-driven, iterativi o agili.

6.1 Modi d'uso e prospettive

6.1.1 Sketch, blueprint e linguaggio eseguibile

Sketch	Diagramma parziale per comunicare, discutere e chiarire le decisioni più importanti. Si realizza anche su lavagna.
Blueprint	Modello dettagliato che dovrebbe guidare quasi meccanicamente l'implementazione; richiede strumenti di modellazione e manutenzione rigorosa.
Linguaggio eseguibile	Modello sufficientemente formale da essere trasformato o eseguito da strumenti specifici.

Nella pratica lo sketch è l'uso più comune: seleziona l'informazione utile invece di rappresentare ogni dettaglio. Un blueprint completo può essere costoso, difficile da validare e rapidamente obsoleto.

6.1.2 Prospettiva concettuale e software

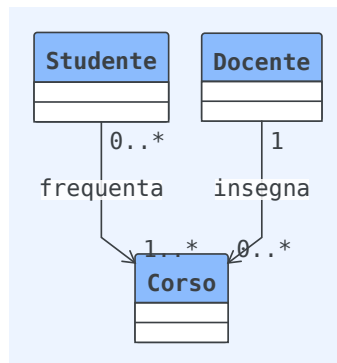
Elemento	Concettuale	Software
Classe	Concetto significativo del dominio.	Classe o modulo da implementare.
Oggetto	Entità concreta del mondo reale.	Istanza a runtime.
Operazione	Responsabilità o azione del dominio.	Servizio implementato da un metodo.
Scopo	Comprendere business e terminologia.	Progettare e documentare il software.

Dichiarare sempre la prospettiva

Lo stesso simbolo può avere significati diversi. Prima di leggere un diagramma bisogna sapere se rappresenta concetti del dominio oppure elementi implementativi.

6.1.3 Modello di dominio

Un modello di dominio è un glossario visuale delle astrazioni rilevanti. Non rappresenta tutto il mondo reale: seleziona concetti, attributi e relazioni utili allo scopo, crea un vocabolario comune tra stakeholder e ispira successivamente nomi e strutture del design.



Codice 2: Modello concettuale del dominio universitario.

6.2 Notazione, meta-modello e viste

UML definisce sia una **notazione** sia un **meta-modello**. La notazione è la sintassi grafica — per esempio rettangoli, linee e molteplicità — mentre il meta-modello definisce i concetti del linguaggio, come Classe, Attributo, Operazione e Associazione.

Per uno sketch è spesso sufficiente conoscere bene la notazione; strumenti di modellazione, trasformazioni e profili richiedono invece maggiore attenzione alla semantica del meta-modello.

6.2.1 Viste e modello 4+1

Un sistema complesso viene descritto mediante più **viste**, ciascuna orientata alle esigenze di uno stakeholder. Un diagramma presenta una vista o una sua parte; un modello è l'insieme coordinato dei diagrammi.

Vista	Destinatario	Domanda principale
Logica	Analista/progettista	Quali funzionalità e astrazioni offre il sistema?
Sviluppo	Developer	Come è organizzato il codice?
Processi	Integrator	Come collaborano processi e thread?
Fisica	System engineer	Dove sono distribuiti software e hardware?
Scenari (+1)	Stakeholder	Come le viste realizzano casi d'uso significativi?

Viste sovrapposte possono essere incoerenti; viste incomplete possono lasciare decisioni scoperte. Completezza e consistenza vanno bilanciate con la leggibilità.

6.3 I diagrammi di UML 2.5

UML 2.5 comprende quattordici tipi di diagramma, divisi tra struttura e comportamento. I diagrammi di interazione sono un sottoinsieme di quelli comportamentali.

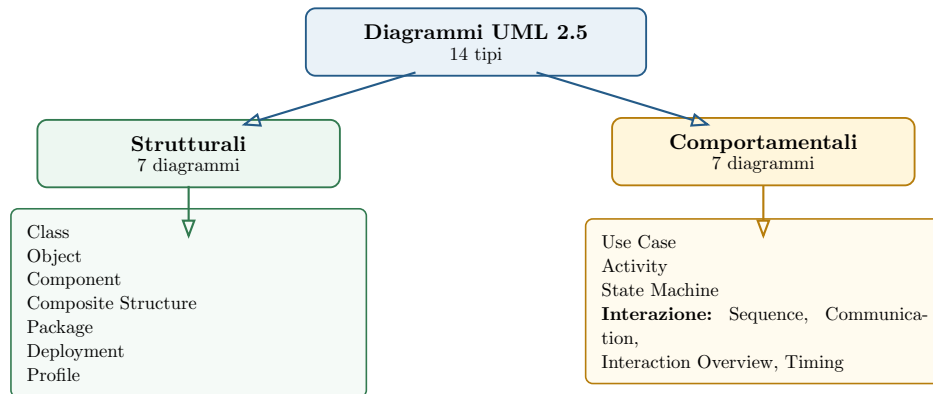


Figura 35: Famiglie dei quattordici diagrammi UML 2.5.

6.3.1 Diagrammi principali nel corso

- **Use Case:** obiettivi funzionali osservati dagli attori.
- **Class e Object:** struttura statica e istanze concrete.
- **Sequence e Communication:** flusso dei messaggi tra oggetti.
- **Activity:** flussi di lavoro e algoritmi.
- **State Machine:** comportamento reattivo dipendente dallo stato.
- **Component e Deployment:** organizzazione software e distribuzione sull'hardware.

Principio 80/20

Non è necessario usare ogni diagramma. Un piccolo sottoinsieme ben compreso copre la maggior parte delle esigenze di comunicazione e design.

6.4 UML nel processo di sviluppo

UML può accompagnare requisiti, architettura e design senza imporre un processo specifico.

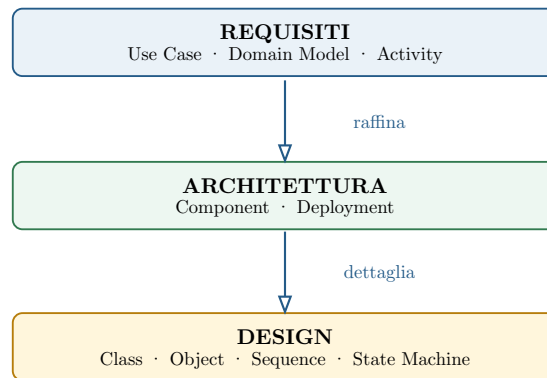


Figura 36: Collocazione tipica dei principali diagrammi nel processo.

I domain model chiariscono il vocabolario; i diagrammi dei componenti descrivono l'architettura; deployment collega nodi hardware e componenti software; class diagram e object diagram guidano il Low-Level Design; sequence e communication mostrano le collaborazioni; activity e state machine precisano algoritmi e comportamento dinamico.

6.5 Profili, codice e uso pragmatico

6.5.1 Profili UML

Un profilo adatta UML a un dominio o a una tecnologia mediante:

- **stereotipi**, scritti come «stereotype» o rappresentati da icone;
- vincoli aggiuntivi sugli elementi ammessi;
- semantica specifica del dominio.

Profili possono descrivere applicazioni web, sistemi embedded, domini ferroviari o spaziali. Sono estensioni controllate, non nuovi linguaggi scollegati da UML.

6.5.2 UML e codice

Non esiste una corrispondenza universale e completa tra UML e ogni linguaggio di programmazione. Un diagramma è normalmente più astratto del codice: offre una buona approssimazione della struttura o del comportamento, ma non determina ogni dettaglio esecutivo.

6.5.3 Consigli pratici

1. Dichiarare uso e prospettiva: per esempio “sketch software”.
2. Ricordare che ogni informazione può essere soppressa: l’assenza non dimostra che qualcosa non esista.
3. Preferire un diagramma chiaro e leggermente convenzionale a uno formalmente perfetto ma illeggibile.
4. Usare soltanto il sottoinsieme di UML realmente compreso e utile.
5. Segnalare le convenzioni non normative.
6. Non esitare a usare diagrammi non UML quando comunicano meglio lo scopo.

Regola fondamentale

La completezza è nemica della chiarezza. Lo scopo primario del diagramma è far comprendere una decisione o una struttura; la conformità allo standard serve la comunicazione, non la sostituisce.

6.6 Riepilogo

UML è una lingua franca visuale, non una metodologia. Può essere usato come sketch, blueprint o modello eseguibile e può assumere prospettiva concettuale o software. Un modello combina viste e diagrammi diversi; la loro scelta dipende dagli stakeholder, dalla fase del processo e dalla domanda a cui si vuole rispondere.

7 Class diagram e Object diagram UML

Il **Class Diagram** descrive la struttura statica di ciò che si sta modellando: classi, feature e relazioni. Le feature comprendono attributi e operazioni; le relazioni principali sono associazione, aggregazione, composizione, generalizzazione e dipendenza.

La prospettiva cambia il significato

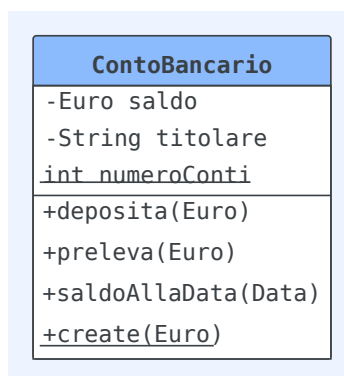
In prospettiva concettuale una classe rappresenta un concetto del dominio e un'operazione una responsabilità. In prospettiva software la classe è un modulo implementabile e l'operazione è realizzata da uno o più metodi.

7.1 Classi, attributi e operazioni

Una classe incapsula caratteristiche comuni a un insieme di oggetti. Gli attributi determinano lo stato; le operazioni descrivono il comportamento disponibile. Oggetti collegati collaborano scambiando messaggi, cioè invocando operazioni.

7.1.1 Notazione della classe

La rappresentazione completa usa tre compartimenti: nome, attributi e operazioni. I compartimenti non necessari possono essere soppressi per mantenere il diagramma leggibile.



Codice 3: Classe UML con i compartimenti di nome, attributi e operazioni.

7.1.2 Attributi

La sintassi generale è:

visibilità nome: tipo [molteplicità] = default {proprietà}

Simbolo	Visibilità	Accesso
+	public	Tutte le classi.
-	private	Soltanto la classe proprietaria.
#	protected	Classe e sottoclassi.
~	package	Classi dello stesso package.

Solo il nome è obbligatorio. Il tipo può essere primitivo, datatype o classe del modello. Default indica il valore iniziale; proprietà come {readOnly}, {ordered}, {static} o {abstract} aggiungono semantica.

Le regole di visibilità dei linguaggi non coincidono sempre con UML: in un design software conviene adottare convenzioni compatibili con il linguaggio target e spesso limitarsi a + e -.

7.1.3 Molteplicità

- 1: esattamente uno, valore predefinito;

- 0..1: opzionale;
- * o 0..*: qualsiasi quantità, anche zero;
- 1..*: almeno uno;
- n..m: intervallo limitato.

Un attributo multivalore è concettualmente un insieme; `{ordered}` specifica che l'ordine è significativo. Per valori con comportamento o semantica propri è spesso preferibile introdurre un datatype, per esempio `CifraTelefono`, invece di usare una stringa generica.

7.1.4 Operazioni

La sintassi è:

```
visibilità nome(direzione parametro: tipo = default): tipoRitornato {proprietà}
```

La direzione è `in`, `out` o `inout`; `in` è predefinita. Le operazioni **query** osservano senza side effect, mentre i modificatori cambiano lo stato. Getter e setter ovvi possono essere omessi.

Un'operazione è la dichiarazione astratta del servizio; il **metodo** è il corpo che la implementa. Una stessa operazione astratta può avere metodi differenti nelle sottoclassi.

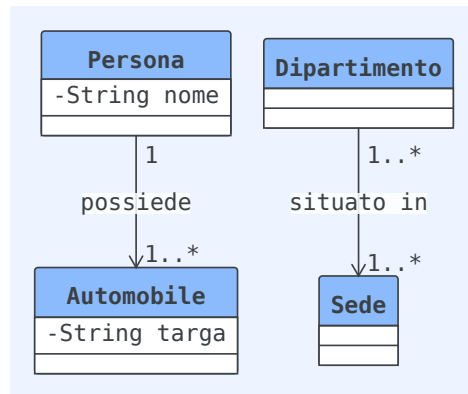
7.1.5 Datatype, costruttori e membri statici

Un oggetto possiede identità: due persone con gli stessi dati possono essere entità diverse. Un datatype rappresenta invece valori: due istanze di `Euro` con valore 10 sono equivalenti. In UML si usa lo stereotipo «datatype».

Attributi e operazioni statici appartengono alla classe e sono tradizionalmente sottolineati. I costruttori possono essere indicati come `create`, `make` o col nome della classe, secondo la convenzione adottata.

7.2 Associazioni e molteplicità

Un'associazione rappresenta una relazione fisica o concettuale tra classi. Può avere nome, ruoli agli estremi, molteplicità e verso di navigazione.



Codice 4: Associazioni orientate con nomi e molteplicità.

La freccia di navigabilità indica che, noto un oggetto all'origine, è possibile reperire quelli alla destinazione. Non coincide necessariamente con la direzione in cui si legge il nome dell'associazione.

Attenzione alle molteplicità

La molteplicità scritta accanto a una classe indica quanti oggetti di quella classe possono essere collegati a una singola istanza dell'altra estremità. Non va letta “dal proprio lato”.

7.2.1 Attributo o associazione?

La stessa informazione può talvolta essere espressa in entrambi i modi. Convenzionalmente si usano attributi per primitivi e datatype (`String`, `Date`, `boolean`) e associazioni quando il tipo è una classe con identità propria.

7.2.2 Associazioni riflessive

Una classe può essere associata a se stessa. Per esempio una cartella contiene altre cartelle o una persona svolge il ruolo di genitore rispetto ad altre persone. I nomi di ruolo sono essenziali per distinguere le estremità.

7.2.3 Associazioni bidirezionali

Una relazione navigabile in entrambe le direzioni richiede che entrambe le estremità siano mantenute coerenti. In una relazione **Man-Woman**, impostare **husband** deve aggiornare anche **wife**; un solo attributo non basta.

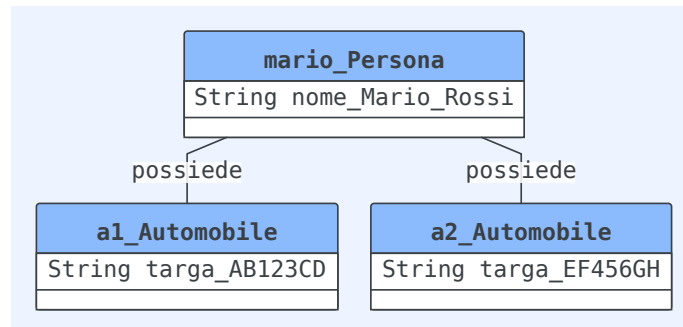
Costo della libertà del modello

Un design astratto e bidirezionale può sembrare semplice nel diagramma, ma trasferisce complessità alla codifica: sincronizzazione, collezioni inverse e vincoli devono essere implementati esplicitamente.

7.3 Object diagram e classi associative

7.3.1 Object diagram

Il Class Diagram descrive tutte le configurazioni valide; l'Object Diagram mostra una configurazione concreta in un istante. Un oggetto è scritto **nome : Classe**, tradizionalmente sottolineato, e un link è un'istanza di un'associazione.

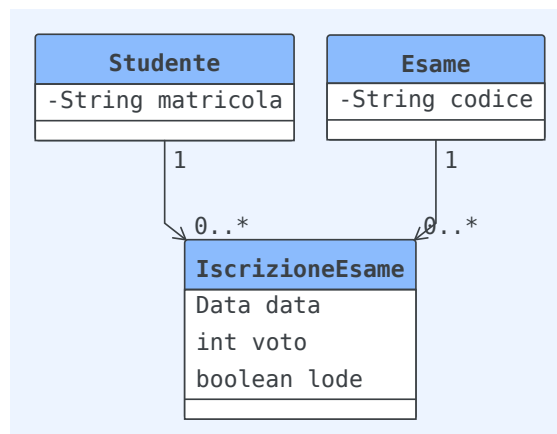


Codice 5: Snapshot di oggetti e link coerente col precedente Class Diagram.

Gli Object Diagram sono utili per spiegare class diagram complessi, verificare le molteplicità con esempi e discutere casi limite.

7.3.2 Classe associativa

Quando data, voto, quantità o altre proprietà appartengono alla relazione e non alle classi coinvolte, si usa una **association class**. In implementazione è spesso più chiaro trasformarla in una normale classe collegata alle due estremità.

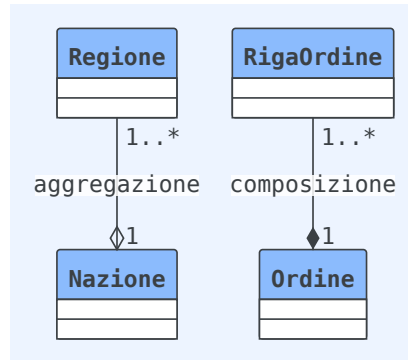


Codice 6: Le proprietà dell'associazione sono modellate come classe implementabile.

7.4 Relazioni tutto-parte e generalizzazione

7.4.1 Aggregazione e composizione

L'aggregazione rappresenta una relazione tutto-parte debole. La composizione è la forma forte: la parte non esiste senza il tutto e può appartenere a un solo composto alla volta.

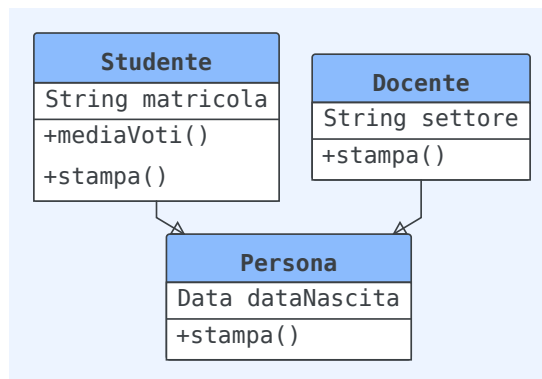


Codice 7: Diamante vuoto per aggregazione e pieno per composizione.

La distinzione dell'aggregazione è spesso ambigua e a livello software si implementa come un'associazione. Conviene usarla soprattutto quando il significato concettuale tutto-parte è davvero informativo.

7.4.2 Generalizzazione ed ereditarietà

La generalizzazione esprime una relazione concettuale “è un”: ogni istanza della sottoclasse è anche istanza della superclasse. L'ereditarietà è il meccanismo implementativo con cui la sottoclasse incorpora e specializza struttura e comportamento.

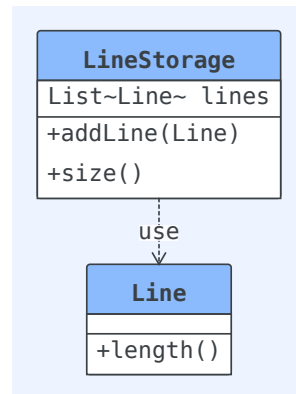


Codice 8: Le sottoclassi ereditano feature e possono aggiungerle o ridefinirle.

Un oggetto **Studente** può essere trattato come **Persona**; la sottoclasse aggiunge `matricola` e `mediaVoti()` e può ridefinire `stampa()` mediante overriding. Operazione e metodo non coincidono: l'operazione è il contratto invocabile, il metodo una sua implementazione concreta.

7.5 Dipendenze e traduzione nel codice

Una classe Cliente dipende da un Fornitore quando una modifica all'interfaccia del fornitore può richiedere una modifica al cliente. La dipendenza è rappresentata da una freccia tratteggiata.



Codice 9: `LineStorage` dipende da `Line` perché la usa come tipo e parametro.

Cause comuni sono invocare operazioni, creare istanze o usare il fornitore come tipo di attributo, variabile locale, parametro o valore restituito. Nel diagramma vanno mostrate soltanto le dipendenze significative: troppe frecce nascondono la struttura che si voleva comunicare.

7.5.1 Dalle molteplicità alle strutture dati

1	Riferimento o attributo singolo.
0..1	Riferimento opzionale.
0..* / 1..*	Collezione tipizzata, per esempio <code>Set<T></code> ; il minimo deve essere validato.
{ordered} 0..*	Lista ordinata, per esempio <code>ArrayList<T></code> o <code>LinkedList<T></code> .

Non esiste una traduzione unica. Le associazioni bidirezionali richiedono due strutture sincronizzate; aggregazione e composizione richiedono politiche di ciclo di vita; vincoli e invarianti devono essere implementati o verificati.

Gestire le dipendenze

Minimizzare dipendenze e cicli, specialmente tra package. Diagrammi troppo completi sono illeggibili; strumenti automatici possono aiutare a individuare dipendenze effettive nel codice.

7.6 Riepilogo

Classe	Nome, attributi e operazioni; i dettagli possono essere soppressi.
Associazione	Relazione con ruoli, navigabilità e molteplicità.
Object Diagram	Esempio concreto di oggetti e link.
Aggregazione	Relazione tutto-parte debole.
Composizione	Tutto-parte forte con ciclo di vita condiviso.
Generalizzazione	Relazione “è un” e base per l’ereditarietà.
Dipendenza	Il cliente può essere influenzato da modifiche al fornitore.

Regola finale

Un buon Class Diagram non mostra tutto: seleziona classi, feature e relazioni necessarie alla domanda corrente, dichiara la prospettiva e usa esempi di oggetti per validare le decisioni più complesse.

8 Interfacce e vincoli UML

Il termine **interfaccia** indica sia l'insieme delle operazioni pubblicamente visibili di una classe, sia un classificatore UML simile a una classe ma privo di implementazione. Un'interfaccia dichiara un contratto; una o più classi possono fornire realizzazioni differenti.

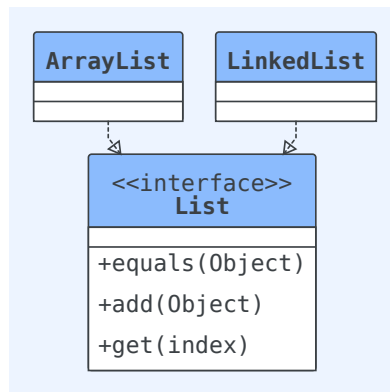
Idea fondamentale

Chi usa un'interfaccia dipende dal contratto, non dalla classe concreta che in quel momento lo implementa. Questo riduce l'accoppiamento e rende le implementazioni sostituibili.

8.1 Interfacce fornite e richieste

8.1.1 Realizzazione di un'interfaccia

Una classe **fornisce** un'interfaccia quando implementa tutte le operazioni che essa dichiara. UML rappresenta la realizzazione con una linea tratteggiata e un triangolo vuoto rivolto verso l'interfaccia. Lo stereotipo «interface» distingue l'interfaccia da una classe ordinaria.



Codice 10: **ArrayList** e **LinkedList** realizzano la stessa interfaccia **List**.

In Java questa relazione corrisponde a `implements List`. Una classe può realizzare più interfacce; un'interfaccia può inoltre specializzarne altre.

8.1.2 Interfaccia richiesta

Una classe **richiede** un'interfaccia quando ha bisogno di un oggetto che la implementi per svolgere il proprio compito. Non richiede necessariamente una particolare classe concreta: qualunque realizzazione compatibile può soddisfare la dipendenza.

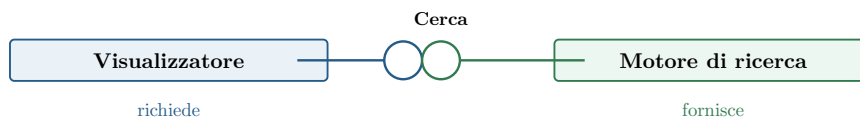


Figura 37: Il visualizzatore richiede il servizio **Cerca**, fornito dal motore di ricerca.

Il cerchio, detto **lollipop**, indica un'interfaccia fornita; il semicerchio o **socket** indica un'interfaccia richiesta. Quando socket e lollipop sono collegati, il componente cliente usa il servizio del fornitore.

8.2 Due notazioni equivalenti

La stessa relazione può essere espressa con la notazione estesa — rettangolo «interface» e realizzazione tratteggiata — oppure con il lollipop. La prima mostra comodamente le operazioni; la seconda è più compatta nei diagrammi architetturali.

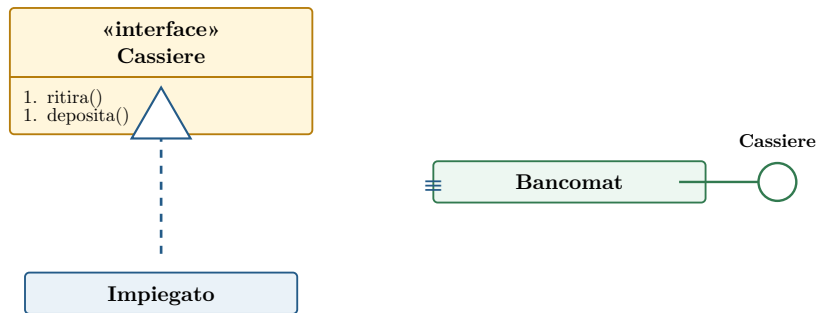


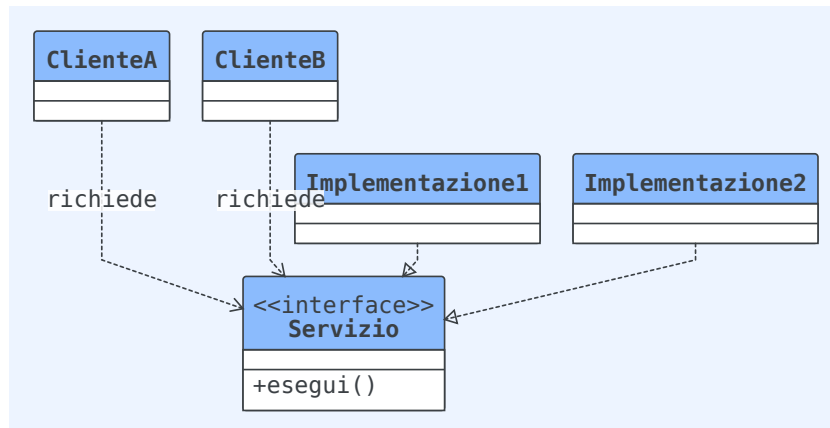
Figura 38: Notazione estesa e lollipop esprimono una realizzazione d'interfaccia.

Come scegliere la notazione

Usare il rettangolo quando interessano operazioni, generalizzazioni o dettagli del contratto; usare lollipop e socket quando interessa soprattutto mostrare come componenti e servizi sono collegati.

8.3 Perché usare le interfacce

Senza interfaccia, un cliente dipende direttamente da una classe concreta. Introducendo un contratto intermedio, sia il cliente sia le implementazioni dipendono da un'astrazione stabile.



Codice 11: Clienti e implementazioni sono disaccoppiati da un contratto comune.

I principali vantaggi sono:

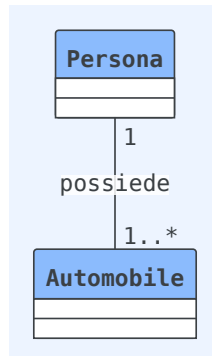
- **sostituibilità**: una realizzazione può essere rimpiazzata senza cambiare il cliente;
- **testabilità**: un test può fornire un'implementazione fittizia (**mock** o **stub**);
- **sviluppo parallelo**: team diversi lavorano contro un contratto concordato;
- **riduzione delle dipendenze**: il cliente conosce solo le operazioni necessarie;
- **polimorfismo**: oggetti di classi diverse sono usati uniformemente.

Interfacce troppo grandi

Un'interfaccia che raccoglie servizi non correlati costringe i clienti a dipendere da operazioni inutili. È preferibile definire contratti piccoli e coesi, orientati alle esigenze dei client.

8.4 Vincoli nei Class Diagram

Un Class Diagram non descrive soltanto elementi strutturali: associazioni, attributi, molteplicità e generalizzazioni impongono già vincoli sulle configurazioni ammesse. Per esempio, la molteplicità 1 accanto a **Persona** stabilisce che ogni automobile abbia esattamente un proprietario.



Codice 12: La molteplicità esprime il vincolo: ogni automobile appartiene a una sola persona.

I costrutti grafici non sono però sufficienti per ogni regola di business. UML consente di aggiungere vincoli mediante linguaggio naturale, codice o **OCL (Object Constraint Language)**.

8.4.1 Object Constraint Language

OCL è il linguaggio formale di specifica associato a UML, basato sulla logica del primo ordine. Può esprimere invarianti, precondizioni, postcondizioni e interrogazioni senza descrivere come implementarle.

Esempio di invariante OCL

```
context Automobile inv UnSoloProprietario: self.proprietario->size() = 1
```

Vincolo dichiarativo

OCL specifica ciò che deve essere vero, non l'algoritmo con cui ottenerlo. Il modello resta quindi indipendente dall'implementazione, ma può essere verificato con strumenti automatici.

Riepilogo

Interfaccia	Contratto pubblico privo di implementazione concreta.
Realizzazione	Una classe fornisce le operazioni dichiarate dall'interfaccia.
Interfaccia richiesta	Il cliente necessita di una realizzazione del contratto.
Lollipop/socket	Notazione compatta per servizi forniti e richiesti.
Vincolo	Regola che limita le configurazioni ammesse dal modello.
OCL	Linguaggio formale e dichiarativo per vincoli UML.

9 State Machine Diagram

Uno **State Machine Diagram** descrive il comportamento di un'entità come variazione del suo stato interno in risposta a eventi. L'entità può essere un sistema software o hardware, una classe, un singolo oggetto o un'entità del mondo reale.

La domanda del diagramma

Dato lo stato corrente e un evento ricevuto, quale transizione può scattare, quali azioni vengono eseguite e quale sarà il nuovo stato?

9.1 Stato e comportamento

9.1.1 Modellazione statica e dinamica

Class e Object Diagram descrivono entità e relazioni, cioè la struttura statica. I diagrammi di interazione mostrano più oggetti che collaborano per un obiettivo; una macchina a stati segue invece un'entità durante il suo ciclo di vita.

Uno **stato** è una situazione durante la quale valgono determinate condizioni e possono essere eseguite attività. Non coincide con la combinazione completa dei valori dei campi: è un'astrazione che raggruppa configurazioni nelle quali l'entità reagisce nello stesso modo agli eventi.

Esempio del distributore

Premere “eroga” produce caffè se il credito è sufficiente, ma mostra un messaggio se è insufficiente. Lo stesso evento genera comportamenti diversi perché cambia lo stato interno.

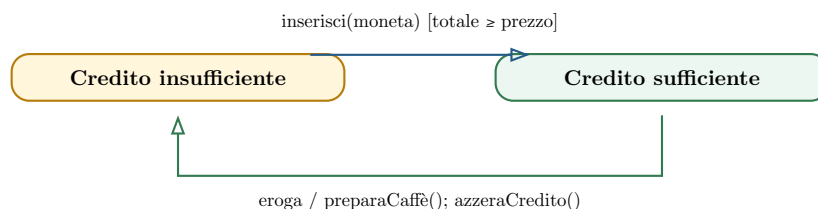


Figura 39: Il comportamento dipende sia dall'evento sia dallo stato corrente.

9.2 Notazione e semantica

9.2.1 Stati, transizioni e pseudo-stati

Graficamente una macchina a stati è un grafo: i nodi arrotondati sono stati e gli archi orientati sono transizioni. Il disco nero è lo pseudo-stato iniziale, unico nel diagramma o nello stato composito; il disco nero bordato è uno stato finale e può comparire più volte.

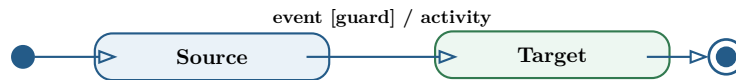


Figura 40: Sintassi fondamentale di una transizione UML.

La forma completa dell'etichetta è:

evento [guardia] / attività

- l'**evento** è il trigger che propone il passaggio;
- la **guardia** è una condizione booleana che deve risultare vera;
- l'**attività** è l'azione atomica eseguita durante la transizione.

Tutti e tre gli elementi sono opzionali. Senza attività si cambia soltanto stato; senza guardia l'evento abilita sempre la transizione; senza evento la transizione è di completamento e scatta immediatamente quando lo stato sorgente termina.

9.2.2 Eventi

Un evento può essere:

- la ricezione di un segnale o di una chiamata a operazione;
- un **change event**, scritto `when(condizione)`, quando la condizione passa da falsa a vera;
- un **time event**, come `after(2 min)` oppure `at(24:00)`.

Le azioni possono usare attributi, operazioni, link e parametri conosciuti dall'oggetto modellato. Riferirsi direttamente a elementi che l'oggetto non conosce rende il modello non implementabile.

9.2.3 Run-to-completion

Gli eventi ricevuti vengono accodati e processati uno alla volta. La semantica è **run-to-completion**: un nuovo evento viene estratto solo quando l'elaborazione del precedente è terminata. Se più transizioni sono contemporaneamente abilitate, ne viene scelta una sola; un modello con guardie sovrapposte può quindi essere non deterministico.

Guardie mutuamente esclusive

Da uno stesso stato, transizioni con lo stesso evento dovrebbero avere guardie disgiunte e possibilmente complete. Se due guardie sono vere, la scelta non è deterministica; se nessuna è vera, l'evento viene ignorato.

9.3 Esempio: ciclo di vita di una copia di libro

La macchina seguente è associata a una singola `CopiaLibro`. Mostra una prospettiva software: eventi e azioni sono operazioni e modifiche dello stato dell'oggetto.

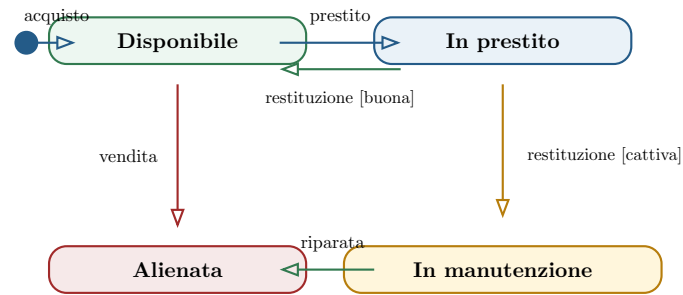


Figura 41: Macchina a stati semplificata di una copia di libro.

La restituzione produce due transizioni alternative: una copia in buone condizioni torna disponibile; una copia danneggiata entra in manutenzione. L'azione associata alla vendita può aggiornare il libro collegato se quella alienata era l'ultima copia disponibile.

9.4 Attività interne

9.4.1 Entry, exit, do e reazioni interne

Il secondo comparto di uno stato può contenere attività che non producono un cambiamento di stato:

- **entry** / azione: eseguita ogni volta che si entra;
- **exit** / azione: eseguita ogni volta che si esce;
- **do** / attività: eseguita mentre lo stato è attivo, può durare ed essere interrotta;
- **evento** / azione: reazione interna a un evento.

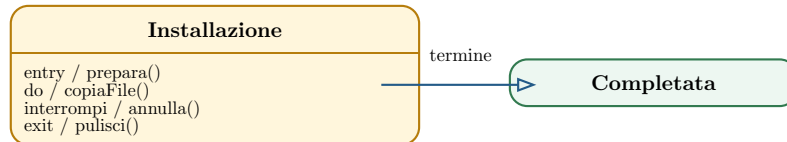


Figura 42: Uno stato può dichiarare attività di ingresso, permanenza, uscita e reazioni interne.

Una **self-transition** esce dallo stato e vi rientra, quindi esegue **exit** ed **entry**. Una reazione interna non abbandona lo stato e non lo attiva: le due notazioni non sono equivalenti.

9.5 Stati composti e concorrenza

9.5.1 Stati composti

Uno stato composto contiene una macchina a stati interna. Permette di nascondere dettagli e suddividere un comportamento complesso. Una transizione disegnata dal bordo del super-stato è disponibile da tutti i suoi sotto-stati; sono possibili anche transizioni da uno stato interno verso l'esterno.

Gli **entry point** e gli **exit point** consentono di entrare o uscire dal composto in punti nominati diversi. Una giunzione (**junction**) compatta ramificazioni e ricongiungimenti condizionali.

9.5.2 Regioni ortogonali

Un composto diviso in regioni ortogonali modella comportamenti concorrenti. L'entità si trova contemporaneamente in uno stato di ogni regione; l'uscita dal composto avviene normalmente quando tutte le regioni hanno raggiunto il proprio stato finale.

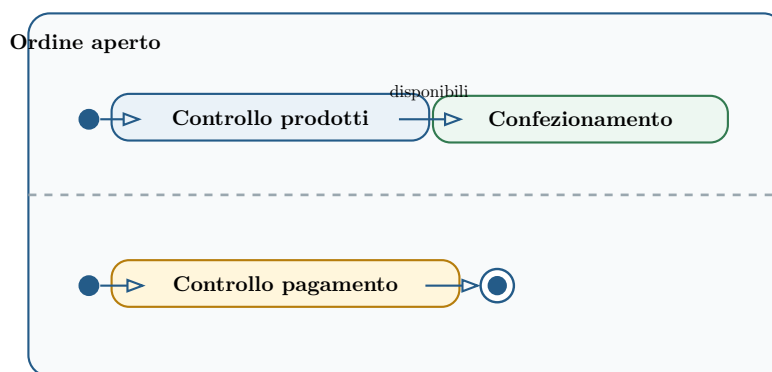


Figura 43: Due regioni ortogonali controllano prodotti e pagamento in concorrenza.

Concorrenza, non necessariamente parallelismo

Le regioni esprimono evoluzioni logicamente concorrenti. L'implementazione può usare thread diversi oppure interleaving su un singolo thread, purché rispetti la semantica del modello.

9.6 Quando usare una State Machine

Le macchine a stati sono utili quando un'entità possiede una logica interna significativa e la validità delle operazioni dipende dalla sua storia. Esempi tipici sono controllori automatici, distributori, protocolli, workflow documentali, dispositivi medicali e interfacce grafiche.

Sono meno utili per classi prevalentemente passive, servizi stateless o oggetti il cui comportamento non cambia sensibilmente nel tempo.

9.6.1 Esempio: controllore di un forno

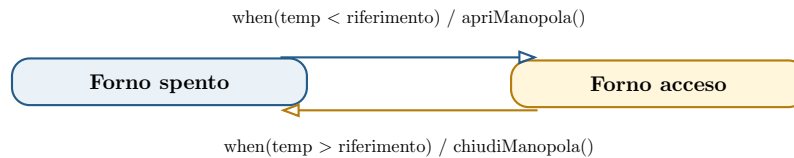


Figura 44: Il controllore reagisce agli eventi di cambiamento della temperatura.

9.6.2 Esempio: workflow documentale

Un documento può attraversare gli stati *In attesa di upload*, *In attesa di assegnazione*, *In revisione* e *In attesa di approvazione*. L'approvazione termina il flusso; il rifiuto lo riporta in revisione. Le azioni — upload, assegnazione, revisione e approvazione — appartengono alle transizioni.

Prima di costruire il diagramma conviene:

1. scegliere l'entità e la prospettiva;
2. individuare gli stati in cui cambia la risposta agli eventi;
3. elencare gli eventi significativi;
4. aggiungere transizioni e guardie;
5. completare con azioni, stati composti e concorrenza solo quando necessari;
6. verificare casi limite, eventi inattesi e raggiungibilità degli stati.

9.7 Implementazione

Nei linguaggi che non supportano direttamente le macchine a stati si usano principalmente tre strategie.

9.7.1 Switch sullo stato corrente

Un attributo memorizza lo stato; ogni metodo-evento esegue uno `switch` e seleziona la transizione valida. È diretto e adatto a modelli piccoli, ma diventa laborioso quando aumentano stati, eventi e modifiche.

```
void restituzione() {
    switch (statoCorrente) {
        case IN_PRESTITO:
            if (condizione == BUONA) statoCorrente = DISPONIBILE;
            else statoCorrente = IN_MANUTENZIONE;
            break;
        default:
            throw new TransizioneNonValida();
    }
}
```

9.7.2 Design Pattern State

Ogni stato è rappresentato da una classe che implementa una stessa interfaccia. Il contesto delega allo stato corrente e il polimorfismo seleziona il comportamento. La soluzione separa le responsabilità e rende più semplici le estensioni, ma introduce più oggetti e classi.

9.7.3 Tabelle di stato e generazione

Una tabella elenca sorgente, destinazione, evento, guardia e azione. È una rappresentazione regolare, verificabile e adatta alla generazione automatica del codice con strumenti model-driven.

Sorgente	Destinazione	Evento	Guardia	Azione
Disponibile	In prestito	prestito	—	registra
In prestito	Disponibile	restituzione	buona	ricollocata
In prestito	Manutenzione	restituzione	cattiva	segnala

Strumenti come SMC trasformano la tabella in una descrizione formale e generano la logica della macchina, spesso secondo il pattern State. Lo sviluppatore implementa poi le azioni applicative ed espone gli eventi come metodi.

Riepilogo

Stato	Astrazione delle situazioni con uguale comportamento agli eventi.
Transizione	evento [guardia] / attività; ogni parte è opzionale.
Semantica	Eventi accodati ed elaborati uno alla volta, run-to-completion.
Attività interne	entry, exit, do e reazioni senza cambio di stato.
Stato composito	Raggruppa sotto-stati e può contenere regioni concorrenti.
Implementazione	Switch, pattern State oppure tabella con generazione automatica.

10 Sequence e Communication Diagram

I **diagrammi di interazione** descrivono come un gruppo di partecipanti collabora scambiandosi messaggi per raggiungere un obiettivo. Il Sequence Diagram privilegia l'ordine temporale; il Communication Diagram mette in evidenza i collegamenti tra gli oggetti.

Messaggio e struttura

In prospettiva software un messaggio rappresenta normalmente la chiamata di un'operazione. Affinché il mittente possa inviarlo deve poter raggiungere il destinatario tramite un attributo, un parametro, un link o un'altra dipendenza.

10.1 Fondamenti dei Sequence Diagram

10.1.1 Partecipanti, lifeline e attivazioni

Ogni partecipante è disposto orizzontalmente nella parte alta. La linea verticale tratteggiata è la sua **lifeline** e il tempo scorre dall'alto verso il basso. Una barra di attivazione indica che il partecipante possiede il controllo o sta eseguendo un'operazione.

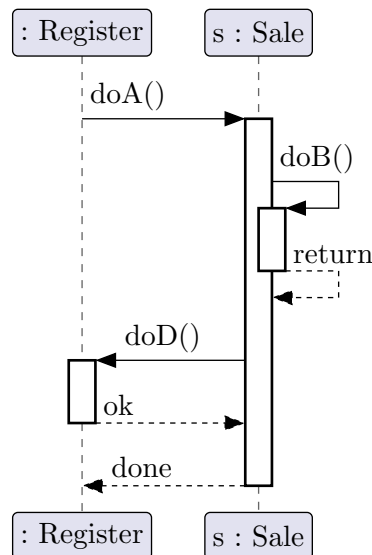


Figura 45: Partecipanti anonimi, chiamata annidata, self-message e valori di ritorno.

Un nome come `s : Sale` identifica l'oggetto e il suo tipo; `: Sale` indica un'istanza anonima. Nei diagrammi di design i partecipanti sono tipicamente oggetti, non classi.

10.1.2 Messaggi sincroni e asincroni

Con un messaggio **sincrono** il mittente attende il completamento dell'operazione. Con un messaggio **asincrono** continua invece l'esecuzione, rendendo possibile rappresentare concorrenza, code ed eventi. Il ritorno può essere mostrato con una freccia tratteggiata, ma viene spesso omesso quando è ovvio.

La sintassi più completa di un messaggio è:

```
valore = messaggio(parametro: Tipo): TipoRitornato
```

Sono valide anche forme più sintetiche, come `getProduct()`, `d = getProduct(id)` o `d = getProduct(id: ItemID): Product`. La precisione va scelta in base allo scopo del diagramma.

10.1.2.1 Dal diagramma al codice

Le chiamate annidate del diagramma corrispondono direttamente alla struttura del seguente pseudocodice. I ritorni e i parametri non interessanti sono stati soppressi nel diagramma.

```
class Register {  
    private Sale s = new Sale();  
  
    void doX() {  
        s.doA();  
        s.doB();  
        s.doC(this);  
    }  
  
    void doD() { /* ... */ }  
}  
  
class Sale {  
    void doA() { /* ... */ }  
    void doB() { /* ... */ }  
    void doC(Register register) {  
        register.doD();  
    }  
}
```

10.2 Ciclo di vita dei partecipanti

Un oggetto creato durante l'interazione compare all'altezza del messaggio di creazione, non all'inizio del diagramma. Una X termina la lifeline e rappresenta distruzione esplicita oppure il momento dal quale l'oggetto non serve più.

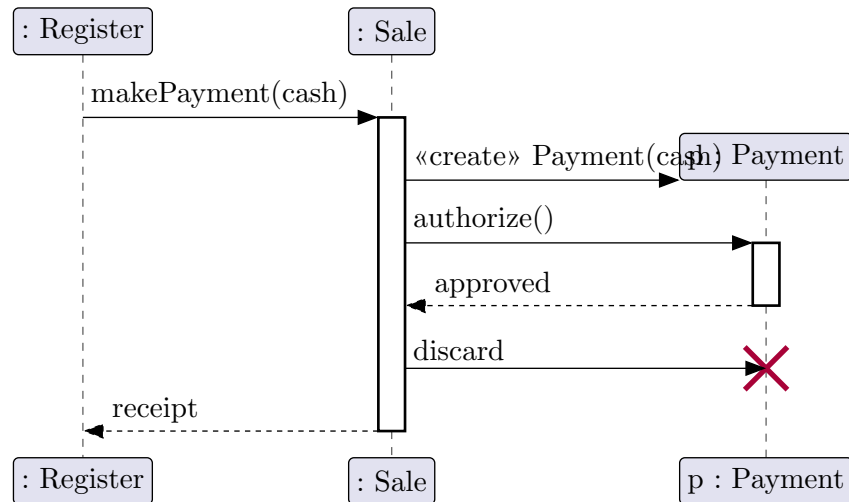


Figura 46: Creazione, uso e fine della lifeline di un pagamento.

Nei linguaggi con garbage collector la distruzione non equivale necessariamente alla liberazione immediata della memoria: segnala soprattutto la fine logica del ruolo dell'oggetto nell'interazione.

10.3 Frame di interazione

I frame racchiudono una parte del diagramma e ne specificano la logica di controllo. L'operatore appare nell'angolo superiore; una guardia opzionale precisa la condizione.

10.3.1 Loop, opt e alt

- **loop**: ripete il frammento finché vale la guardia o per gli elementi di una collezione;
- **opt**: esegue il frammento soltanto se la guardia è vera;
- **alt**: sceglie una sola tra più alternative, come **if/else** o **case**.

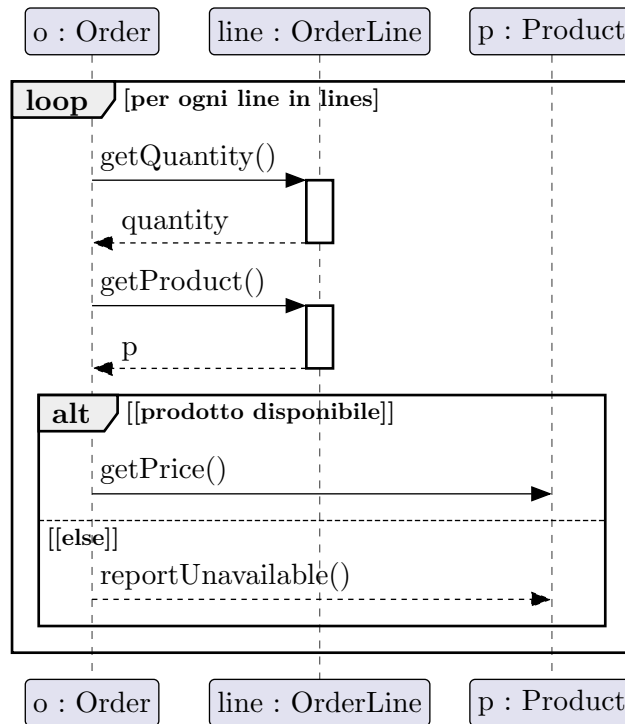


Figura 47: Frame **loop** con un frame **alt** annidato.

Lo stesso controllo può essere espresso più precisamente con pseudocodice. Il Sequence Diagram mette in risalto i destinatari delle chiamate; il codice rende più immediata la logica condizionale.

```
for (OrderLine line : lines) {
    int quantity = line.getQuantity();
    Product product = line.getProduct();

    if (product.isAvailable(quantity))
        total += product.getPrice(quantity);
    else
        product.reportUnavailable();
}
```

10.3.2 Par, ref e sd

par divide il frame in regioni eseguite concorrentemente. **ref** rimanda a un'interazione descritta altrove e permette di scomporre diagrammi troppo grandi; i parametri e i valori di ritorno esplicitano i dati scambiati. **sd** può racchiudere e nominare l'intero Sequence Diagram.

Operatore	Equivalente	Significato
alt	if / else / case	Una sola regione la cui guardia è vera.
opt	if senza else	Il frammento può non essere eseguito.
loop	for / while	Ripetizione del frammento.
par	esecuzione concorrente	Le regioni possono avanzare in parallelo.
ref	chiamata di procedura	Riferimento a un'altra interazione.

Non trasformare il diagramma in codice grafico

I Sequence Diagram rendono molto bene responsabilità e chiamate, ma non sono ideali per algoritmi ricchi di cicli e condizioni. Per logica complessa è spesso più leggibile affiancare pseudocodice o usare diagrammi separati per i diversi scenari.

10.4 Sequence Diagram di sistema

Un **System Sequence Diagram** (SSD) tratta il sistema come una scatola nera. I partecipanti sono gli attori esterni e un unico oggetto **Sistema**; i messaggi rappresentano eventi di input e output osservabili in uno scenario di Use Case.

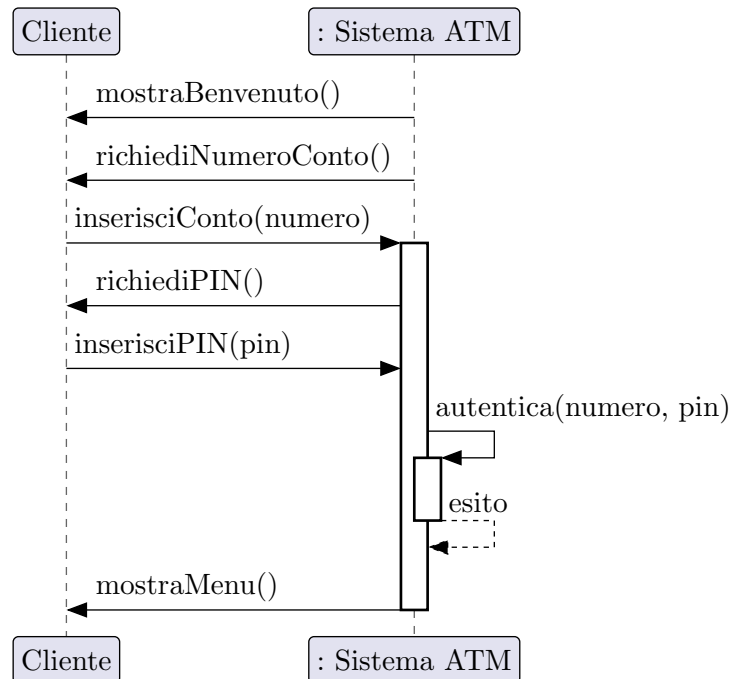


Figura 48: SSD dello scenario principale di autenticazione presso un ATM.

L'SSD serve a precisare operazioni di input, parametri e dati restituiti; aiuta a derivare test e contratti delle operazioni. Non mostra ancora gli oggetti interni che realizzano il comportamento: il passaggio da requisiti a design consiste proprio nell'assegnare queste responsabilità.

SSD e Sequence Diagram di design

Nell'SSD il sistema è un solo partecipante opaco. Nel design quel partecipante viene aperto e sostituito da controller, entità e servizi che collaborano per realizzare gli stessi eventi di sistema.

10.5 Sequence Diagram nel design

Nel Low-Level Design il diagramma mostra quali oggetti realizzano un'operazione e in quale ordine collaborano. Deve essere coerente con il Class Diagram: ogni messaggio deve corrispondere a un'operazione disponibile e ogni collegamento deve essere strutturalmente possibile.

10.5.1 Esempio del registro di cassa

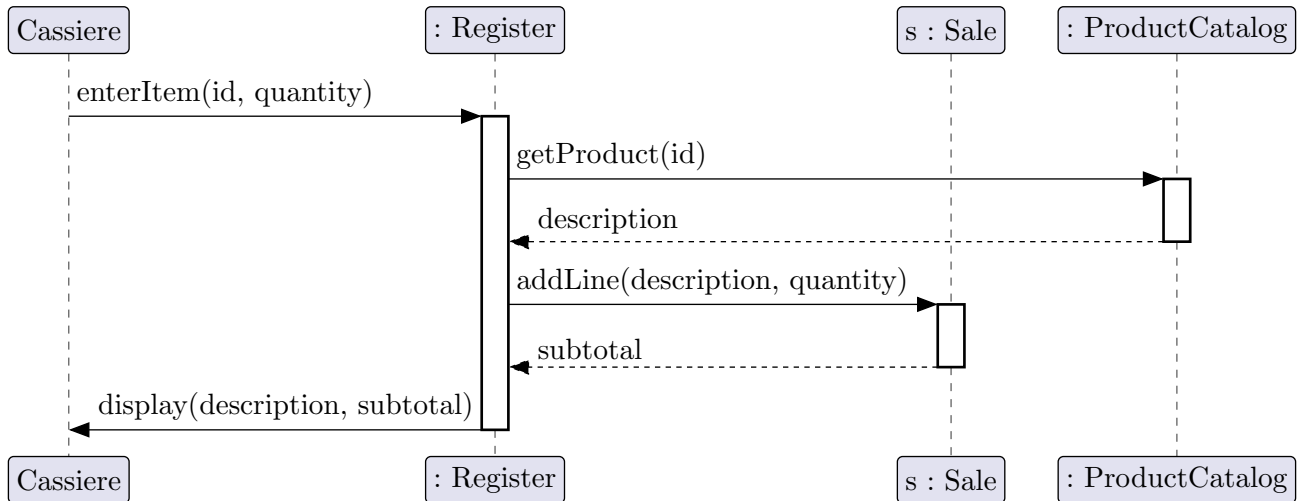


Figura 49: Collaborazione interna che realizza l'evento di sistema `enterItem`.

Il diagramma aiuta a distribuire responsabilità, ma non determina automaticamente il design. Più collaborazioni possono soddisfare lo stesso SSD; vanno confrontate per coesione, accoppiamento e comprensibilità.

10.6 Esempio: prezzo di un ordine

Un ordine contiene linee con quantità e prodotto. Il prezzo totale scontato è la somma dei prezzi delle linee meno uno sconto globale dipendente dal cliente.

$\text{prezzo} = \text{somma}(\text{quantità} \times \text{prezzoUnitario}) - \text{scontoCliente}$

10.6.1 Soluzione centralizzata

Nella prima soluzione `Ordine` coordina ogni dettaglio: legge quantità e prodotto dalle linee, interroga il prodotto, calcola importi, totale e sconto.

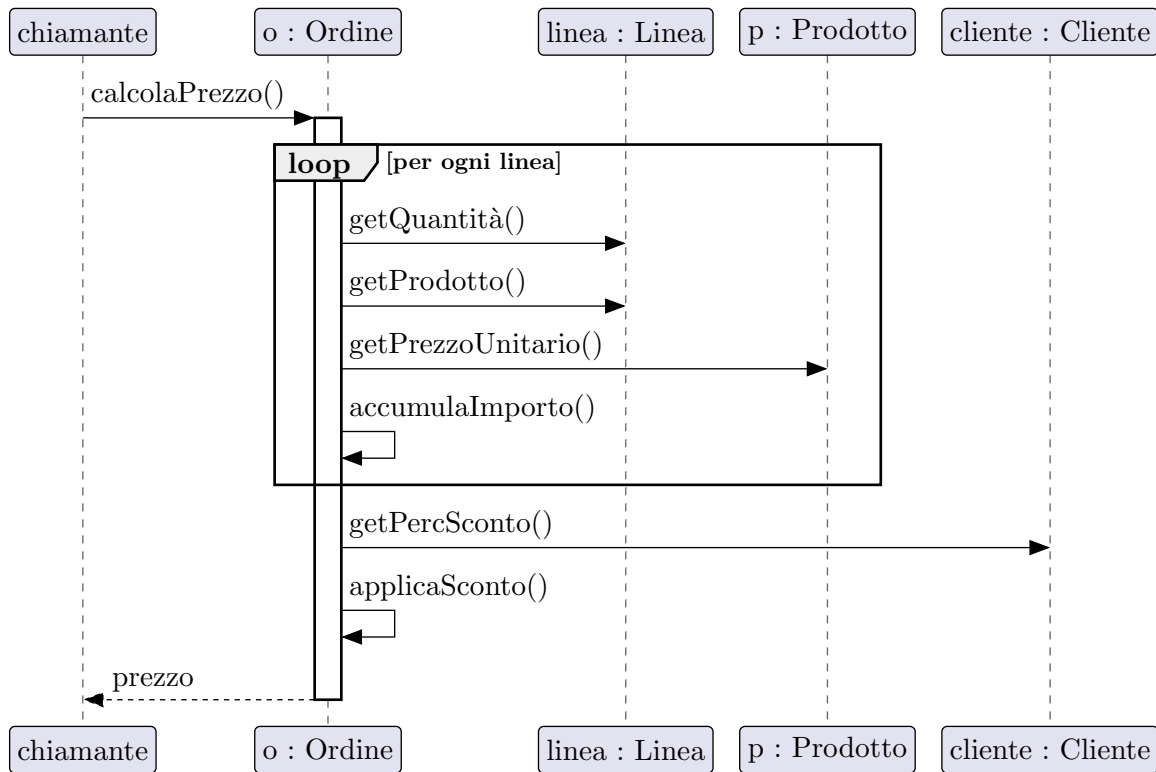


Figura 50: Controllo centralizzato: `Ordine` contiene quasi tutta la logica.

La soluzione funziona, ma rende `Ordine` poco coeso e molto dipendente dai dettagli delle altre classi. Con l'aumentare delle responsabilità può trasformarsi in una **God Class** o **Blob**. Il codice conferma ciò che emerge dal diagramma: `Ordine` estrae i dati dagli altri oggetti e svolge direttamente quasi tutta l'elaborazione.

10.6.1.1 Pseudocodice della soluzione centralizzata

```
class Ordine {
    Set<Linea> linee;
    Cliente cliente;

    Euro calcolaPrezzo() {
        Euro totale = 0;
        for (Linea linea : linee) {
            int quantità = linea.getQuantità();
            Prodotto prodotto = linea.getProdotto();
            Euro unitario = prodotto.getPrezzoUnitario();
            linea.setImporto(quantità * unitario);
            totale += linea.getImporto();
        }
        int percentuale = cliente.getPercSconto();
        Euro sconto = totale * percentuale / 100;
        return totale - sconto;
    }
}
```

10.6.2 Soluzione distribuita

Nella seconda soluzione ogni oggetto gestisce i dati che possiede: la linea calcola il proprio prezzo, il prodotto applica la politica relativa alla quantità e il cliente calcola il totale scontato.

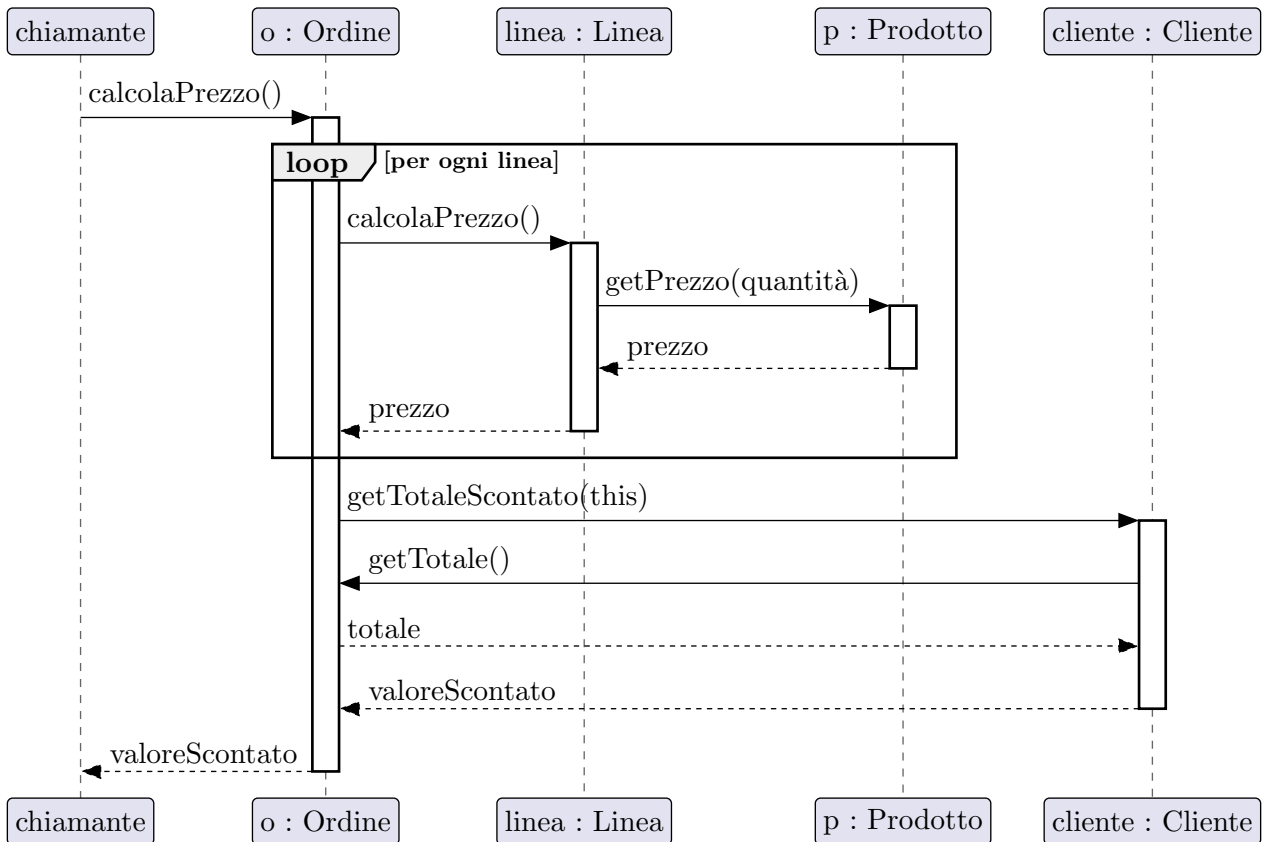


Figura 51: Controllo distribuito: dati e operazioni correlate restano nella stessa classe.

La soluzione distribuita tende a essere preferibile perché aumenta la coesione, limita le dipendenze sui dati interni e riduce il rischio di classi onniscenti. Non significa però distribuire ogni singola operazione: serve un controller che coordini il caso d'uso senza assorbitne tutta la business logic.

10.6.2.1 Pseudocodice della soluzione distribuita

```
class Ordine {
    Set<Linea> linee;
    Cliente cliente;
    Euro totale;

    Euro calcolaPrezzo() {
        totale = 0;
        for (Linea linea : linee)
            totale += linea.calcolaPrezzo();
        return cliente.getTotaleScontato(this);
    }

    Euro getTotale() { return totale; }
}

class Linea {
    Prodotto prodotto;
    int quantità;

    Euro calcolaPrezzo() {
        return prodotto.getPrezzo(quantità);
    }
}

class Cliente {
    int percentualeSconto;

    Euro getTotaleScontato(Ordine ordine) {
        Euro totale = ordine.getTotale();
        return totale - totale * percentualeSconto / 100;
    }
}
```

Ora ogni metodo opera principalmente sui dati della propria classe: **Linea** conosce quantità e prodotto, mentre **Cliente** conosce la politica di sconto.

10.7 Communication Diagram

Un Communication Diagram rappresenta la stessa interazione come rete di oggetti collegati. La sequenza temporale non deriva dalla posizione verticale, ma dalla numerazione dei messaggi.

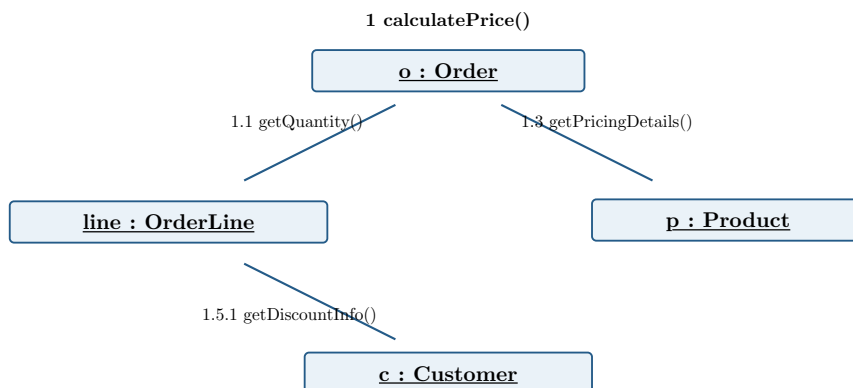


Figura 52: Nel Communication Diagram la numerazione annidata ricostruisce le chiamate.

La numerazione progressiva 1, 1.1, 1.2, 1.5.1 mostra l'annidamento: 1.5.1 avviene dentro il messaggio 1.5, a sua volta dentro 1. Senza numerazione annidata si perde l'informazione sulla struttura delle chiamate.

Aspetto	Sequence Diagram	Communication Diagram
Enfasi	Ordine temporale.	Struttura dei collegamenti.
Ordine	Posizione verticale.	Numerazione dei messaggi.
Punto forte	Chiamate e attivazioni annidate.	Rete delle collaborazioni.
Limite	Può diventare molto alto.	Il tempo è meno immediato.

10.7.1 Linee guida

1. Dichiarare se il diagramma è un SSD di requisiti o un diagramma di design.
2. Modellare uno scenario o una responsabilità precisa, non l'intero sistema.
3. Mantenere il diagramma coerente con operazioni e navigabilità del Class Diagram.
4. Mostrare ritorni, tipi e frame soltanto quando aggiungono informazione utile.
5. Usare pseudocodice per algoritmi complessi e diagrammi separati per alternative importanti.
6. Cercare responsabilità distribuite e coese, evitando controller che incorporano tutta la logica.

Regola conclusiva

Il valore di un Sequence Diagram non è mostrare ogni chiamata: è rendere evidente chi conosce chi, chi svolge ciascuna responsabilità e come uno scenario attraversa gli oggetti del design.

11 Activity Diagram

Un **Activity Diagram** descrive come viene svolta un'attività: quali azioni avvengono, in quale ordine, quali dati attraversano il processo e quali flussi possono procedere in concorrenza. L'attività modellata può essere un'operazione, un caso d'uso, un processo di business, una procedura o un algoritmo.

Diagrammi poliedrici

Gli Activity Diagram si usano in analisi per workflow e Use Case, nel design come alternativa al pseudocodice e, nei profili UML eseguibili, persino come linguaggio di programmazione.

11.1 Concetti fondamentali

11.1.1 Azioni, nodi e flussi

Un'attività è composta da **azioni**, unità di comportamento che costituiscono i suoi mattoni. Il testo dell'azione può essere informale, può indicare un'operazione o il richiamo a un'altra attività, oppure può contenere pseudocodice quando il diagramma è vicino all'implementazione.

Categoria	Elemento	Funzione
Azione	Action node	Esegue un comportamento atomico rispetto al diagramma.
Dati	Object node / pin	Rappresenta input, output e oggetti che fluiscono.
Controllo	Initial/final	Avvia o termina attività e flussi.
Controllo	Decision/merge	Seleziona o riunisce alternative.
Controllo	Fork/join	Avvia e sincronizza flussi concorrenti.

Un piccolo simbolo a rastrello (**rake**) segnala che un'azione è dettagliata da un altro Activity Diagram. È un meccanismo di decomposizione: il diagramma corrente mostra il livello alto, quello richiamato descrive i passi interni.



Figura 53: Azioni espresse in linguaggio naturale, tramite richiamo, operazione e pseudocodice.

Nel secondo nodo il rastrello indica il richiamo a un'attività separata. Per esempio **Conferma ordine ↘** può essere dettagliata da un diagramma contenente **Fornisci dati di pagamento** e **Fornisci dati di spedizione**.

11.1.2 Semantica dei token

La semantica UML è basata su **token**, simili a pedine che percorrono gli archi:

- un'azione è abilitata quando possiede token su tutti gli ingressi richiesti;
- l'esecuzione consuma i token in entrata;
- al termine vengono prodotti token sugli archi in uscita;
- decision, fork, join e nodi finali trasformano o eliminano i token secondo regole precise.

11.2 Esempio: gestione di un ordine

Quando l'azienda riceve un ordine, avvia due flussi concorrenti: prepara e spedisce il pacco; invia la fattura e attende il pagamento. L'ordine viene chiuso soltanto quando entrambi sono terminati.

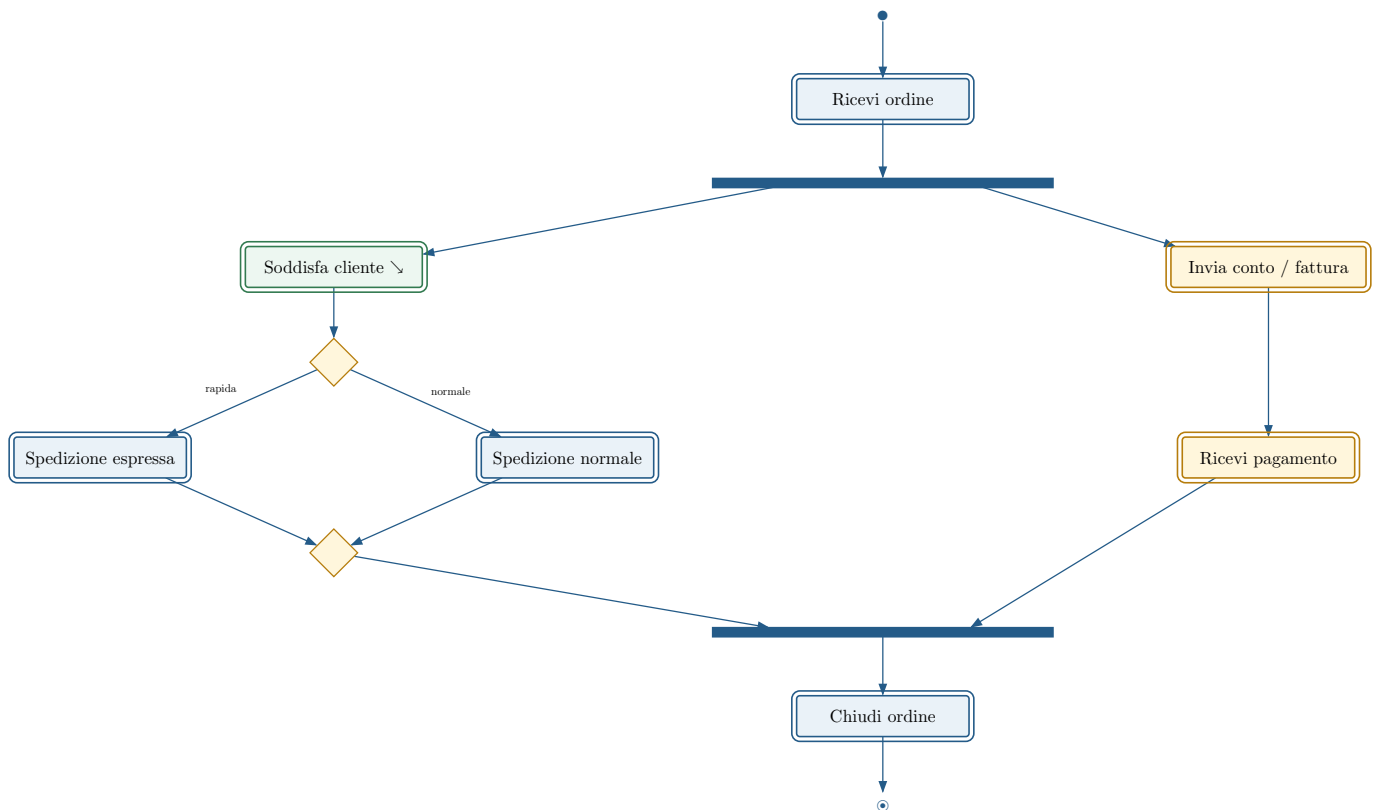


Figura 54: Fork e join coordinano i due rami concorrenti della gestione ordine.

Il fork duplica il token ricevuto: un token entra nel ramo logistico e uno in quello amministrativo. Il join attende un token da entrambi; perciò **Chiudi ordine** non può iniziare finché pacco e pagamento non sono completati.

11.2.1 Evoluzione dei token, passo per passo

Le slide mostrano lo stesso diagramma in istanti successivi. La sequenza completa è:

1	Il nodo iniziale abilita Ricevi ordine .
2	Ricevi ordine deposita il token sul fork.
3	Il fork duplica il token sui due rami.
4	Soddisfa cliente prepara il pacco mentre viene inviata la fattura.
5	Il decision sceglie spedizione espressa oppure normale.
6	Il merge riunisce le alternative di spedizione.
7	Ricevi pagamento attende il cliente.
8	Il join trattiene il primo token arrivato.
9	Con entrambi i token, il join abilita Chiudi ordine .
10	Il nodo finale termina l'attività e distrugge gli altri token.

11.3 Nodi di controllo

11.3.1 Inizio e terminazione

Il nodo iniziale genera il primo token. Quando un token raggiunge un **activity final**, termina l'intera attività e vengono eliminati anche i token ancora presenti negli altri rami. Un **flow final** termina invece soltanto il flusso che lo raggiunge.

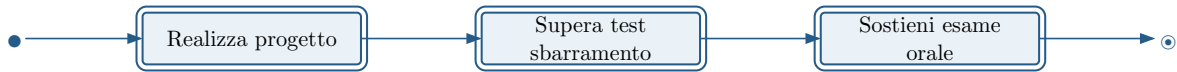


Figura 55: Il primo nodo finale raggiunto termina l'intera attività e tutti i suoi flussi.

11.3.2 Decisione e fusione

Un nodo decisione ha un ingresso e più uscite dotate di guardie. Il token percorre una sola uscita la cui guardia è vera. Guardie sovrapposte producono una scelta non deterministica; se nessuna è vera, il flusso si blocca. Il merge ha più ingressi alternativi e un'unica uscita.

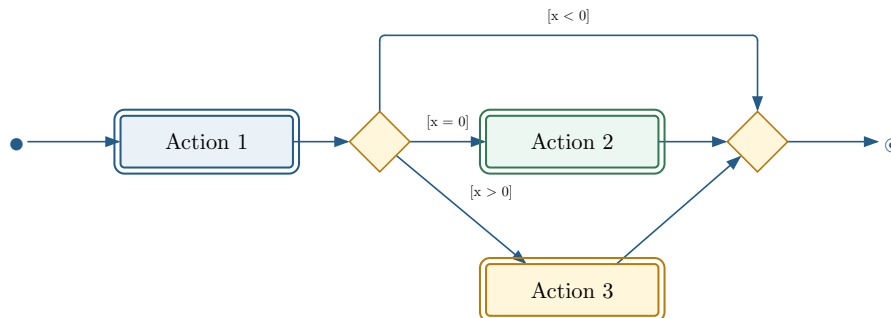


Figura 56: Il decision instrada il token; il merge riunisce i percorsi alternativi.

11.3.3 Fork e join

Un fork ha un ingresso e più uscite: duplica il token su tutti i rami. Un join ha più ingressi e una sola uscita: attende un token su ogni ingresso prima di procedere. Le attività nei rami sono concorrenti; non è necessario che siano implementate da thread distinti.

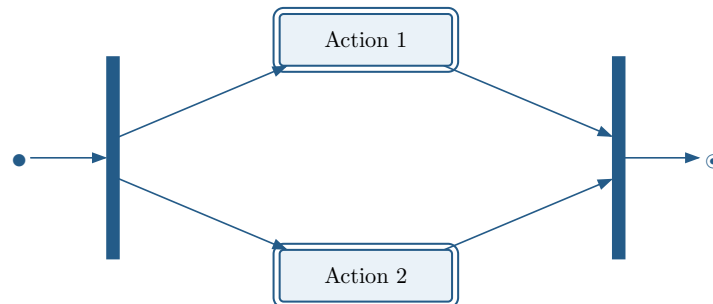


Figura 57: Il fork duplica il token; il join attende il completamento di entrambi i rami.

11.3.4 Nodo finale di flusso

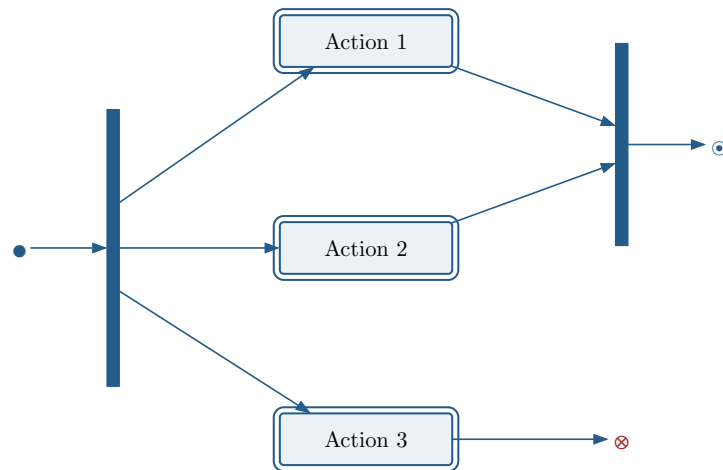


Figura 58: Action 3 termina soltanto il proprio flusso; le altre due terminano l'attività tramite il join.

11.4 Oggetti, pin e stati

Gli **object node** modellano oggetti usati come input o prodotti come output. Possono apparire tra due azioni oppure come piccoli **pin** sul bordo dell'azione. In prospettiva concettuale rappresentano oggetti del dominio; in prospettiva software, istanze a runtime.

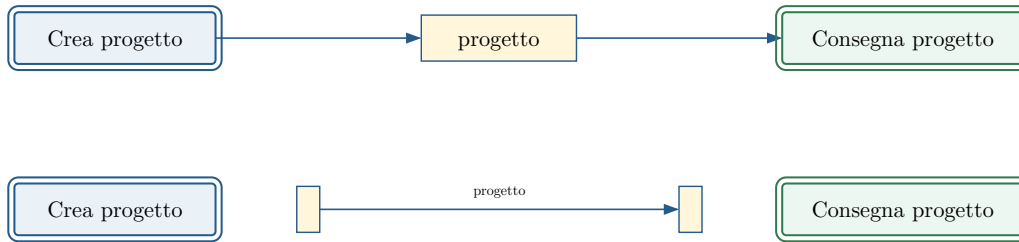


Figura 59: Un oggetto può essere mostrato come nodo autonomo oppure tramite pin sulle azioni.

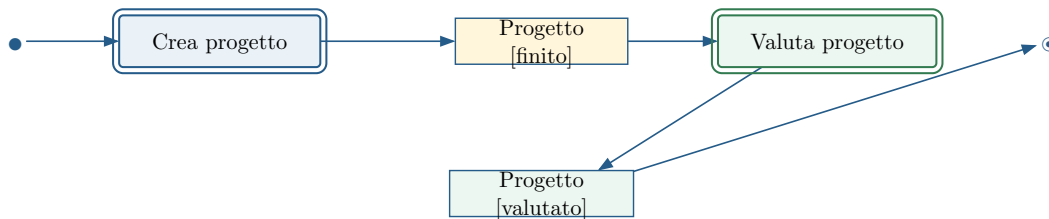


Figura 60: Gli stati dell'oggetto rendono visibile la sua evoluzione durante l'attività.

Gli stati tra parentesi quadre devono essere coerenti con l'eventuale State Machine dell'oggetto. In termini di pseudocodice, il primo flusso equivale a:

```
Progetto progetto = creaProgetto();  
consegnaProgetto(progetto);
```

I parametri di input e i valori restituiti di un'intera attività vengono rappresentati come object node sul bordo esterno dell'attività.

11.5 Partizioni e responsabilità

Le **swimlane** specificano chi esegue ogni azione. Suddividono graficamente il flusso senza cambiarne la semantica; possono rappresentare persone, ruoli, reparti, componenti o classi.

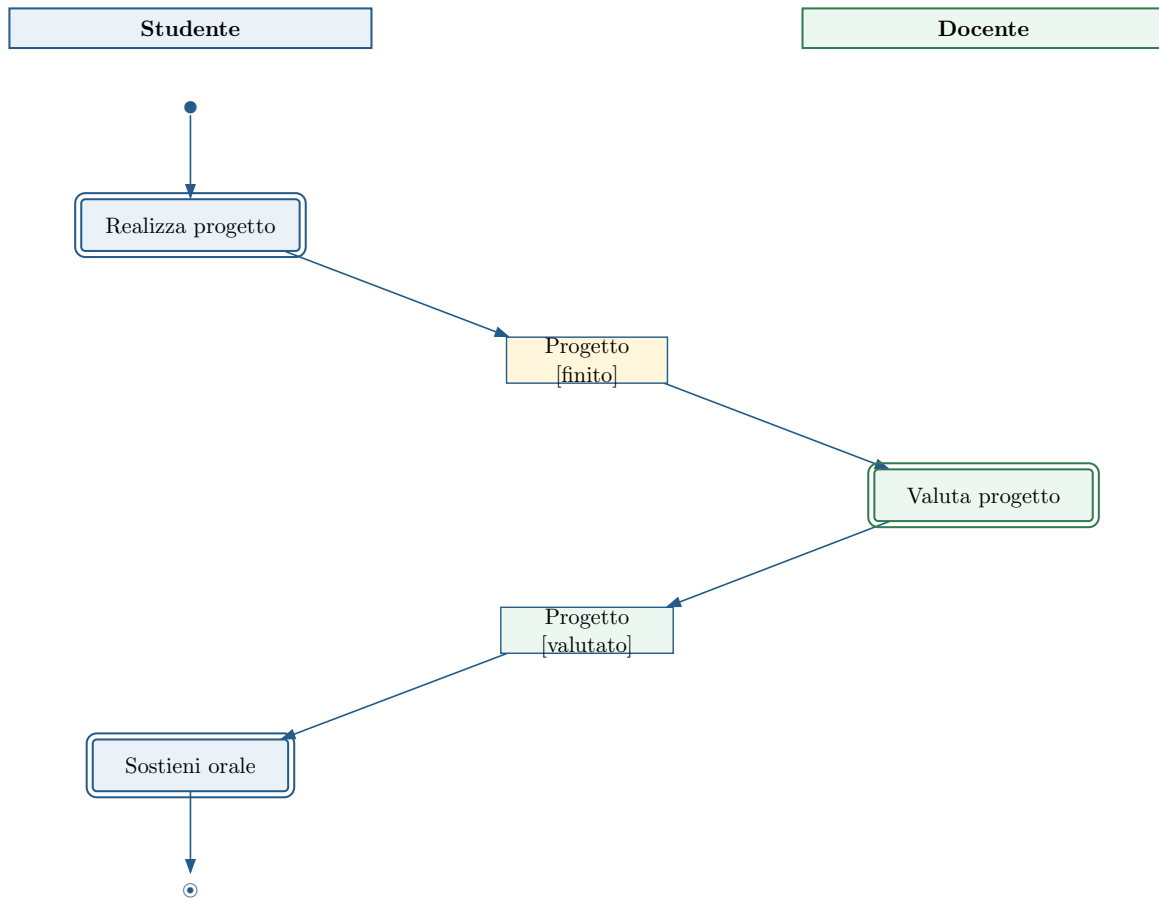


Figura 61: Il flusso attraversa le responsabilità di studente e docente.

Swimlane e concorrenza

Due azioni in lane differenti non sono automaticamente parallele. La concorrenza nasce dalla struttura fork/join, non dalla disposizione nelle partizioni.

11.6 Eventi e interruzioni

11.6.1 Invio e ricezione di eventi

UML offre azioni specializzate per inviare e ricevere segnali, compresi eventi temporali. L'invio è asincrono e non blocca; la ricezione sospende il flusso finché l'evento non si verifica. Un nodo di ricezione senza archi entranti rimane attivo per tutta la vita dell'attività.



Figura 62: Azioni specializzate per invio, ricezione e attesa temporale.

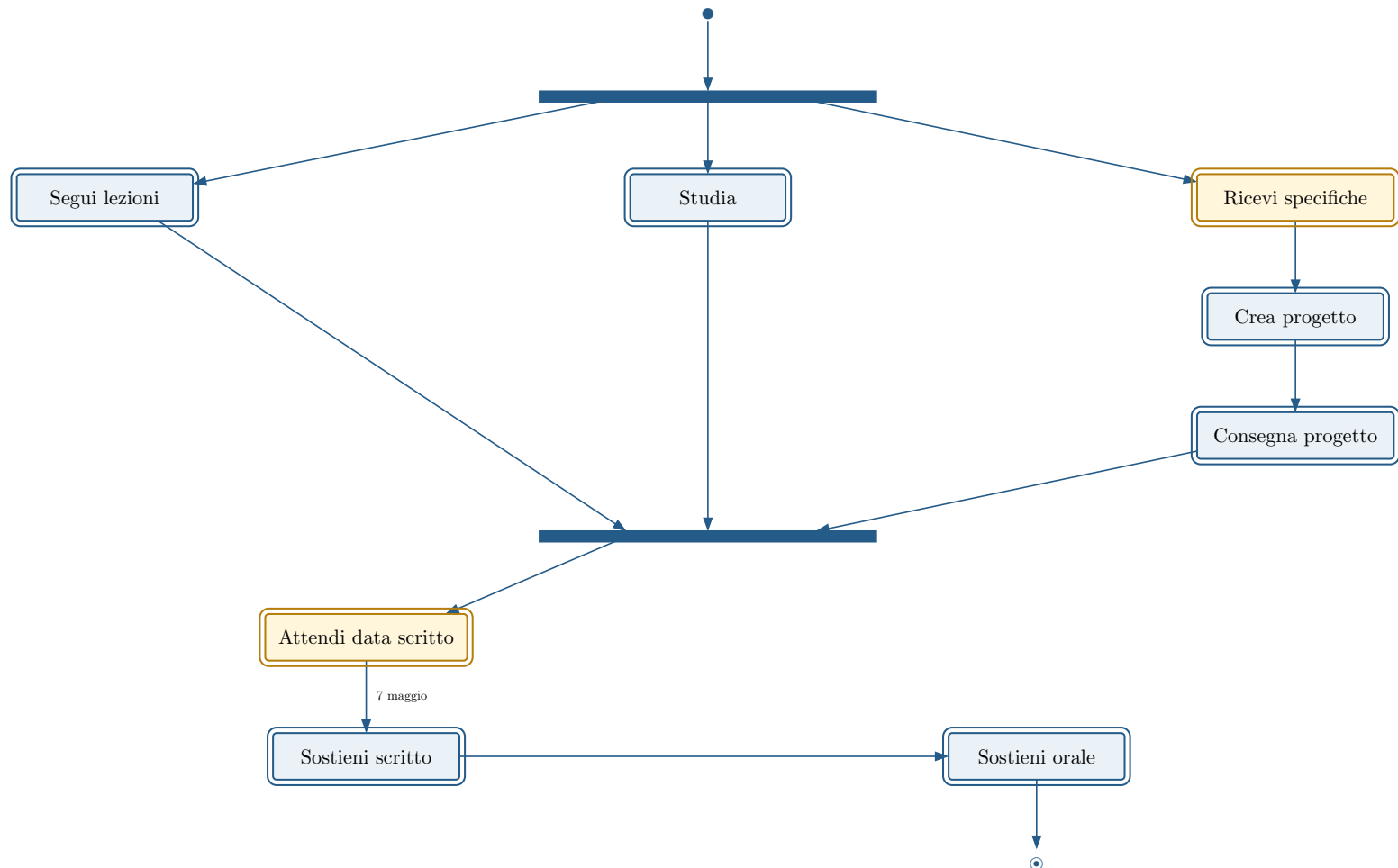


Figura 63: Ricezione di specifiche ed evento temporale nel processo d'esame.

11.6.2 Regioni interrompibili

Una regione interrompibile delimita azioni che possono essere cancellate da un evento. Se un token si trova nella regione quando arriva il segnale di interruzione, i flussi interni vengono terminati e il controllo passa lungo l'arco di interruzione. Si usa per eccezioni, cancellazioni, timeout e condizioni di errore.

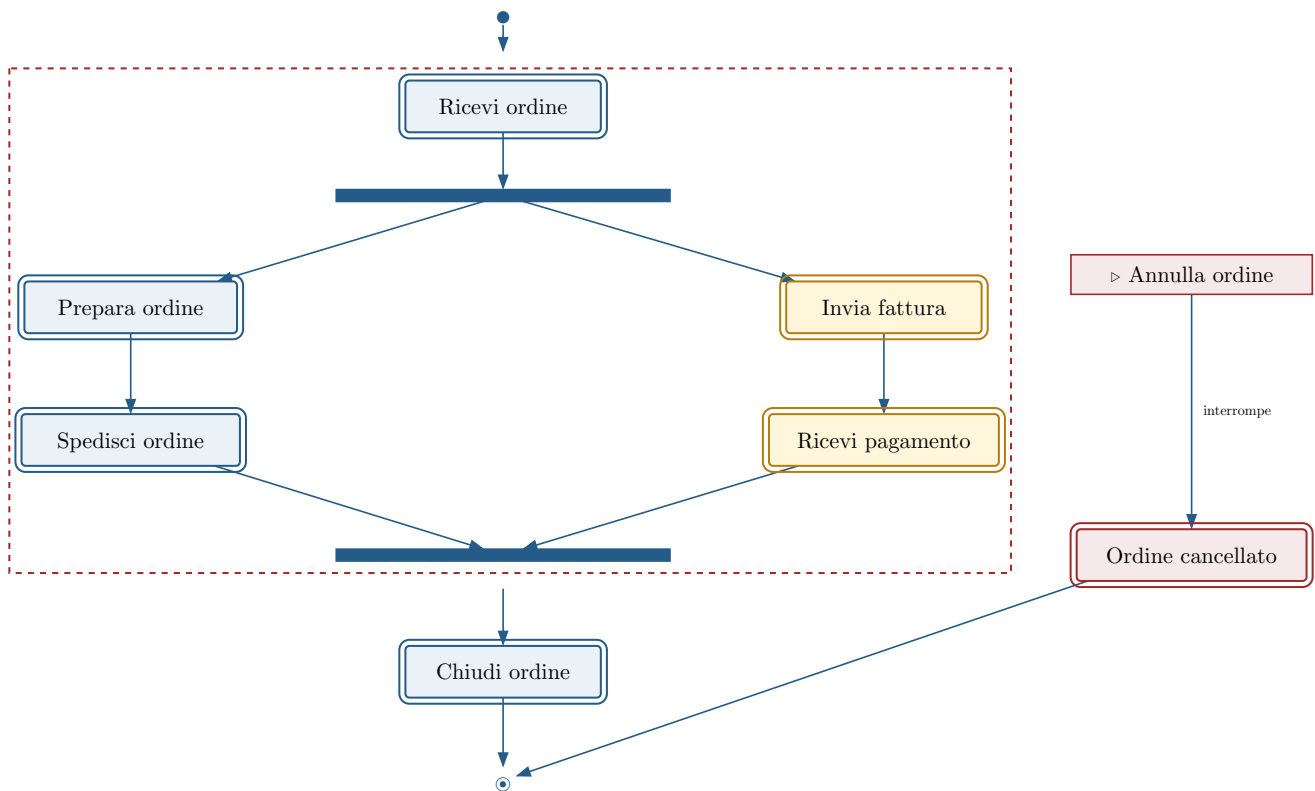


Figura 64: La cancellazione interrompe i token nella regione, ma non è più ammessa dopo la spedizione.

Confine temporale

Nel processo d'ordine la cancellazione può essere ammessa durante preparazione e pagamento, ma non dopo la consegna: il confine della regione rende esplicito fino a quando l'interruzione è valida.

11.6.3 Parametri e valori di ritorno: fare il bucato

I parametri di input e i valori restituiti dell'intera attività sono object node posti sul bordo. Nell'esempio `dirtyClothes: Clothes` entra nell'attività e `cleanClothes: Clothes` ne costituisce il risultato.

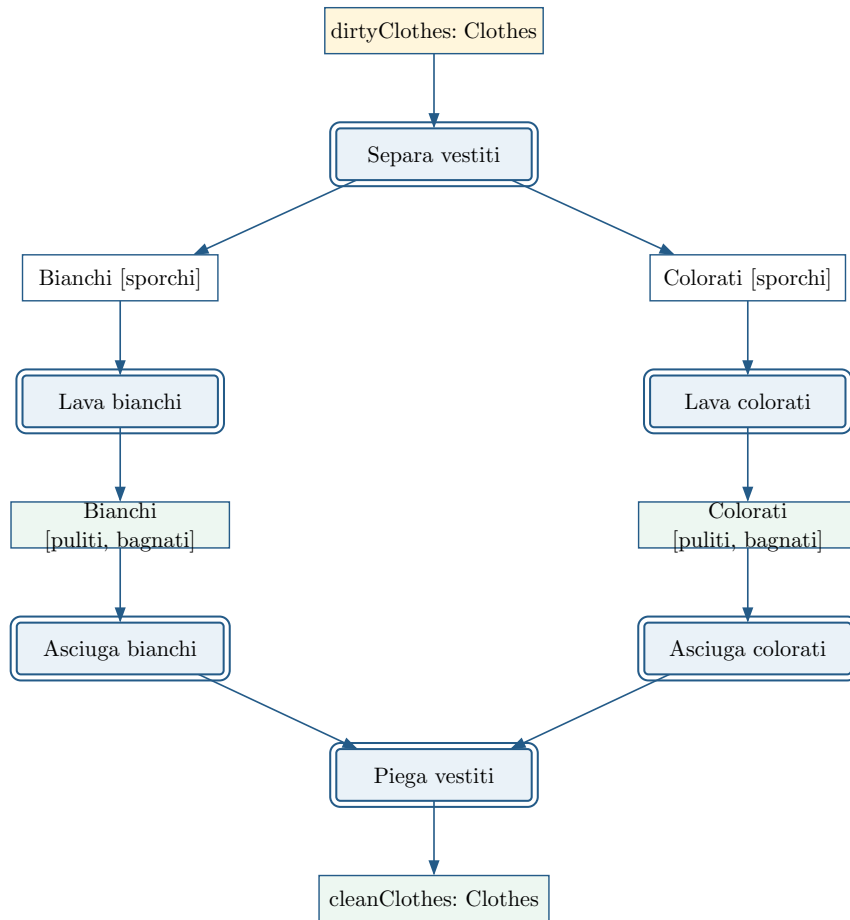


Figura 65: Parametri, object node e trasformazioni dell'attività Fare bucato.

11.7 Modellare Use Case e operazioni

11.7.1 Use Case ATM: prelevare contante

L'Activity Diagram rende espliciti il percorso principale e la ripetizione richiesta quando saldo o disponibilità dello sportello non sono sufficienti.

Attore primario: Cliente. Il sistema:

1. visualizza 20, 40, 60, 100, 200 e cancel;
2. riceve l'importo selezionato tramite tastiera;
3. se importo, saldo e contante nello sportello sono compatibili, addebita il conto, eroga le banconote e ricorda di ritirarle;
4. altrimenti mostra l'errore e torna al menu;
5. tratta `cancel` come scenario alternativo di annullamento.

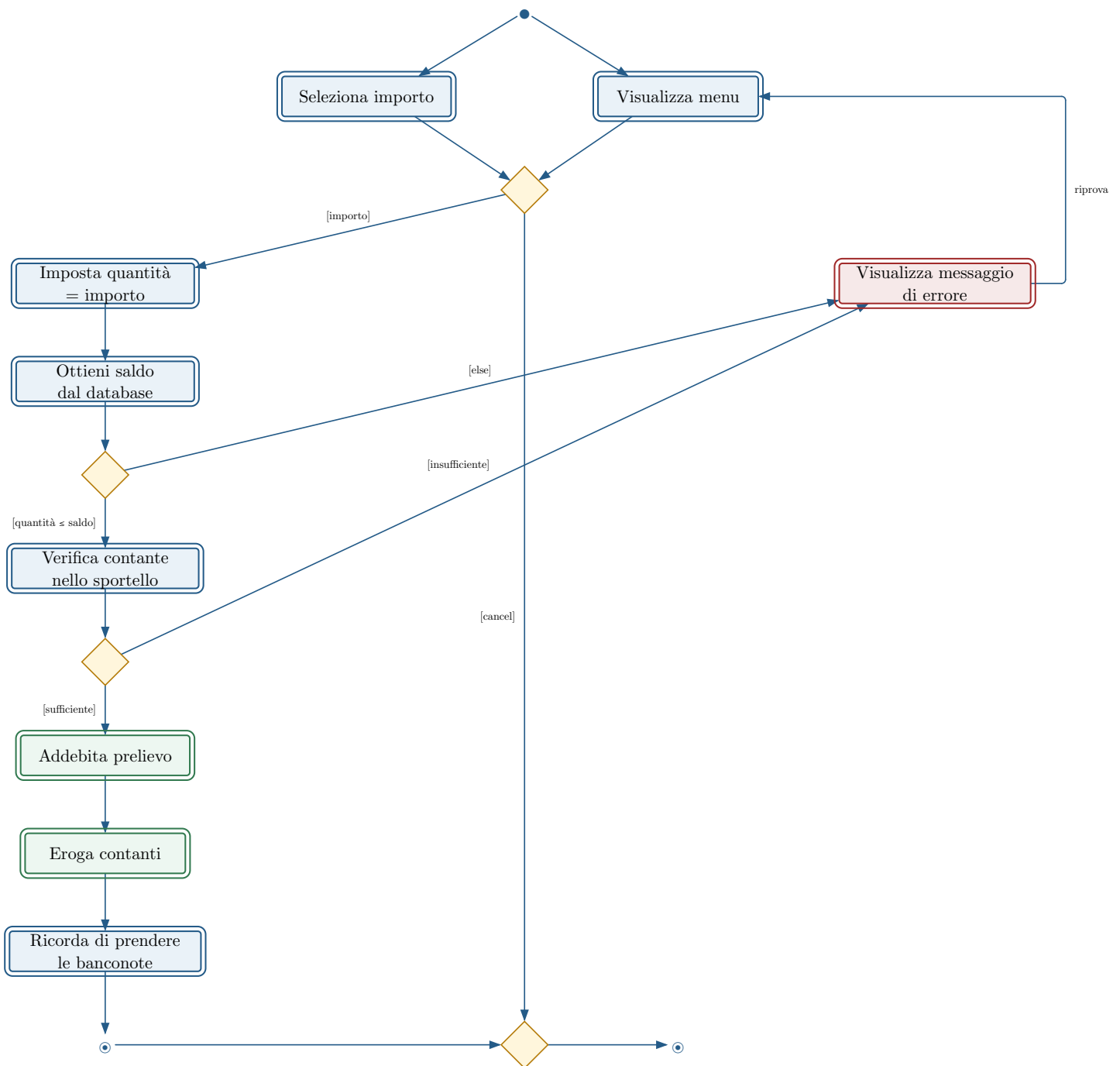


Figura 66: Use Case Prelevare contante: cancel, controllo del saldo e disponibilità dello sportello.

11.7.2 Operazione `authenticateUser`

L'esempio delle slide modella l'operazione del bancomat `authenticateUser(login: string, PWD: string): Answer`, dove `Answer` può assumere `invalidUser`, `incorrectPWD` oppure `ok`.

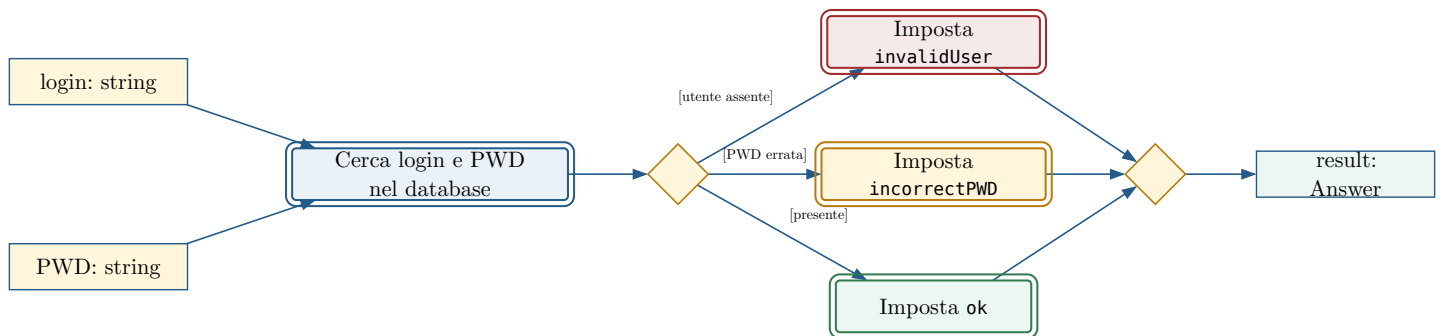


Figura 67: Parametri e valore restituito dell'operazione `authenticateUser`.

Operazione	<code>authenticateUser(login: string, PWD: string): Answer</code>
Risultati	<code>invalidUser</code> , <code>incorrectPWD</code> , <code>ok</code>
Passo centrale	Ricerca di login e password nel database.

11.8 Modellare un algoritmo

11.8.1 Ricerca binaria

La ricerca binaria combina decision e merge per aggiornare gli estremi dell'intervallo finché il valore centrale coincide con il target oppure l'intervallo diventa vuoto.

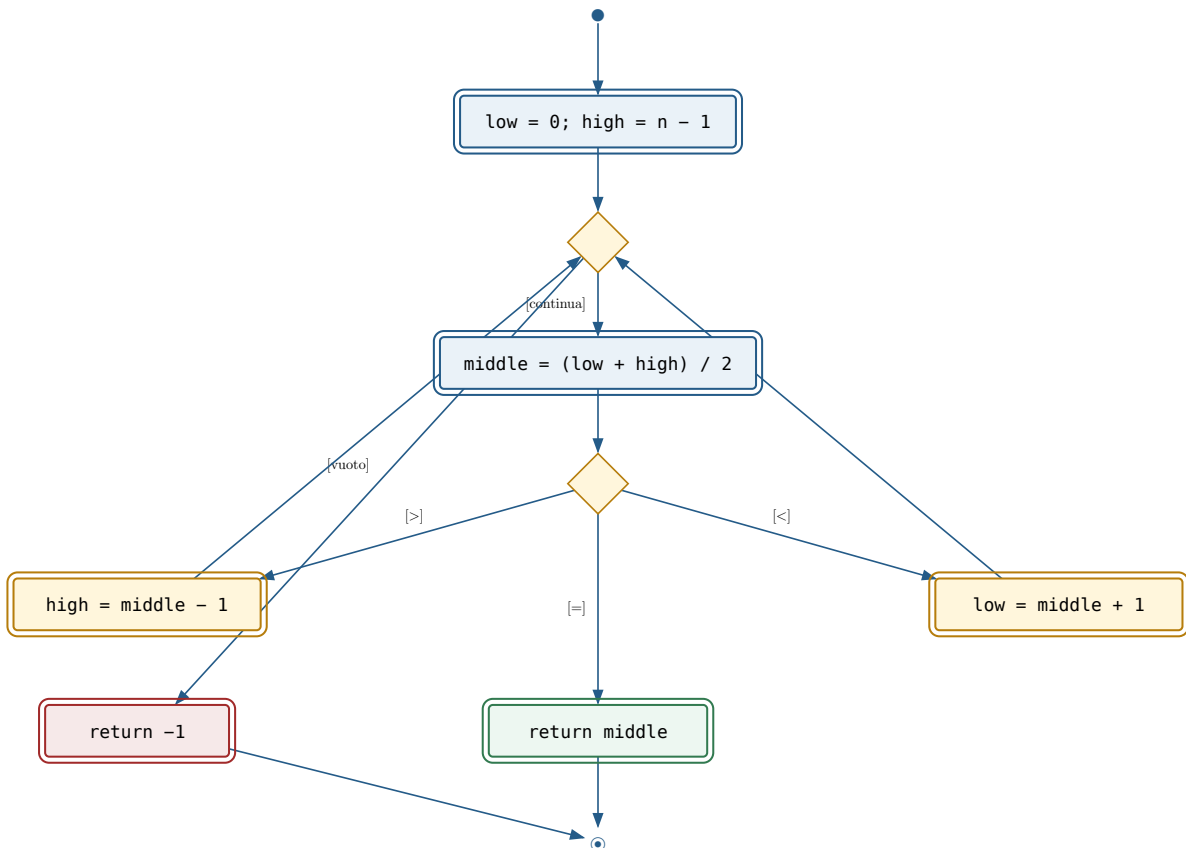


Figura 68: Activity Diagram dell'algoritmo di ricerca binaria.

```
int binarySearch(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int middle = (low + high) / 2;
        if (a[middle] == target) return middle;
        if (a[middle] < target) low = middle + 1;
        else high = middle - 1;
    }
    return -1;
}
```

Nel caso mostrato nelle slide, con `target = 76`, l'intervallo si restringe così:

Passo	low	middle	high	Confronto
1	2	47	94	47 < 76
2	51	77	94	77 > 76
3	51	64	76	64 < 76
4	76	76	76	76 = 76

12 Component, Deployment e Package Diagram

Component, Deployment e Package Diagram sono viste strutturali usate per scopi differenti. Il primo descrive l'architettura software, il secondo collega software e hardware, il terzo organizza elementi UML e dipendenze.

Tre viste complementari

Il Component Diagram ragiona per unità software sostituibili; il Deployment Diagram mostra dove tali unità vengono eseguite o installate; il Package Diagram tiene sotto controllo la struttura logica del modello e del codice.

12.1 Component Diagram

12.1.1 Concetto di componente

Il significato concreto di **componente** varia tra piattaforme e ambienti — Java, .NET, servizi web o microservizi — ma UML adotta una definizione generale: un'unità modulare con interfacce ben definite, sostituibile nel proprio ambiente.

Un componente è quindi una **scatola nera** il cui comportamento esterno è completamente definito dalle interfacce. Componenti compatibili possono essere rimpiazzati e composti, come mattoncini LEGO, mediante infrastruttura e **glue code**.

Sviluppo basato sui componenti

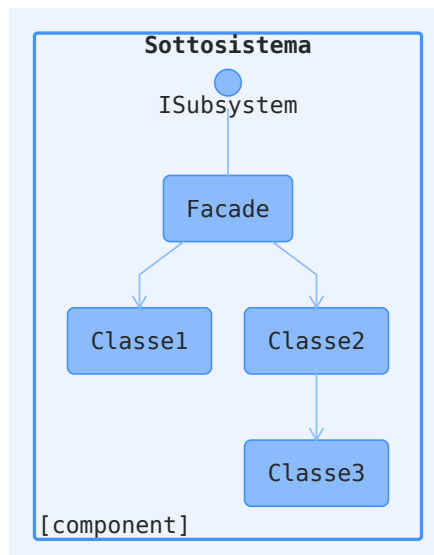
Componenti provenienti da marketplace o team differenti vengono collegati attraverso un composition framework. Il valore non è soltanto il riuso del codice, ma la possibilità di assemblare sistemi funzionanti a partire da contratti stabili.

12.1.2 Componenti e artefatti

I componenti sono entità **logiche**; gli artefatti sono entità **fisiche** che li realizzano. Esempi delle slide:

- un sottosistema realizzato da classi raccolte in un file JAR;
- un componente WebRatio realizzato mediante JAR;
- un servizio REST implementato con Jersey/JAX-RS;
- un microservizio Spring Boot;
- EmailVerify per .NET distribuito in un package .msi.

Un **sottosistema** è una parte del sistema eseguibile anche autonomamente e offre all'esterno una o più interfacce.



Codice 13: Un sottosistema espone un'interfaccia e nasconde la struttura interna.

12.1.3 Perché componenti e non classi

Il Component Diagram mostra i componenti del sistema e le loro dipendenze ed è quindi adatto all'High-Level Design. Le classi hanno una granularità troppo fine per offrire una panoramica architeturale: guardando i singoli alberi si rischia di perdere di vista il bosco.

12.1.4 Notazione

Un componente è rappresentato da un rettangolo con nome e parola chiave «component», oppure con la specifica icona UML. Stereotipi più precisi includono «Web service», «Microservice» e «Subsystem»; il modello può introdurne altri.

nt>> CorsoIS2

CorsoIS2

<<Web service>> Catalogo

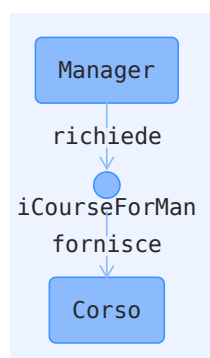
<<Microservice>> Pagamenti

<<Subsystem>> Gestione

Codice 14: Notazione compatta e stereotipi specifici per i componenti.

12.1.5 Interfacce fornite e richieste

Il lollipop indica un'interfaccia fornita; il socket indica un'interfaccia richiesta. Collegandoli si crea un **assembly connector**: il componente cliente usa il contratto implementato dal fornitore.

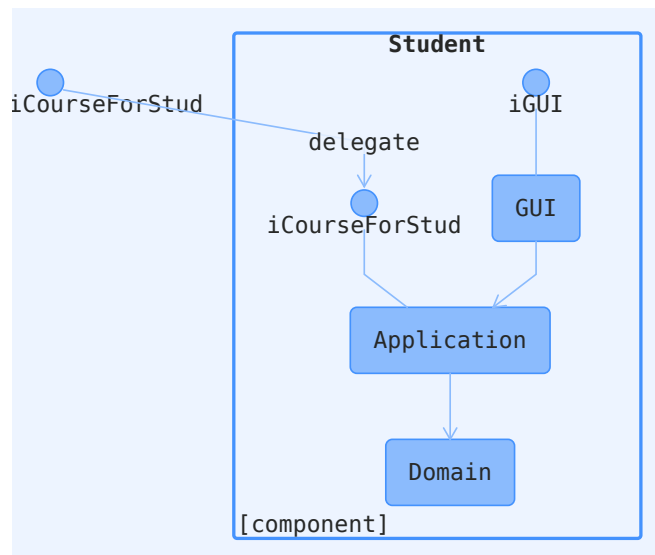


Codice 15: Manager richiede l'interfaccia iCourseForMan, fornita da Corso.

12.1.6 Porte e vista interna

Una **porta** raggruppa un insieme semanticamente coeso di interfacce ed è un punto preciso d'interazione tra esterno e interno. Può avere un nome, per esempio p1, p2 o p3.

La vista interna specifica come il componente è realizzato ed è utile a chi deve implementarlo. I componenti possono essere costruiti ricorsivamente da altri componenti.

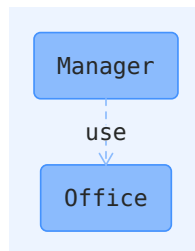


Codice 16: Vista interna ricorsiva con interfacce e porta delegata.

Una descrizione molto prolissa può mostrare contemporaneamente parola chiave, icona, nome, interfacce richieste e fornite, struttura interna, realizzazioni e artefatti. Questa completezza è utile soltanto quando serve davvero all'implementazione.

12.1.7 Dipendenze tra componenti

La dipendenza mantiene il significato UML consueto: il cliente ha bisogno del fornitore per funzionare. Si usa spesso quando esistono più interfacce ma non interessa elencarle; altrimenti si preferiscono interfacce fornite e richieste.

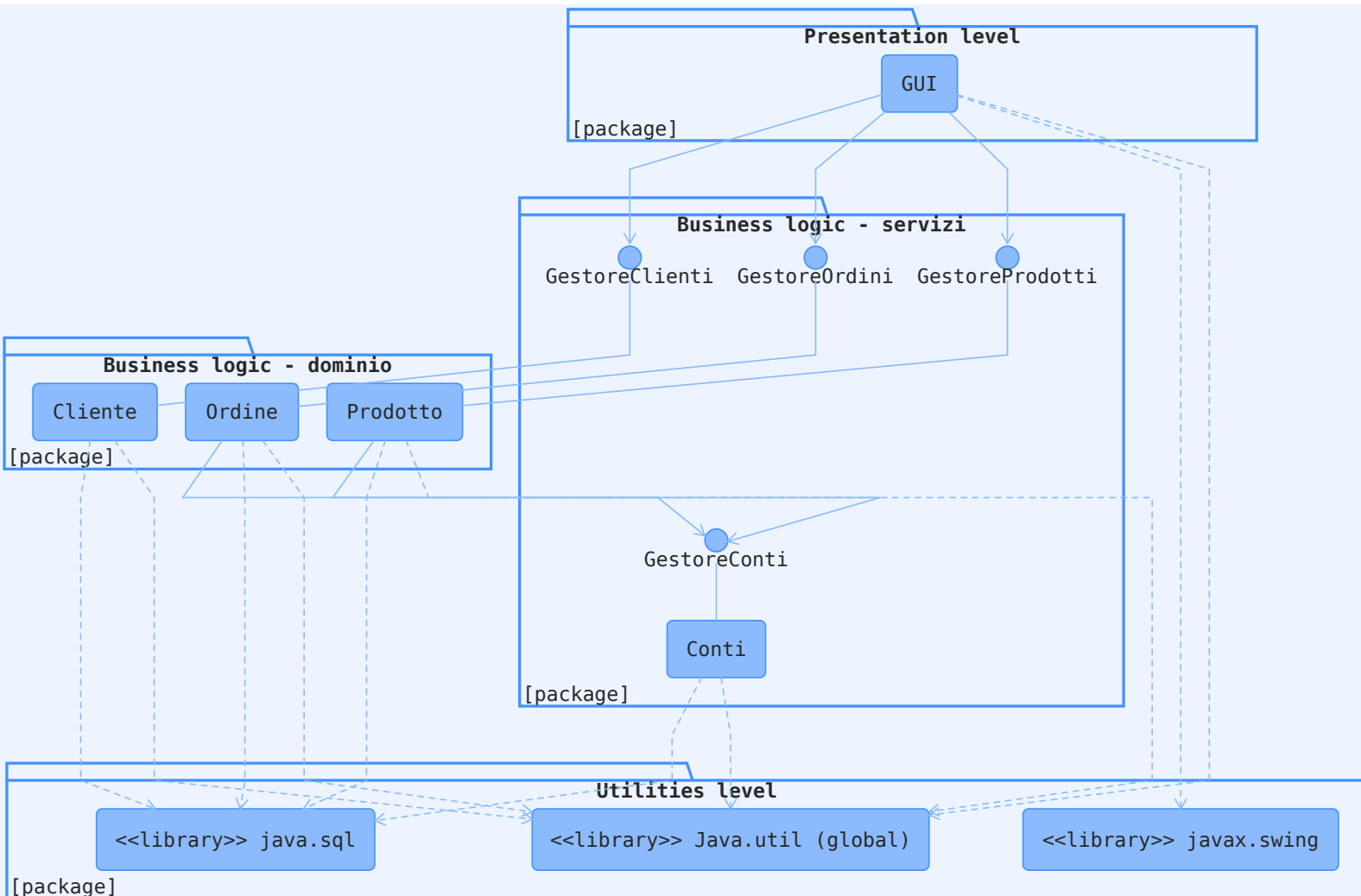


Codice 17: Manager dipende da Office.

12.2 Esempio: sistema di gestione ordini

Il sistema è organizzato in tre livelli concettuali, con la business logic ulteriormente divisa in dominio e servizi:

1. **Presentation logic:** GUI;
2. **Business logic — dominio:** Cliente, Ordine, Prodotto;
3. **Business logic — servizi:** Conti e le interfacce dei gestori;
4. **Utilities:** java.sql, javax.swing e Java.util (global).



Codice 18: Architettura completa del sistema di gestione ordini.

Le slide costruiscono progressivamente questo stesso diagramma:

- la GUI Swing inserisce, cancella, modifica e lista clienti, ordini e prodotti;
- opera tramite **GestoreClienti**, **GestoreOrdini** e **GestoreProdotti**;
- **Ordine** e **Prodotto** usano **GestoreConti**, per esempio **dollaroToEuro()**;
- **Cliente**, **Ordine**, **Prodotto** e **Conti** usano **java.sql** per la persistenza;
- **GestoreProdotti** può offrire **Prodotto** **getProdotto(int idProdotto)**, ricostruendo un oggetto dai dati del database;
- la libreria **Java.util** è globale perché viene usata da tutti.

12.3 Deployment Diagram

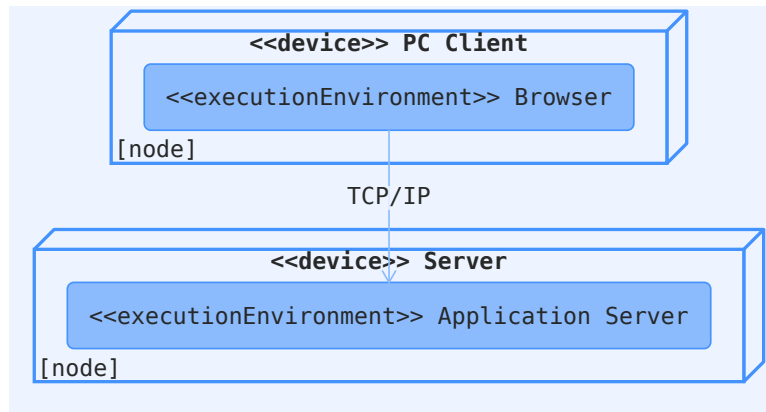
Il Deployment Diagram è una vista fortemente implementativa che mostra la relazione tra hardware e software. È strettamente collegato al Component Diagram.

12.3.1 Nodi e connessioni

Un nodo è una risorsa capace di ospitare software:

- un dispositivo fisico, per esempio PC o server;
- un ambiente software di esecuzione, come browser, macchina virtuale, application server o container Docker.

Una connessione tra nodi rappresenta un canale di comunicazione e può indicarne il protocollo, per esempio TCP/IP.

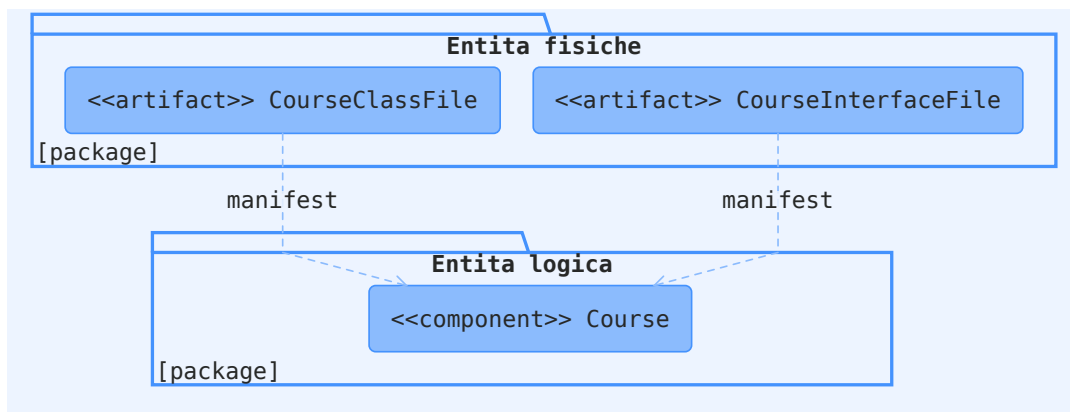


Codice 19: Dispositivo, ambiente di esecuzione e canale di comunicazione.

12.3.2 Artefatti e relazione manifest

Gli artefatti sono entità concrete: sorgenti, eseguibili, script, tabelle database e pagine HTML. Vengono dislocati sui nodi e possono dipendere da altri artefatti.

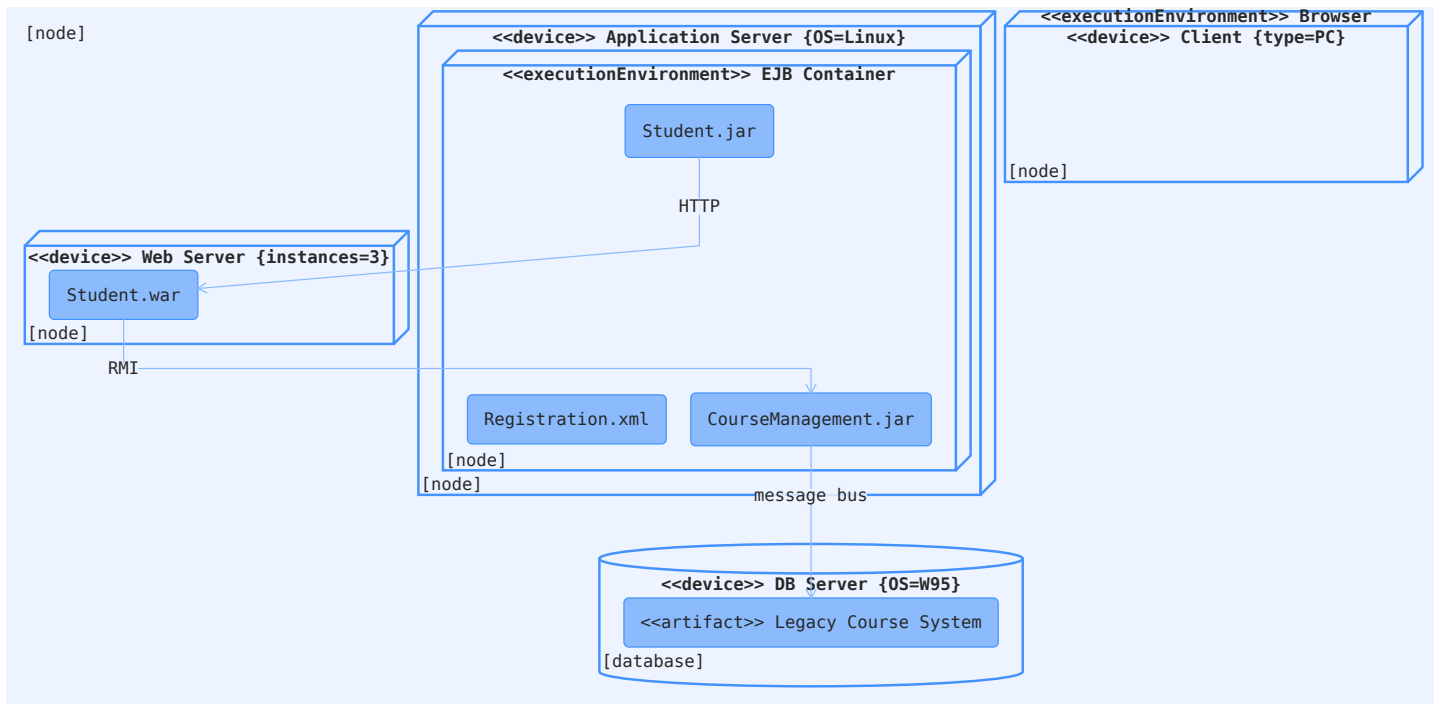
La relazione **manifest** non riguarda i manifest file: indica che un artefatto è la rappresentazione fisica di un componente logico. Per esempio, file distinti possono contenere il codice della classe e dell'interfaccia che costituiscono **Course**.



Codice 20: Due artefatti manifestano il componente logico Course.

12.3.3 Nodi annidati e tagged value

I nodi possono essere annidati: un browser viene eseguito in un PC; un EJB container in un application server. Tagged value come {type = PC}, {OS = Linux} o {instances = 3} aggiungono informazioni di deployment.



Codice 21: Architettura legacy con dispositivi, execution environment, istanze e artefatti.

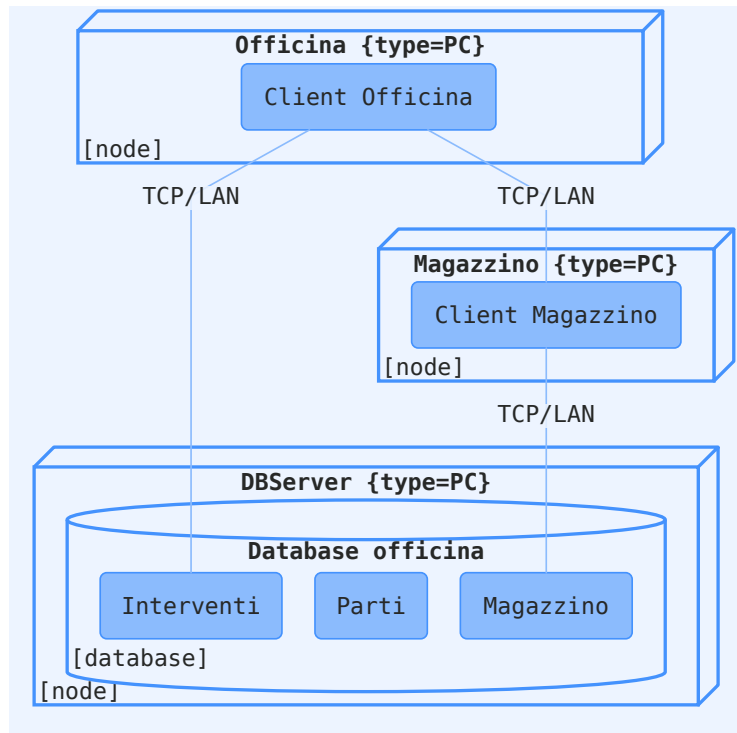
12.4 Esempio: officina meccanica

Si deve progettare l'architettura software e hardware di un sistema per officina. I meccanici scelgono un intervento, identificano operazioni e ricambi e inviano una richiesta. I magazzinieri controllano periodicamente le richieste, verificano la disponibilità, preparano e scaricano i ricambi e avvisano il meccanico; quando le scorte sono basse ordinano ai fornitori.

12.4.1 Requisiti degli utenti

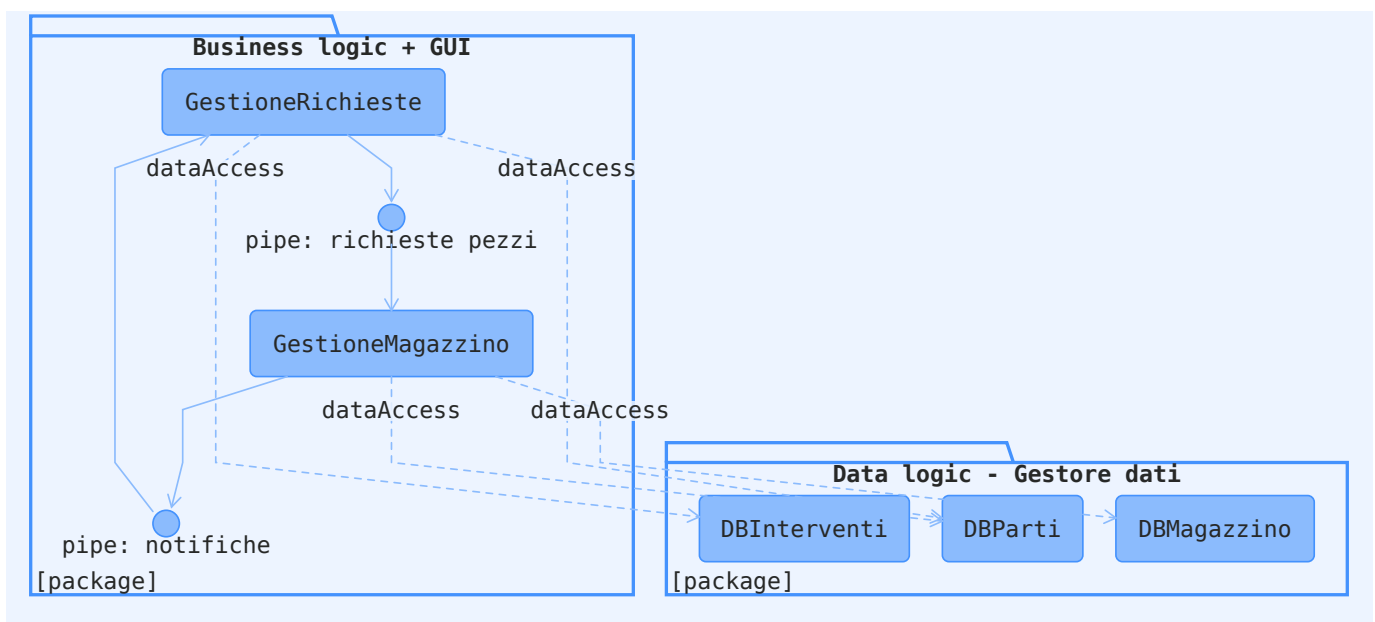
- **Meccanico:** richiesta dei pezzi necessari a ogni intervento.
- **Magazziniere:** carico e scarico, evasione richieste e riordino delle scorte.

12.4.2 Possibile architettura hardware



Codice 22: Tre personal computer in rete: officina, database e magazzino.

12.4.3 Possibile architettura software

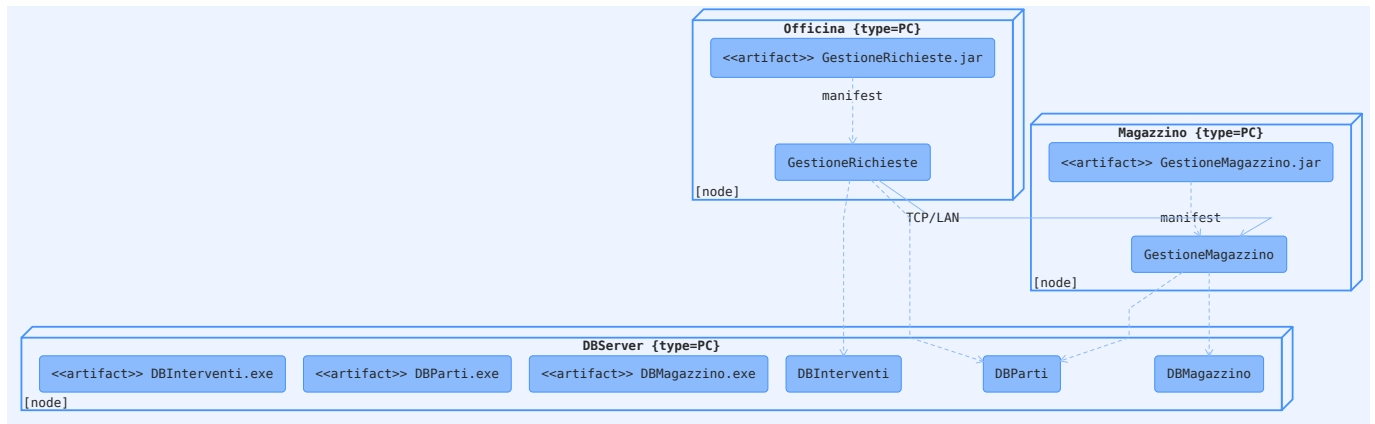


Codice 23: Component Diagram completo dell'officina meccanica.

Il diagramma riassume tutte le annotazioni progressive delle slide:

- `GestioneRichieste` contiene GUI e business logic dei meccanici;
- invia le richieste a `GestioneMagazzino` e riceve notifiche;
- `GestioneMagazzino` gestisce GUI, carico e scarico;
- i tre componenti dati persistono e recuperano interventi, ricambi e quantità.

12.4.4 Integrazione Component e Deployment



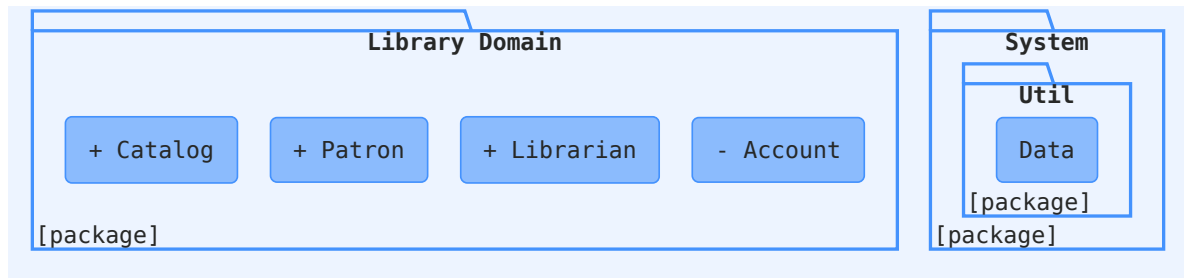
Codice 24: Componenti, artefatti e nodi riuniti nello stesso diagramma.

12.5 Package Diagram

12.5.1 Package e namespace

Un package UML raggruppa un numero arbitrario di elementi — classi, Use Case, modelli e sotto-package — creando gerarchie. È simile ai package dei linguaggi come Java, ma più generale.

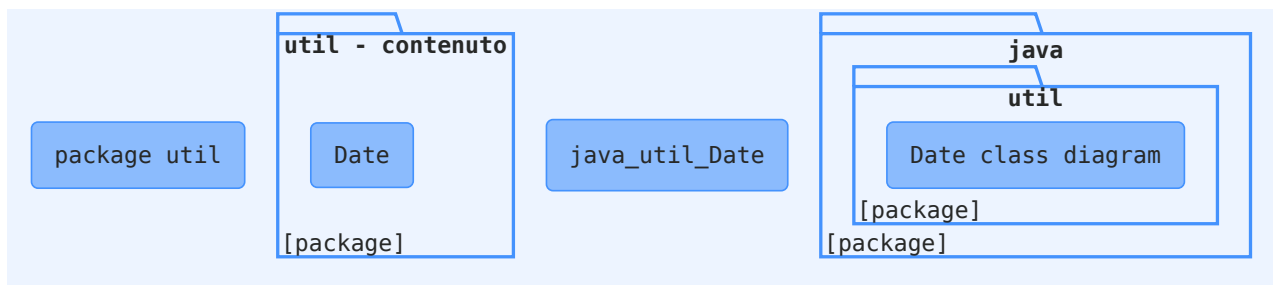
Ogni package definisce un **namespace** nel quale i nomi devono essere univoci. Un nome completamente qualificato elimina le ambiguità, per esempio `System::Data` o `MartinFowler::Util::Data`.



Codice 25: Package, sotto-package, elementi pubblici e privati.

12.5.2 Modi di rappresentazione e visibilità

Un package può essere mostrato vuoto, con l'elenco degli elementi oppure con un diagramma completo al suo interno. Sono equivalenti anche package annidati e nomi qualificati come `java::util::Date`.

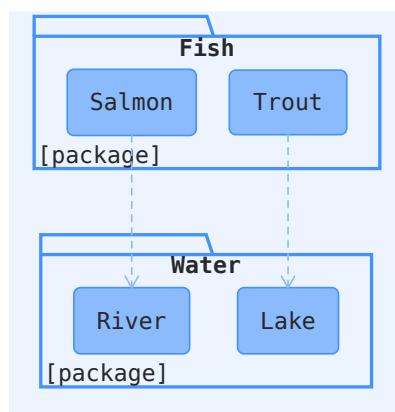


Codice 26: Package vuoto, contenuto elencato, gerarchia annidata e nome qualificato.

La visibilità degli elementi è + pubblica o - privata. Gli elementi pubblici costituiscono l'interfaccia del package verso i client.

12.5.3 Dipendenze tra package

Una dipendenza `Fish → Water` significa che almeno un elemento di `Fish` dipende da un elemento pubblico di `Water`. La dipendenza tra package riassume quindi quelle esistenti tra gli elementi contenuti.



Codice 27: Il package `Fish` dipende dal package `Water`.

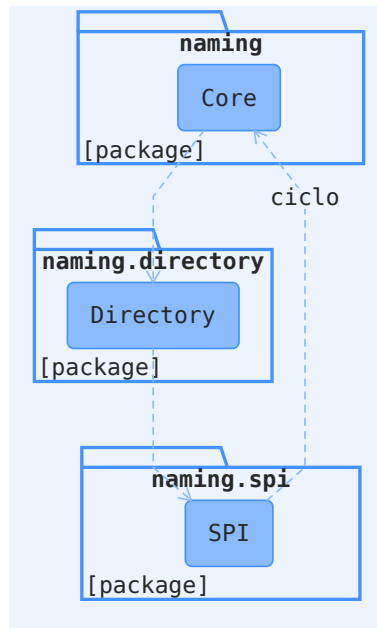
12.6 Progettare i package

Suddividere le classi è difficile e richiede compromessi ed esperienza. Le linee guida principali sono:

- **high cohesion e low coupling;**
- **Common Reuse Principle:** classi riusate insieme dovrebbero stare nello stesso package;
- **Acyclic Dependency Principle:** il grafo delle dipendenze non deve contenere cicli.

12.6.1 Cicli e dipendenze entranti

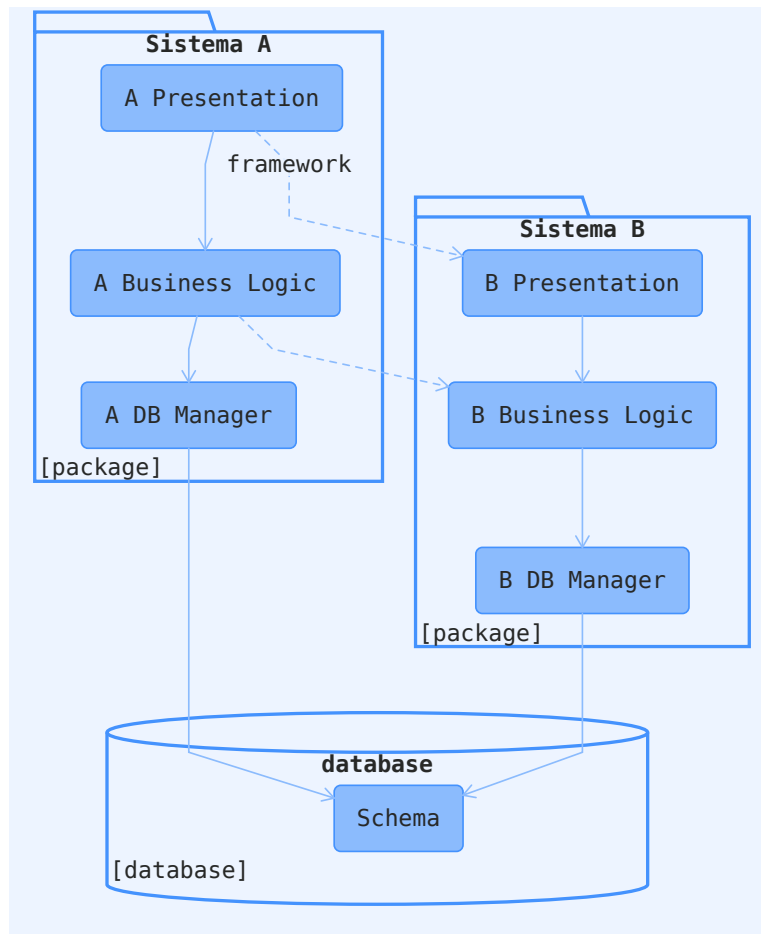
Un ciclo propaga le modifiche a cascata e impedisce una direzione stabile delle dipendenze. Anche un package con moltissime dipendenze entranti è rischioso: una sua modifica può impattare gran parte del sistema.



Codice 28: Ciclo di package contrario all'Acyclic Dependency Principle.

In un sistema medio-grande il Package Diagram è uno degli strumenti più utili per controllare la complessità strutturale del codice.

12.6.2 Esempio di architettura A/B



Codice 29: Due stack applicativi e dipendenze verso un database condiviso.

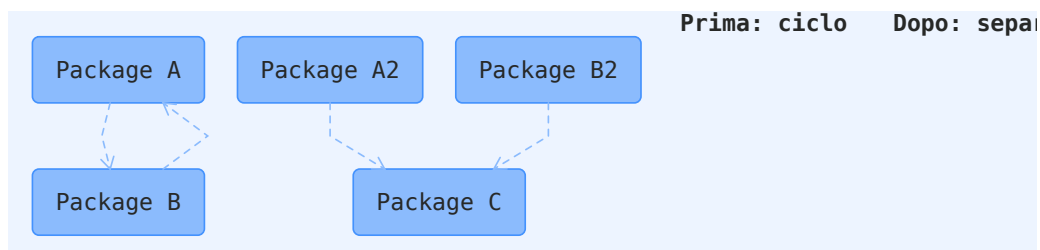
12.6.3 Procedura pratica

1. Progettare seguendo i principi di buona progettazione.
2. Generare il Package Diagram dal codice mediante reverse engineering, per esempio con Stan4J.
3. Identificare cicli, package poco coesi e nodi con molte dipendenze entranti.
4. Rifattorizzare e rigenerare il diagramma.

12.6.4 Risoluzione dei cicli

Le strategie mostrate nelle slide sono:

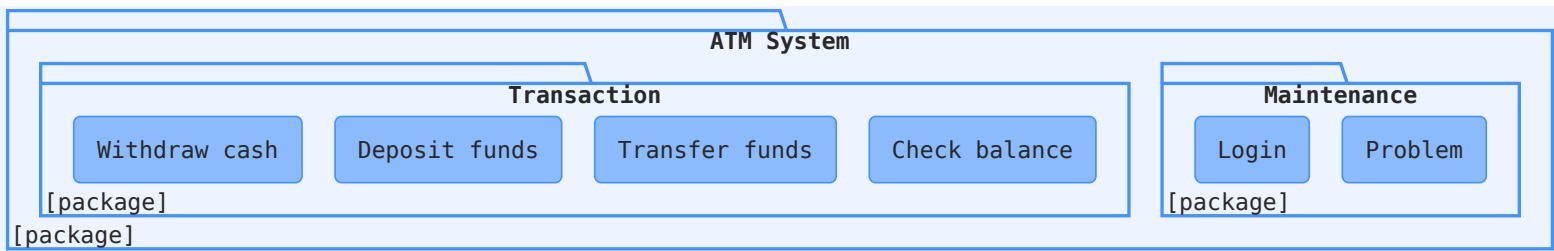
- eliminare una o più dipendenze, scegliendo se possibile il percorso più breve suggerito dallo strumento;
- fondere i package coinvolti;
- separare le entità responsabili del ciclo in un nuovo package condiviso.



Codice 30: Separazione delle entità condivise per eliminare il ciclo.

12.6.5 Non soltanto classi

I package possono raggruppare qualsiasi elemento UML. Nel caso di un ATM, Use Case come Withdraw cash, Deposit funds, Transfer funds e Check balance possono essere raccolti nel package Transaction, mentre Login e Problem appartengono a Maintenance.



Codice 31: Package Diagram applicato a gruppi di Use Case.

Riepilogo

Diagramma	Scopo	Momento tipico
Component	Architettura software.	Design architetturale.
Deployment	Relazione hardware–software.	Implementazione e deployment.
Package	Raggruppamento di elementi UML.	Dipende dagli elementi raggruppati.

13 Software Design Pattern

Un **design pattern** è una soluzione elegante, dotata di un nome, a un problema ricorrente di progettazione o programmazione orientata agli oggetti. Non è codice da copiare, ma una coppia problema–soluzione riutilizzabile: descrive ruoli, responsabilità e collaborazioni lasciando al progettista l'adattamento al contesto.

Idea fondamentale

Un pattern raccoglie esperienza progettuale consolidata. Non è necessariamente la soluzione che verrebbe in mente per prima, non dipende da un linguaggio specifico e costituisce un'unità di riuso a un livello più astratto del codice.

L'idea viene fatta risalire a Kent Beck e fu resa autorevole e popolare nel 1995 dal catalogo della **Gang of Four** — Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. Oggi esistono cataloghi per domini e tecnologie differenti, dalle applicazioni enterprise alla programmazione di videogiochi.

13.1 Progettazione, riuso e cataloghi

13.1.1 Perché usare i pattern

La progettazione non è un atto creativo privo di riferimenti: si alimenta di esperienza, sistemi simili, modelli di riferimento, principi, convenzioni, stili architetturali e pattern. Questi ultimi aiutano a:

- applicare principi di buona progettazione OO, come **alta coesione** e **basso accoppiamento**;
- creare progetti di qualità sia durante la modellazione sia durante la scrittura del codice;
- comunicare rapidamente una struttura complessa usando un vocabolario condiviso.

Dire «qui serve una Facade» comunica più di una lunga descrizione informale: il nome richiama immediatamente problema, struttura e conseguenze note.

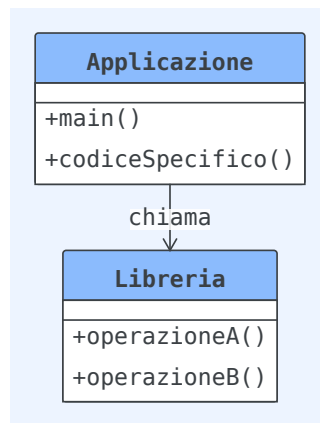
13.1.2 Il panorama del riuso

Il riuso è una pratica fondamentale, ma non riguarda soltanto i pattern. Può avvenire mediante:

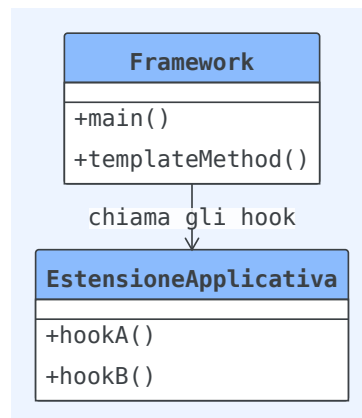
- **librerie** di funzioni o classi;
- componenti riusabili e COTS (**Commercial Off-The-Shelf**), acquistabili con interfaccia e implementazione;
- **framework** applicativi;
- famiglie o linee di prodotti con un nucleo comune;
- modelli di dominio, requisiti e design;
- sistemi legacy, generatori di programmi, servizi e software aspect-oriented.

13.1.3 Libreria e framework

Un **framework** è un insieme di classi e interfacce cooperanti che realizza un design riusabile e personalizzabile per uno specifico dominio applicativo. Esempi Java citati nelle slide sono Struts 2, Spring e Hibernate.



Codice 32: Con una libreria, il codice dell'applicazione conserva il controllo e chiama i servizi necessari.



Codice 33: In un framework il flusso di controllo è invertito: è il framework a chiamare il codice scritto dallo sviluppatore.

La libreria offre servizi che l'applicazione invoca; il framework possiede il **main** e chiama i punti di estensione implementati dall'applicazione. La personalizzazione avviene spesso creando sottoclassi specifiche: «riusa il corpo principale e scrivi il codice che esso chiamerà».

Un pattern è più astratto, piccolo e generale di un framework: non è utilizzabile **as is**, descrive pochi elementi architetturali e può essere applicato in domini e tecnologie differenti. Un framework è invece software concreto per una famiglia di applicazioni.

13.1.4 GRASP e pattern classici

I pattern **GRASP** (**General Responsibility Assignment Software Patterns**) guidano l'assegnazione delle responsabilità nel software e costituiscono il fondamento della progettazione OO. Fra essi rientrano Creator, Information Expert, Controller, Low Coupling, High Cohesion, Polymorphism, Pure Fabrication, Indirection e Protected Variations.

Il catalogo GoF comprende 23 pattern, suddivisi in tre categorie:

Creazionali	Strutturali	Comportamentali
Factory Method Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Flyweight Proxy	Interpreter Template Method Chain of Responsibility Command Iterator Mediator Memento

		Observer State Strategy Visitor
--	--	--

13.1.5 Come si descrive un pattern

Una scheda di pattern contiene almeno:

- **nome:** significativo e descrittivo, suggerisce lo scopo;
- **problema:** quando applicarlo e quali precondizioni devono valere;
- **soluzione:** uno schema astratto, non una ricetta completa, con elementi, relazioni e responsabilità;
- **conseguenze:** vantaggi, costi e compromessi prodotti dall'applicazione.

Pattern come schema

La soluzione non è mai completamente precisa: l'adattamento e le eventuali trasformazioni dei parametri restano responsabilità di chi implementa il pattern.

13.2 Abstract Factory

13.2.1 Dalla creazione alla fabbrica

I pattern creazionali controllano la creazione delle istanze. GRASP **Creator** assegna a una classe B la responsabilità di creare A quando B contiene, aggrega, registra, usa strettamente A o possiede i dati necessari a inizializzarla. In casi più complessi conviene però separare questa responsabilità.

Una **Factory** nasconde una logica di creazione potenzialmente complessa, la separa dalla logica applicativa e rende possibili strategie come caching, riciclaggio e gestione della memoria.

Abstract Factory fornisce un'interfaccia per creare **famiglie di prodotti correlati o dipendenti** senza specificarne nel client le classi concrete.

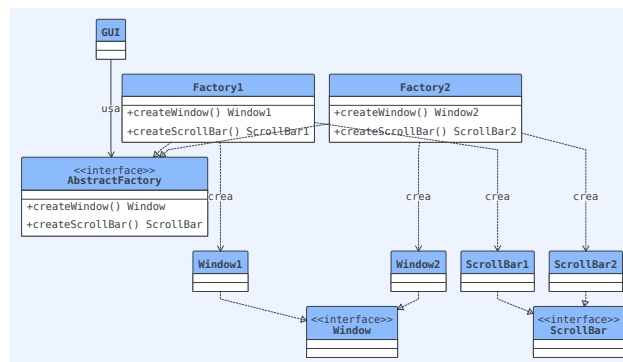
13.2.2 Quando usarla

È indicata quando:

- il sistema deve essere indipendente da come i prodotti vengono creati, composti e rappresentati;
- il sistema deve poter essere configurato con una fra più famiglie disponibili;
- i prodotti di una stessa famiglia devono essere usati insieme e il vincolo va garantito.

13.2.3 Esempio: GUI multiplatforma

Supponiamo di costruire una GUI per due piattaforme. La famiglia 1 contiene `Window1` e `ScrollBar1`; la famiglia 2 contiene `Window2` e `ScrollBar2`. La GUI non deve conoscere le classi concrete né accoppiare per errore `Window1` con `ScrollBar2`.



Codice 34: Una factory concreta crea sempre prodotti appartenenti alla medesima famiglia.

Senza fabbrica, il client incorpora la scelta della piattaforma:

```
Window w = new Window1();
ScrollBar s = new ScrollBar1();
```

Con la fabbrica astratta, la scelta è concentrata in una sola riga:

```
AbstractFactory f = new Factory1();
Window w = f.createWindow();
ScrollBar s = f.createScrollBar();
```

Il client chiede una famiglia e poi i prodotti; è la factory a garantirne l'accoppiamento corretto. La responsabilità della creazione viene disaccoppiata dalla responsabilità dell'uso.

13.2.4 Conseguenze

- isola le classi concrete: il client conosce soltanto astrazioni;
- rende semplice cambiare l'intera famiglia sostituendo una sola concrete factory;
- favorisce la consistenza tra prodotti correlati;

- rende invece costoso aggiungere un **nuovo tipo di prodotto**: occorre modificare l'interfaccia della factory e tutte le sue implementazioni.

13.2.5 Struttura d'implementazione

La factory astratta dichiara un'operazione per ciascun tipo di prodotto; ogni factory concreta costruisce una variante coerente di tutti quei tipi.

```
interface GUIFactory {
    Window createWindow();
    ScrollBar createScrollBar();
}

class PlatformFactory implements GUIFactory {
    Window createWindow() { return new Window1(); }
    ScrollBar createScrollBar() { return new ScrollBar1(); }
}

class Application {
    private final GUIFactory factory;
    Application(GUIFactory factory) { this.factory = factory; }

    void buildUI() {
        Window window = factory.createWindow();
        ScrollBar bar = factory.createScrollBar();
    }
}
```

Abstract Factory o Factory Method?

Factory Method delega alle sottoclassi la creazione di un prodotto attraverso una singola operazione sovrascrivibile. **Abstract Factory** espone invece più operazioni coordinate per creare un'intera famiglia. Una concrete factory può comunque implementare ciascuna operazione usando un Factory Method.

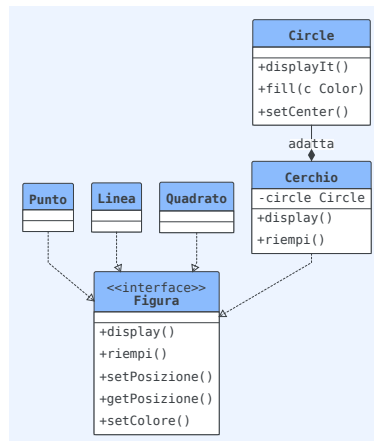
13.3 Adapter

13.3.1 Problema e scopo

Adapter converte l'interfaccia di una classe in un'altra interfaccia attesa dal client. Permette a classi già esistenti di collaborare anche quando le loro interfacce sono incompatibili, senza modificare il codice sorgente della classe adattata.

13.3.2 Esempio: aggiungere il cerchio

Un'applicazione gestisce figure mediante l'interfaccia **Figura**, che espone `display()` e `riempi()`. **Punto**, **Linea** e **Quadrato** la implementano già. Vogliamo riusare una classe esterna **Circle**, non modificabile, che offre invece `displayIt()`, `fill(Color)` e `setCenter()`.



Codice 35: **Cerchio** presenta al client l'interfaccia **Figura** e delega il lavoro a **Circle**.

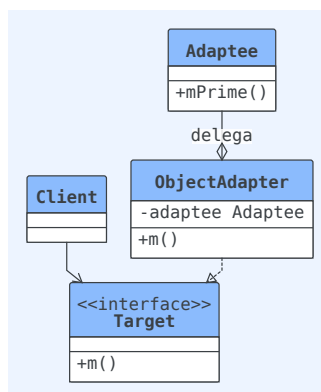
```
class Cerchio implements Figura {
    private Circle circle;

    void display() { circle.displayIt(); }
    void riempi() { circle.fill(colore); }
}
```

L'adapter non si limita necessariamente a rinominare le operazioni: può tradurre parametri, risultati e convenzioni.

13.3.3 Object Adapter e Class Adapter

L'**Object Adapter** usa composizione e delega; l'adapter possiede un riferimento all'adattato e sa come invocarlo. Il **Class Adapter** usa ereditarietà: eredita sia dall'interfaccia attesa sia dalla classe adattata e richiede quindi ereditarietà multipla o un linguaggio che consenta una combinazione equivalente tra classe e interfaccia.

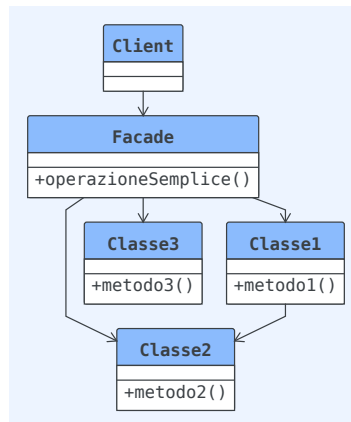


Codice 36: Struttura generale dell'Object Adapter: `m()` converte la richiesta e invoca `mPrime()`.

13.4 Facade

13.4.1 Un ingresso semplice a un sottosistema

Facade fornisce un'interfaccia unificata e più semplice a un insieme di interfacce complesse di un sottosistema. Il client chiama un solo metodo di alto livello, mentre la facciata coordina le classi interne necessarie.



Codice 37: La Facade nasconde la rete di collaborazioni necessaria a realizzare un compito.

13.4.2 Esempi delle slide

Per compilare un programma, un client potrebbe coordinare direttamente `LexicalAnalyzer`, `SyntaxAnalyzer`, `SemanticAnalyzer` e `CodeGenerator`. Una classe `Compiler` espone invece `compile()` e orchestra internamente tutte le fasi.

Analogamente, per spostare un oggetto un robot deve usare `Camera` per identificarlo, `Arm` per muoversi e `Pliers` per afferrarlo. La classe `Robot` può offrire una singola operazione `moveObject()`.

```
class Facade {
    void doSomething() {
        Class1 c1 = new Class1();
        Class2 c2 = new Class2();
        Class3 c3 = new Class3();
        c1.doStuff();
        c3.set(c1.getX());
        return c3.getZ();
    }
}
```

13.4.3 Conseguenze e confronto con Adapter

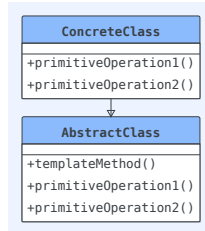
- promuove accoppiamento debole fra client e sottosistema;
- riduce le dipendenze e nasconde componenti complessi;
- non vieta al client esperto di usare direttamente le classi interne quando necessario.

Facade e Adapter sono entrambi **wrapper**, ma risolvono problemi differenti: Facade **semplifica** un'interfaccia o un insieme di interfacce; Adapter **converte** un'interfaccia incompatibile in quella richiesta.

13.5 Template Method

13.5.1 Scheletro e passi variabili

Template Method definisce lo scheletro di un algoritmo in un metodo, rimandando la definizione di alcuni passi alle sottoclassi. La parte invariante viene implementata una sola volta; le parti variabili vengono ridefinite senza alterare la sequenza generale.



Codice 38: `templateMethod()` chiama nell'ordine previsto le operazioni primitive ridefinite dalla sottoclasse.

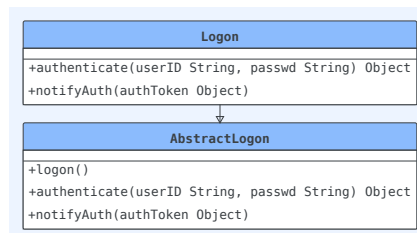
```
abstract class AbstractClass {
    final void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
    }
    protected abstract void primitiveOperation1();
    protected abstract void primitiveOperation2();
}
```

13.5.2 Esempio: login riusabile

Il login dell'applicazione segue cinque passi:

1. richiede identificativo e password;
2. autentica l'utente e produce un oggetto con le informazioni ottenute;
3. mostra un'animazione durante l'autenticazione;
4. notifica il completamento e rende disponibile il risultato;
5. esegue eventuale logica dipendente dall'applicazione.

I passi 1, 3 e 4 sono invarianti; autenticazione e notifica applicativa possono cambiare.



Codice 39: `AbstractLogon.logon()` contiene la sequenza comune; `Logon` fornisce soltanto i passi variabili.

13.5.3 Inversione di controllo e conseguenze

Template Method realizza il cosiddetto **Hollywood principle**: «non chiamarci, ti chiameremo noi». Normalmente una sottoclasse chiama metodi ereditati; qui è il metodo della superclasse a richiamare operazioni ridefinite nelle sottoclassi.

- è una tecnica fondamentale di riuso, molto usata in librerie e framework;
- realizza inversione del flusso di controllo;
- consente molte sottoclassi concrete, ciascuna con passi variabili differenti;
- richiede di distinguere chiaramente operazioni obbligatorie e **hook** opzionali.

13.6 Observer

13.6.1 Problema: mantenere viste consistenti

Vogliamo associare più viste o osservatori a un modello e mantenerli coerenti quando il suo stato cambia. Rendere pubblico lo stato viola l'incapsulamento; interrogare continuamente l'oggetto con polling fa scoprire tardi le variazioni, spreca risorse e scala male con molti osservatori.

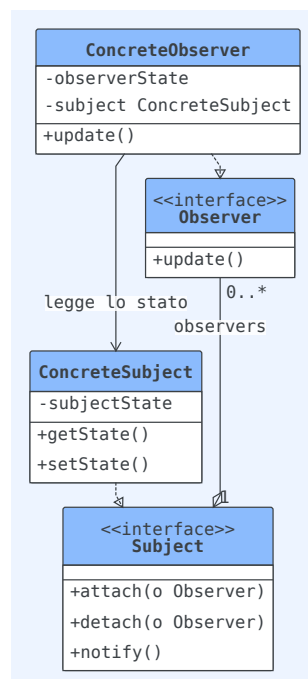
Observer definisce una dipendenza lasca uno-a-molti: quando il **Subject** cambia stato, tutti gli oggetti dipendenti vengono avvertiti e possono aggiornarsi.

Alla base di MVC

In Model–View–Controller, tabelle, grafici e altre viste osservano lo stesso modello. La notifica garantisce che rappresentazioni differenti rimangano consistenti con i dati.

13.6.2 Registrazione e notifica

Gli osservatori si registrano dinamicamente presso l'osservato mediante `attach()` e possono rimuoversi con `detach()`. Quando cambia stato, il Subject esegue `notify()`, che invoca `update()` su tutti i registrati. Ogni Observer può non fare nulla oppure chiedere lo stato corrente con `getState()`.



Codice 40: Struttura del pattern Observer: il Subject conosce una collezione di Observer soltanto attraverso l'interfaccia.

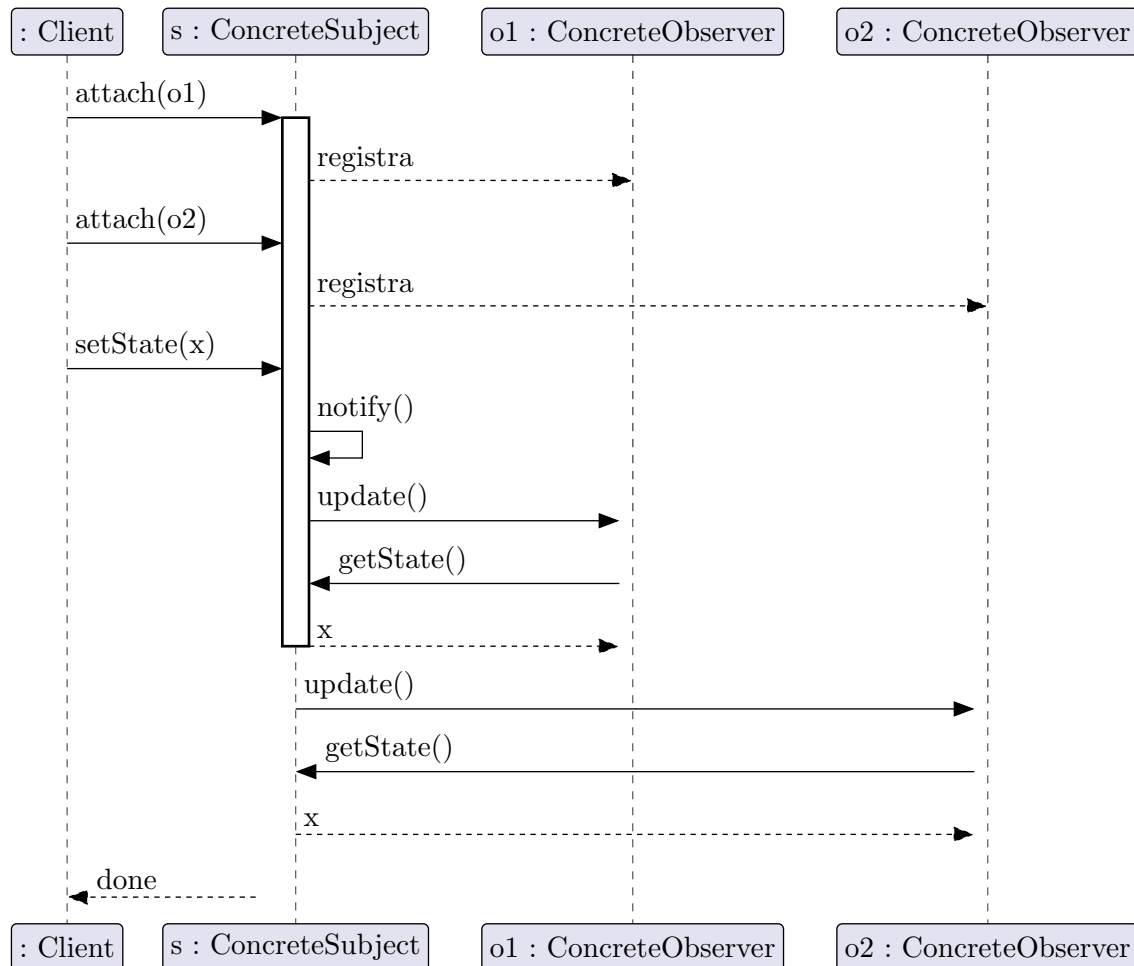


Figura 69: Sequence Diagram completo: registrazione, cambiamento di stato, broadcast e recupero dello stato.

13.6.3 Conseguenze

- accoppiamento lasco: il Subject sa che esiste una lista dinamica di Observer, ma non ne conosce la struttura;
- Observer e Subject possono appartenere a strati di astrazione differenti;
- supporto naturale alla comunicazione broadcast e al modello publish/subscribe;
- una modifica innocente può provocare una cascata di aggiornamenti, anche costosi o inattesi.

13.7 State

13.7.1 Comportamento dipendente dallo stato

State consente a un oggetto di cambiare comportamento a runtime al variare del proprio stato interno. È un modo pulito per implementare macchine a stati in linguaggi che non le supportano nativamente: la rappresentazione dello stato viene estratta in una gerarchia e il polimorfismo seleziona il comportamento.

13.7.2 Esempio: distributore automatico

La vending machine attraversa tre stati:

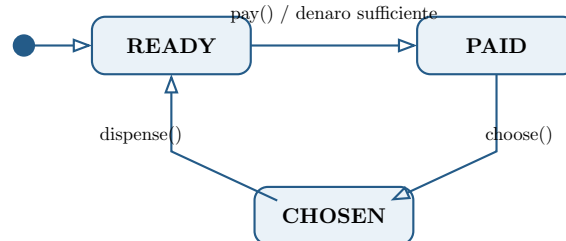


Figura 70: Macchina a stati del distributore: pagamento, scelta ed erogazione.

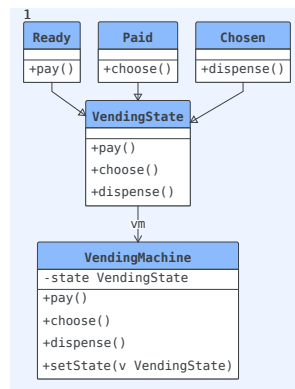
Con una soluzione diretta, `VendingMachine` conserva un campo `state` e ogni evento contiene un `if` o uno `switch`:

```
void pay(Money money) {
    switch (state) {
        case READY:
            totale += money;
            if (totale >= prezzo) {
                msg("money ok");
                state = PAID;
            } else {
                msg("insert more money");
            }
            break;
        default:
            msg("Operation not possible");
    }
}
```

Questa strategia genera molti controlli distribuiti. Aggiungere uno stato richiede modifiche in numerosi metodi — tipicamente un ramo per ogni evento — e il codice diventa difficile da comprendere e mantenere.

13.7.3 Implementazione con il pattern

Ogni stato diventa una sottoclasse di `VendingState`; ogni sottoclasse ridefinisce le operazioni significative in quello stato. Le implementazioni ereditate delle transizioni non ammesse producono invece un errore. Il contesto delega tutti gli eventi allo stato corrente.



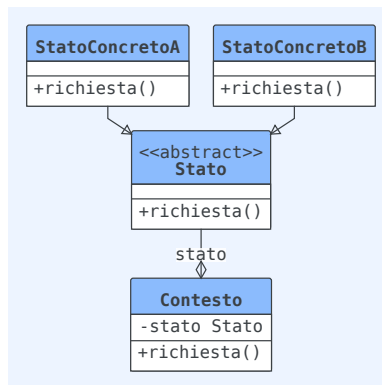
Codice 41: Le classi Ready, Paid e Chosen incapsulano le sole operazioni valide nei rispettivi stati.

```

class VendingMachine {
    private VendingState state;
    void pay(Money m) { state.pay(m); }
    void choose(Product p) { state.choose(p); }
    void dispense() { state.dispense(); }
    void setState(VendingState v) { state = v; }
}

class Ready extends VendingState {
    void pay(Money m) {
        // aggiorna il totale e comunica l'esito
        vm.setState(new Paid(vm));
    }
}
  
```

13.7.4 Struttura generale e conseguenze



Codice 42: Contexto.richiesta() delega a stato.richiesta(); il dispatch dinamico sceglie il comportamento.

Se richiesta() è astratta, ogni stato concreto deve implementare anche gli eventi che rappresentano transizioni non ammesse, normalmente sollevando un errore.

- incapsula comportamento e transizioni in oggetti, eliminando un grande switch;
- il comportamento di ciascuno stato dipende da una sola classe;
- semplifica la modifica di uno stato e l'aggiunta di nuovi stati;
- aumenta il numero complessivo delle classi.

13.8 Quadro riepilogativo

Pattern	Problema risolto	Idea della soluzione
Abstract Factory	Creare famiglie coerenti senza dipendere dalle classi concrete.	Delegare la creazione a factory intercambiabili.
Adapter	Far collaborare interfacce incompatibili.	Convertire e delegare mediante un wrapper.
Facade	Ridurre la complessità percepita di un sottosistema.	Offrire un unico ingresso di alto livello.
Template Method	Riutilizzare un algoritmo con alcuni passi variabili.	Fissare lo scheletro e ridefinire operazioni primitive.
Observer	Sincronizzare molti dipendenti con un oggetto.	Registrazione e notifica broadcast lasca.
State	Cambiare comportamento al cambiare dello stato.	Rappresentare gli stati come oggetti polimorfici.

Lettura trasversale

Abstract Factory controlla la **creazione**; Adapter e Facade riorganizzano la **struttura**; Template Method, Observer e State governano il **comportamento**. In ogni caso il valore del pattern non è il diagramma in sé, ma la distribuzione consapevole delle responsabilità e la conoscenza dei compromessi introdotti.

14 Software Testing

Il **software testing** è una procedura sistematica che esegue un sistema software con l'intento di trovare comportamenti anomali. Non può dimostrare in generale l'assenza di difetti: seleziona una parte del dominio degli input e confronta i risultati osservati con quelli attesi.

Obiettivo del capitolo

Distinguere errore, fault e failure; comprendere testing e debugging; scegliere tra approcci black-box e white-box; riconoscere i livelli di test; automatizzare casi di test Java con JUnit 5.

14.1 Terminologia attraverso un caso reale

14.1.1 La storia di InvestireOK

Un trader della società **InvestireOK** usa la nuova funzionalità “acquisto ciclico facile” di **EasyGuadagno 2.1** per comprare futures EuroStoxx. Durante l'operazione osserva un comportamento scorretto — un **failure** — che degenera nel crash del sistema.

Il trader contatta l'help desk, che segue un protocollo interno:

1. ascolta e annota il problema;
2. spiega una procedura di **recovery** per riavviare il sistema;
3. apre un ticket con priorità massima nel bug tracking system;
4. avverte project manager e senior software developer.

Recovery non significa correzione

Il recovery ripristina il servizio, ma il fault che ha prodotto il failure rimane nel sistema. Il trader può quindi non essere ancora in grado di usare la funzionalità coinvolta.

14.1.2 Debugging del “guru”

Lo sviluppatore esperto parte dalla descrizione e dagli stessi input del trader per riprodurre il failure. Grazie alla conoscenza del sistema individua dapprima il file o componente e poi la linea difettosa: è la **fault localization**. Rimuove il bug mediante una patch: è la **fault removal** o **bug fix**.

La patch passa quindi al senior software tester, che riesegue la testsuite per verificare che la modifica non abbia danneggiato altre funzionalità. È **testing di regressione**. Se tutto è corretto, il ticket viene chiuso, patch e documentazione entrano nel repository della futura versione 2.2 e l'assistenza distribuisce la correzione al cliente.

Il termine **bug** è precedente all'informatica, ma un episodio celebre risale al 1947: Grace Hopper e il suo gruppo trovarono una falena nei relè del computer Mark II e la conservarono nel registro come il primo caso “reale” di insetto trovato.

14.1.3 Quando l'esperto non è disponibile

Se il guru è in vacanza, il project manager nomina un sostituto che non conosce il sistema. Il vice-guru riesce a riprodurre il failure, ma decine di file e breakpoint non gli consentono di localizzare il difetto.

Può allora usare uno strumento di **fault localization** che analizza la descrizione e ordina i file sospetti mediante tecniche NLP o LLM. Non corregge il programma, ma restringe lo spazio di ricerca e facilita il bug fix.

14.2 Errore, fault e failure

14.2.1 La catena causale

- **Errore umano**: azione o decisione sbagliata dello sviluppatore, per esempio un refuso o un fraintendimento concettuale.
- **Fault, defect o bug**: manifestazione statica dell'errore nel software, come un'istruzione scorretta.
- **Failure**: comportamento osservabile del sistema che diverge da quello atteso durante l'esecuzione.

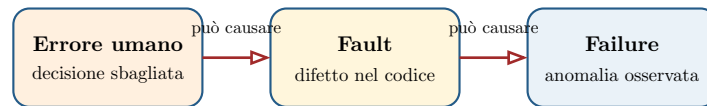


Figura 71: Catena causale: un errore può introdurre un fault, che in determinate condizioni può produrre un failure.

14.2.2 Esempio: reDouble

La funzione dovrebbe raddoppiare x , ma usa x al posto della costante 2:

```
int reDouble(int x) {  
    int y;  
    y = x * x;          // fault: dovrebbe essere 2 * x  
    return y;  
}
```

L'errore umano può essere un refuso oppure l'interpretazione errata di “raddoppiare”. Il fault è nell'assegnamento. Chiamando `reDouble(3)` si osserva il failure: il risultato è 9 anziché 6.

Il fault può restare latente

Un fault può sopravvivere a lungo nel programma. Diventa osservabile soltanto quando un input esegue — e “sollecita” nelle condizioni opportune — l'istruzione che lo contiene. Non ogni esecuzione di codice difettoso produce necessariamente un failure visibile.

14.2.3 Causa indiretta

Il crash può dipendere indirettamente da una quantità insufficiente di testing o dall'applicazione non sistematica delle tecniche appropriate. Il testing è difficile, richiede competenze per scegliere input efficaci, è costoso e spesso arriva vicino al rilascio, quando tempo e risorse sono già esauriti.

14.3 Testing e debugging

14.3.1 System Under Test e oracolo

Il software testing esegue il **System Under Test** (SUT) con input scelti da un sottoinsieme del dominio e verifica se l'output ottenuto coincide con quello atteso. Il meccanismo che stabilisce il risultato corretto è detto, in generale, **oracolo di test**.

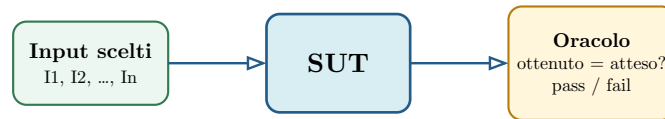


Figura 72: Anatomia essenziale di un test: input, esecuzione del SUT e confronto con il risultato atteso.

14.3.2 Differenza dal debugging

Il testing **rivela failure**, cioè comportamenti anomali. Il debugging parte da un failure e cerca di trovare e rimuovere il fault associato. Comprende due attività:

1. **fault localization**: individuare componente, file e linea responsabili;
2. **fault removal**: modificare il software per eliminare il difetto.

Un test fallito è quindi un punto di partenza diagnostico, non contiene necessariamente la localizzazione del bug.

14.4 Perché il testing è difficile

14.4.1 L'impossibilità del testing esaustivo

Per sommare due interi a 32 bit esistono tutte le combinazioni di due sequenze da 32 bit:

$$2^{32} \cdot 2^{32} = 2^{64} = 18.446.744.073.709.551.616$$

Con una sola esecuzione per millisecondo servirebbero circa **584.942.417 anni**. E questo esempio elementare trascura sequenze di operazioni, stato, concorrenza, ambiente e configurazioni.

Conseguenza

Nei sistemi realistici il testing esaustivo non è realizzabile. La difficoltà centrale consiste nel selezionare “buoni” input: pochi casi con alta probabilità di rivelare failure e fault.

14.4.2 White-box e black-box

Aspetto	White-box / strutturale	Black-box / funzionale
Conoscenza	Usa esplicitamente codice e struttura interna del SUT.	Considera il SUT come una scatola nera.
Origine dei test	Flusso di controllo, istruzioni, rami, condizioni.	Requisiti, specifiche e comportamento atteso.
Esempio	Statement coverage: eseguire ogni istruzione almeno una volta.	Casi derivati dalle funzioni richieste e dai domini degli input.
Punto di forza	Misura ciò che è stato esercitato nel codice.	Verifica il prodotto dal punto di vista esterno.

Nessuno dei due approcci è universalmente migliore: efficacia e costo dipendono da SUT, tecnica e competenza del tester. Sono complementari. Le slide citano lo studio di Basili e Selby, **Comparing the Effectiveness of Software Testing Strategies** (IEEE TSE, 1987), secondo il quale tester esperti trovarono più difetti con il testing funzionale nel contesto sperimentale considerato; il risultato non trasforma però il black-box in una scelta sempre superiore.

14.5 Test case e testsuite

14.5.1 Definizioni

Un **test case** comprende input, precondizioni di esecuzione e risultati attesi costruiti per uno scopo specifico, come esercitare un cammino o verificare un requisito. Una **testsuite** è una collezione organizzata di casi di test.

La definizione va istanziata nel contesto: un input può essere un valore, un file, una configurazione o un'intera sequenza di azioni.

14.5.2 Due esempi

Test	Input e precondizioni	Output atteso
Desktop Sort	C, 12, -29, 32, con C = ordine crescente.	-29, 12, 32.
Web app “Carrello”	Login; apertura pagina acquisti; selezione di X; aggiunta al carrello.	Il prodotto X compare nel carrello.

Nel primo caso l'input è quasi soltanto un insieme di dati. Nel secondo è una sequenza di passi che prepara lo stato dell'applicazione, fornisce l'input e rende osservabile l'esito.

14.6 Livelli di testing

14.6.1 Unità

Durante l'implementazione, lo sviluppatore testa una singola unità — funzione, procedura, classe o componente — isolandola il più possibile dal resto. Stub e mock object sostituiscono collaboratori reali e controllano le dipendenze. Il testing di unità usa spesso tecniche white-box.

14.6.2 Integrazione

Quando i moduli vengono assemblati, il testing di integrazione verifica come comunicano e quali informazioni si scambiano. Le interfacce sono fonti frequenti di difetti:

- ordine o tipo errato dei parametri;
- precondizioni non rispettate;
- formati o protocolli di comunicazione violati;
- assunzioni incompatibili tra componenti.

14.6.3 Sistema e accettazione

Sul sistema integrato, il testing group verifica i requisiti e i bisogni dell'utente. L'**acceptance testing** stabilisce se il prodotto è accettabile per cliente o utenti. L'approccio è di solito black-box e considera non soltanto correttezza funzionale, ma anche prestazioni, usabilità, sicurezza e altri requisiti di qualità.

14.6.4 Regressione

Ogni modifica può generare effetti collaterali anche in zone apparentemente non coinvolte. Il testing di regressione riesegue test già disponibili per verificare che funzionalità precedentemente corrette non siano regredite. Non è confinato a una sola fase: accompagna correzioni, evoluzione e manutenzione.

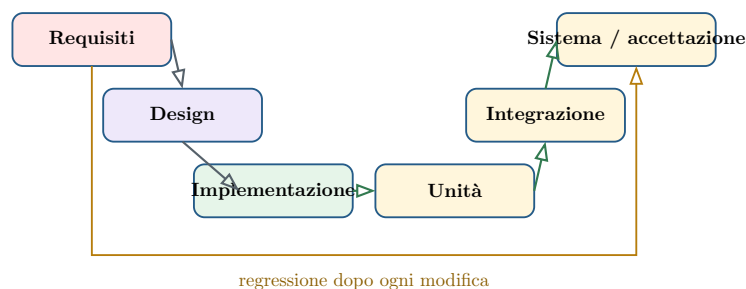


Figura 73: Relazione concettuale tra fasi di sviluppo e livelli di test; la regressione attraversa l'intero ciclo evolutivo.

14.7 Automazione e JUnit 5

14.7.1 Dal test manuale al test eseguibile

Nel testing automatizzato i tester scrivono **test script**, cioè codice eseguito da un framework che prepara il contesto, invoca il SUT, valuta gli esiti e riporta **pass** o **fail**. L'automazione rende ripetibile la verifica e rende economicamente sostenibile il testing di regressione.

JUnit è un framework per il testing di programmi Java, sviluppato originariamente da Erich Gamma e Kent Beck. Usa classi, convenzioni e annotazioni come `@Test`; gli IDE possono eseguirlo mediante integrazioni grafiche.

Con JUnit è possibile:

1. scrivere test class e test method in Java;
2. eseguire i casi e osservare l'esito;
3. raggrupparli in testsuite.

Una convenzione utile associa a ogni classe di produzione una classe di test, per esempio `Stack` e `StackTest`.

14.7.2 Perché non usare main

Un `main()` con stampe richiede che una persona conosca l'output corretto e lo confronti manualmente:

```
public static void main(String[] args) {
    Stack<Integer> stack = new Stack<>();
    if (!stack.isEmpty())
        System.out.println("Stack should be empty!");

    stack.push(10);
    stack.push(-4);
    System.out.println("Last element: " + stack.pop());
    System.out.println("First element: " + stack.pop());
}
```

Un test JUnit documenta invece l'oracolo mediante asserzioni eseguibili.

14.8 Primo caso di test JUnit

14.8.1 Classe e metodo di test

Il wizard dell'IDE può generare lo scheletro, ma gli elementi essenziali sono gli import statici delle asserzioni, l'annotazione `@Test`, la test class e uno o più test method.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
```

```
class StackTest {
    @Test
    void testStack() {
        Stack<Integer> stack = new Stack<>();
        assertTrue(stack.isEmpty());

        stack.push(10);
        assertFalse(stack.isEmpty());
        stack.push(-4);

        assertEquals(-4, stack.pop());
        assertEquals(10, stack.pop());
    }
}
```

`assertEquals(expected, actual)` chiarisce quale valore sia previsto e quale venga prodotto dal SUT. Un messaggio opzionale, valutato in caso di fallimento, aggiunge contesto:

```
assertTrue(stack.isEmpty(), "lo stack deve essere vuoto");
```

14.8.2 Esecuzione: verde e rosso

Il test runner esegue una test class o una testsuite. Se tutte le asserzioni sono vere il caso passa, tradizionalmente con indicatore verde. Se `pop()` restituisce sempre `-1`, un confronto fallisce e il runner mostra `expected, actual, test` e linea coinvolta con indicatore rosso.

14.8.3 Failure ed error nel test runner

- **Failure del test:** un'asserzione non è soddisfatta, per esempio `expected <-3> but was <-4>`.
- **Error:** il codice testato termina in modo anomalo, per esempio con `ArrayIndexOutOfBoundsException`.

In entrambi i casi il test non passa, ma la distinzione aiuta la diagnosi: nel primo il SUT restituisce un risultato osservabile scorretto, nel secondo l'esecuzione viene interrotta da un problema runtime.

14.9 Progettare test coesi

14.9.1 Una cosa alla volta

Il primo `testStack()` verifica contemporaneamente stato vuoto e operazioni LIFO. È poco coeso: quando fallisce, il nome del test non identifica con precisione la responsabilità violata. È preferibile separare gli obiettivi.

```
class StackTest {
    @Test
    void newStackIsEmpty() {
        Stack<Integer> stack = new Stack<>();
        assertTrue(stack.isEmpty());
        stack.push(10);
        assertFalse(stack.isEmpty());
    }

    @Test
    void popUsesLifoOrder() {
        Stack<Integer> stack = new Stack<>();
        stack.push(10);
        stack.push(-4);
        assertEquals(-4, stack.pop());
        assertEquals(10, stack.pop());
    }
}
```

Separation of concerns nei test

Ogni test dovrebbe esprimere un comportamento preciso. Nomi descrittivi, poche cause di fallimento e struttura Arrange–Act–Assert rendono la suite più leggibile e diagnostica.

14.9.2 Fixture: `@BeforeEach` e `@AfterEach`

La duplicazione dell'inizializzazione si elimina con una **test fixture**. `@BeforeEach` viene eseguito prima di ogni test method e ricrea uno stato noto; `@AfterEach` viene eseguito dopo ogni test e rilascia risorse. Ogni test deve rimanere indipendente dagli altri.

```
class StackTest {
    private Stack<Integer> stack;
    @BeforeEach
    void setUp() {
        stack = new Stack<>();
    }
    @Test
    void newStackIsEmpty() {
        assertTrue(stack.isEmpty());
        stack.push(10);
        assertFalse(stack.isEmpty());
    }
    @Test
    void popUsesLifoOrder() {
        stack.push(10);
        stack.push(-4);
        assertEquals(-4, stack.pop());
        assertEquals(10, stack.pop());
    }
}
```

La stessa idea vale per oggetti più ricchi: una fixture di `ShoppingCart` può creare il carrello e due libri e aggiungerli prima di ogni test.

14.9.3 Ciclo di vita completo

- @BeforeEach e @AfterEach: prima e dopo **ogni** test method;
- @BeforeAll e @AfterAll: una sola volta prima e dopo **tutti** i test della classe.

In una classe con tre metodi, il primo modello produce `setUp` → `test1` → `tearDown` per tre volte; il secondo produce `setUpAll` → `test1` → `test2` → `test3` → `tearDownAll`. Le fixture condivise a livello di classe richiedono cautela perché lo stato mutabile può rendere i test dipendenti dall'ordine.

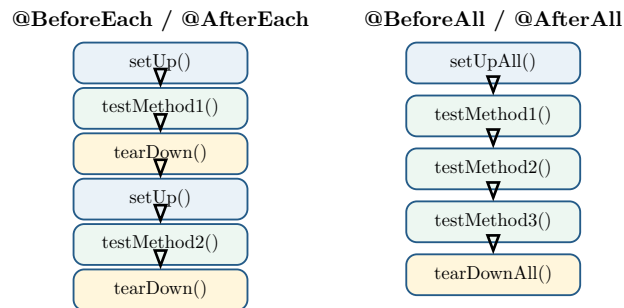


Figura 74: Differenza fra fixture ricreata per ogni test e fixture condivisa per l'intera classe.

14.10 Asserzioni JUnit

14.10.1 Esecuzione e arresto

Le asserzioni sono affermazioni verificabili. Se un'asserzione è vera, il metodo continua; se è falsa, il test corrente fallisce immediatamente, la parte restante del metodo viene saltata e il messaggio opzionale viene riportato. Il caso passa soltanto se tutte le asserzioni eseguite risultano vere.

14.10.2 Operazioni principali

Asserzione	Significato
<code>assertTrue(cond)</code> / <code>assertFalse(cond)</code>	Verifica una condizione booleana.
<code>assertEquals(expected, actual)</code>	Confronta primitivi o oggetti; per gli oggetti usa <code>equals</code> .
<code>assertArrayEquals(expected, actual)</code>	Confronta il contenuto degli array, non il loro riferimento.
<code>assertEquals(expected, actual, delta)</code>	Confronta <code>float</code> o <code>double</code> usando una tolleranza.
<code>assertSame(expected, actual)</code>	Verifica che i riferimenti indichino lo stesso oggetto.
<code>assertNotSame(expected, actual)</code>	Verifica che i riferimenti siano differenti.
<code>fail(message)</code>	Fa fallire immediatamente il test.

Il messaggio opzionale va posto come ultimo argomento in JUnit 5. Per i numeri floating-point è essenziale indicare una tolleranza coerente con il dominio.

14.11 Test parametrici

Un test parametrico esegue lo stesso comportamento con più input, riducendo duplicazione e rendendo esplicito l'insieme dei casi.

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;

class WordTest {
    @ParameterizedTest
    @ValueSource(strings = {"Hello", "JUnit"})
    void suppliedWordsAreNotNull(String word) {
        assertNotNull(word);
    }
}
```

Il runner crea due invocazioni distinte, una per "Hello" e una per "JUnit", e ne mostra separatamente l'esito.

14.12 Quadro riepilogativo

Concetto	Idea da ricordare
Errore–fault–failure	Una decisione umana può introdurre un difetto che, con input opportuni, produce un'anomalia osservabile.
Testing	Esegue il SUT per rivelare failure confrontando risultato ottenuto e atteso.
Debugging	Localizza e rimuove il fault che ha causato un failure.
White/black box	Struttura interna e specifiche esterne forniscono prospettive complementari.
Livelli	Unità, integrazione, sistema/accettazione e regressione rispondono a rischi differenti.
JUnit	Trasforma l'oracolo in asserzioni eseguibili e rende i test ripetibili.
Buon test	È focalizzato, indipendente, leggibile e prepara uno stato noto.

Collocazione nel corso

Il testing completa il percorso che parte dal processo di sviluppo, attraversa requisiti, design, UML e pattern e arriva alla verifica del software. Nella valutazione del corso, i concetti teorici accompagnano il progetto e possono essere richiesti sia nelle domande preliminari sia nella discussione orale.